APPASERVER Data Structure

November 26, 2021

SECURITY

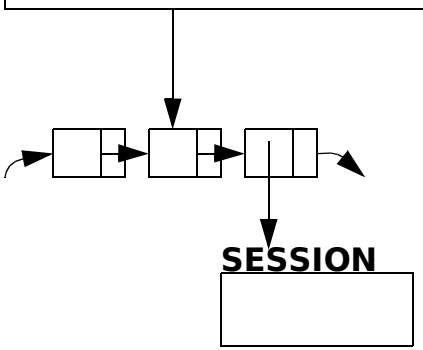
```
enum password_function password_function;{ no_encryption
                                          old_password_function
                                          regular_password_function
                                          sha2_function

enum password_function
    security_database_password_function(
        char *database_password );

char *security_encrypted_password(
    char *password_sql_injection_escape,
    enum password_function password_function );

char *security_mysql_version( void );

enum password_function
    security_mysql_version_password_function(
        security_mysql_version() );

char *security_encryption_select_clause(
    enum password_function password_function,
    char *password );

boolean security_password_match(
    database_password,
    security_encrypted_password(
        security_sql_injection_escape(
            typed_in_password ),
        security_database_password_function() );

boolean security_password_encrypted(
    char *database_password );

char *security_replace_special_characters(
    char *data );

char *security_sql_injection_escape(
    char *data )
{
    char destination[ STRING_WHERE_BUFFER ];

    return strdup(
        string_escape_character_array(
            destination,
            data,
            “‘’$;%=_” ) );
}

boolean security_system_role(
    char *role_name );
```

SECURITY_ENTITY

```
/* Usage */
SECURITY_ENTITY *security_entity_new(
    char *login_name,
    boolean folder->non_owner_forbid,
    boolean role->override_row_restrictions );

/* Input */
char *login_name;
boolean non_owner_forbid;
boolean override_row_restrictions;

/* Process */
char *login_name_only;
char *full_name_only;
char *street_address_only;

/* Returns null if no ENTITY->login_name */
char *security_login_name_full_name_only(
    &this->street_address_only,
    this->login_name );

/* Returns full_name_only */
char *security_entity_fetch(
    &this->street_address_only,
    this->login_name );

char *security_entity_where(
    SECURITY_ENTITY *security_entity,
    LIST *folder_attribute_list );
```

APPASERVER_USER

```
/* Usage */
APPASERVER_USER *appaserver_user_fetch(
    char *login_name,
    boolean fetch_role_name_list,
    boolean fetch_session_list );

/* Input */
char *login_name;
char *full_name;
char *typed_in_password;
char *database_password;
char *user_date_format;

/* Process */
APPASERVER_USER *appaserver_user_calloc( void );

APPASERVER_USER *appaserver_user_parse(
    char *input,
    boolean fetch_role_name_list,
    boolean fetch_session_list );

LIST *appaserver_user_session_list(
    char *login_name );

FILE *appaserver_user_update_open(
    void );

void appaserver_user_update(
    appaserver_user_update_open(),
    security_encrypted_password(
        security_sql_injection_escape(
            database_password ),
        security_mysql_version_password_function(
            security_mysql_version() ) ),
    login_name );
```



FRAMESET

```
/* Usage */
FRAMESET *frameset_new(
    char *applicatation_name,
    char *session_key,
    boolean create_menu_frame );

/* Process */
APPASERVER_PARAMETER_FILE *
    appaserver_parameter_file =
        appaserver_parameter_file_new();

FRAMESET_FRAME *frameset_frame_menu =
    frameset_frame_new(
        application_name,
        appaserver_parameter_file->document_root,
        session_key,
        FRAMESET_MENU_FRAME );

FRAMESET_FRAME *frameset_frame_prompt =
    frameset_frame_new(
        application_name,
        appaserver_parameter_file->document_root,
        session_key,
        FRAMESET_PROMPT_FRAME );

FRAMESET_FRAME *frameset_frame_edit =
    frameset_frame_new(
        application_name,
        appaserver_parameter_file->document_root,
        session_key,
        FRAMESET_EDIT_FRAME );

char *frameset_html(
    frameset_frame_menu->html,
    frameset_frame_prompt->html,
    frameset_frame_edit->html,
    boolean create_menu_frame );
```

FRAMESET_FRAME

```
/* Usage */
FRAMESET_FRAME *frameset_frame_new(
    char *application_name,
    char *document_root_directory,
    char *session_key,
    char *frame_name );

/* Process */
APPASERVER_LINK *appaserver_file =
    appaserver_file_new();

char *prompt_filename =
    appaserver_link_prompt_filename();

char *output_filename =
    appaserver_link_output_filename();

char *frameset_frame_html(
    char *frame_name,
    char *prompt_filename );
```

APPASERVER Data Structure

DOCUMENT

```
/* Usage */
DOCUMENT *document_calloc( void );

DOCUMENT *document_new(
    char *application_name,
    MENU *menu_fetch() );

/* Process */
DOCUMENT_HEAD *document_head =
    document_head_new(
        application_title_string( application_name ),
        document_head_menu_setup_string(),
        (char *)0 /* calendar_setup_string */,
        (char *)0 /* javascript_include_string */ );

DOCUMENT_BODY *document_body =
    document_body_new(
        document_body_onload_string(),
        menu );

char *document_type_string( void );

char *document_standard_string( void );
```

```
/* External */
void document_output_content_type(
    void );

/* Need to output document_close_html() */
void document_quick_output(
    char *application_name );

/* Need to output document_close_html() */
void document_begin(
    FILE *output_stream,
    DOCUMENT *document_new() );

char *document_begin_html(
    document_type_string(),
    document_standard_string() );

char *document_close_html( void );
```

DOCUMENT_HEAD

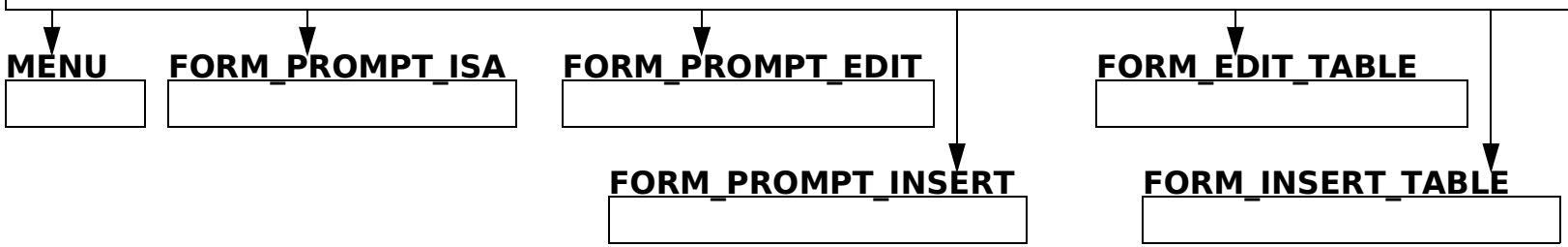
DOCUMENT_BODY

DOCUMENT_HEAD

```
/* Usage */
DOCUMENT_HEAD *document_head_calloc( void );

DOCUMENT_HEAD *document_head_new(
    char *title_string,
    document_head_menu_setup_string(),
    document_head_calendar_setup_string(),
    document_head_javascript_include_string() );

/* Input */
char *title_string;
char *menu_setup_string;
char *calendar_setup_string;
char *javascript_include_string;

/* Process */
char *document_head_meta_string(
    void );

char *document_head_stylesheet_string(
    char *environment_application_name() );

char *document_head_title_tag(
    char *title_string );

/* External */
char *document_head_menu_setup_string(
    void );

char *document_head_calendar_setup_string(
    void );

char *document_head_javascript_include_string(
    void );

char *document_head_begin_html(
    DOCUMENT_HEAD *document_head );

char *document_head_close_html( void );
```

DOCUMENT_BODY

```
/* Usage */
DOCUMENT_BODY *document_body_new(
    char *document_body_onload_string(),
    MENU *menu_fetch() );

/* Input */
char *onload_string;
MENU *menu;

/* Attributes */
FORM_PROMPT_ISA *form_prompt_isa;
FORM_PROMPT *form_prompt;
FORM_TABLE *form_table;
FORM_DETAIL *form_detail;

/* External */
char *document_body_menu_onload_string(
    void );

char *document_body_onload_string(
    document_body_menu_onload_string(),
    char *additional_onload_string );

void document_body_begin(
    FILE *output_stream,
    DOCUMENT_BODY *document_body );

char *document_body_begin_html(
    document_body_onload_string() );

char *document_body_hide_preload_message(
    void );

void document_body_horizontal_menu_output(
    FILE *output_stream,
    document_body_hide_preload_message(),
    MENU *menu );

void document_body_vertical_menu_output(
    FILE *output_stream,
    MENU *menu );
```



DOCUMENT (Choose isa)

```
DOCUMENT *document_choose_isa(
    char *choose_isa_title(),
    char *choose_isa_prompt_message(),
    char *one2m_isa_folder_name,
    MENU *menu_fetch(),
    LIST *folder->primary_key_list,
    LIST *query->delimited_list,
    boolean folder->no_initial_capital,
    char *choose_isa_action_string() );

/* Process */
DOCUMENT *document_new();

DOCUMENT_HEAD *document->document_head_new(
        title,
        document_head_menu_setup_string(),
        (char *)0 /* calendar_setup_string */
        document_head_javascript_include_string() );

document->document_body =
    document_body_choose_isa(
        char *title,
        char *prompt_message,
        char *one2m_isa_folder_name,
        MENU *menu,
        LIST *primary_key_list,
        LIST *delimited_list,
        boolean no_initial_capital,
        choose_isa_action_string() );
```

DOCUMENT_HEAD

DOCUMENT_BODY (Choose isa)

```
/* Usage */
DOCUMENT_BODY *document_body =
    document_body_choose_isa(
        char *title,
        char *prompt_message,
        char *one2m_isa_folder_name,
        MENU *menu,
        LIST *primary_key_list,
        LIST *delimited_list,
        boolean no_initial_capital,
        char *action_string );

/* Process */
DOCUMENT_BODY *document_body =
    document_body_new(
        document_body_onload_string(),
        menu );

document_body->form_prompt_isa =
    FORM_PROMPT_INSERT_ISA *form_prompt_isa_new(
        title,
        prompt_message,
        one2m_folder_name,
        primary_key_list,
        delimited_list,
        no_initial_capital,
        action_string );
```

FORM_PROMPT_ISA

APPASERVER Data Structure

DOCUMENT (Edit Table)

```
DOCUMENT *document_edit_table(
    char *edit_table_title(),
    char *edit_table_message(),
    char *folder_name,
    MENU *menu_fetch(),
    LIST *folder->folder_attribute_isa_name_list,
    LIST *query->delimited_list,
    boolean folder->no_initial_capital,
    char *edit_table_action_string() );

/* Process */
DOCUMENT *document_new();

DOCUMENT_HEAD *document->document_head_new(
        title,
        document_head_menu_setup_string(),
        (char *)0 /* calendar_setup_string */
        document_head_javascript_include_string() );

document->document_body =
    document_body_edit_table(
        char *title,
        char *message,
        char *folder_name,
        MENU *menu,
        LIST *folder_attribute_isa_name_list,
        LIST *delimited_list,
        boolean no_initial_capital,
        edit_table_action_string() );
```

DOCUMENT_HEAD

DOCUMENT_BODY (Edit Table)

```
/* Usage */
DOCUMENT_BODY *document_body =
    document_body_edit_table(
        char *title,
        char *message,
        char *folder_name,
        MENU *menu,
        LIST *folder_attribute_isa_name_list,
        LIST *delimited_list,
        boolean no_initial_capital,
        char *action_string );

/* Process */
DOCUMENT_BODY *document_body =
    document_body_new(
        document_body_onload_string(),
        menu );

document_body->form_edit_table =
    FORM_EDIT_TABLE *form_edit_table_new(
        title,
        message,
        folder_name,
        folder_attribute_isa_name_list,
        delimited_list,
        no_initial_capital,
        action_string );
```

FORM_EDIT_TABLE

DOCUMENT (Detail)

```
DOCUMENT *document_detail_new(
);

/* Process */
DOCUMENT *document_new();

DOCUMENT_HEAD *document->document_head_new(
        title,
        document_head_menu_setup_string(),
        (char *)0 /* calendar_setup_string */
        document_head_javascript_include_string() );

document->document_body =
    document_body_detail_new(
);
```

DOCUMENT_HEAD

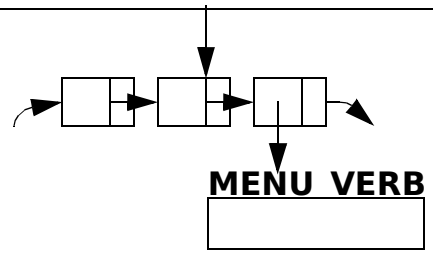
DOCUMENT_BODY (Detail)

```
/* Usage */
DOCUMENT_BODY *document_body =
    document_body_detail_new(
 );

/* Process */
DOCUMENT_BODY *document_body =
    document_body_new(
        document_body_onload_string(),
        menu );
```

APPASERVER Data Structure

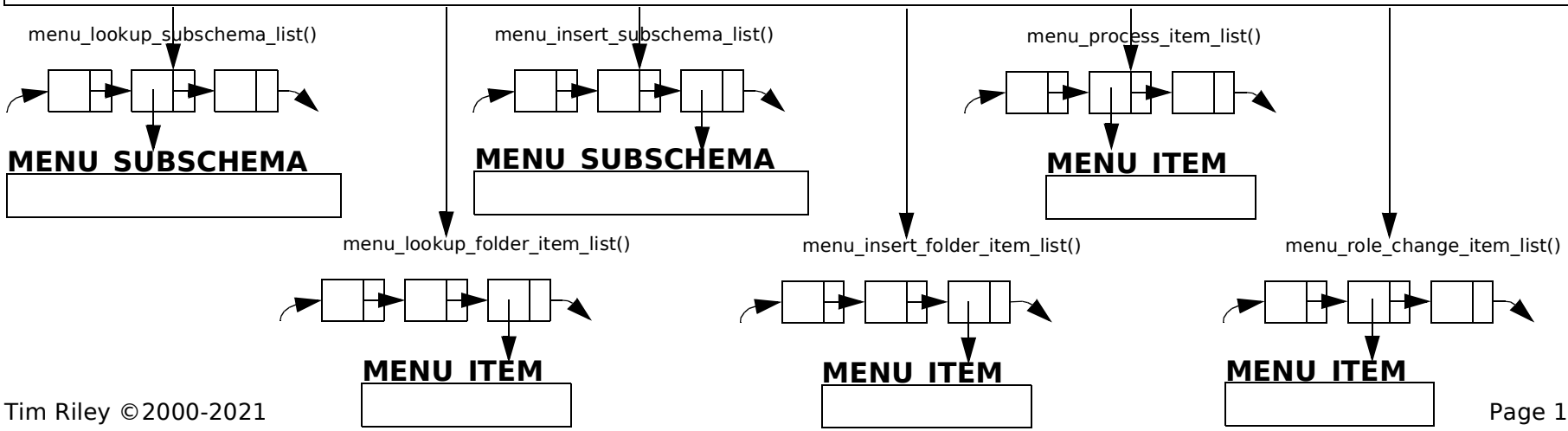
MENU

```
/* Usage */
MENU *menu_fetch(
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame,
    LIST *folder_menu_lookup_count_list() );

/* Process */
LIST *menu_verb_list = list_new();

list_set(
    menu_verb_list,
    MENU_VERB *menu_verb_lookup_subschema_fetch(
        role_folder_lookup_list(),
        folder_menu_lookup_count_list,
        application_name,
        login_name,
        session_key,
        role_name,
        target_frame ) );

list_set(
    menu_verb_list,
    MENU_VERB *menu_verb_insert_subschema_fetch(
        role_folder_insert_list(),
        application_name,
        login_name,
        session_key,
        role_name,
        target_frame ) );

LIST *role_process_list(
    role_name );

LIST *role_process_set_list(
    role_name );

LIST *menu_process_group_name_list(
    role_process_group_name_list(
        role_process_list() ),
    role_process_set_group_name_list(
        role_process_set_list() ) );

for group_name in
    'menu_process_group_name_list()'
{
    list_set(
        menu_verb_list,
        MENU_VERB *menu_verb_process_fetch(
            group_name /* verb */,
            role_process_list(),
            role_process_set_list(),
            application_name,
            login_name,
            session_key,
            role_name,
            target_frame ) );
}

list_set(
    menu_verb_list,
    MENU_VERB *menu_verb_role_change_fetch(
        role_name_list(
            role_list_fetch(
                login_name ) ),
        application_name,
        login_name,
        session_key,
        role_name ) );

list_set(
    menu_verb_list,
    MENU_VERB *menu_verb_role_display(
        role_name ) );

list_set(
    menu_verb_list,
    MENU_VERB*menu_verb_login_name_display(
        login_name ) );
```



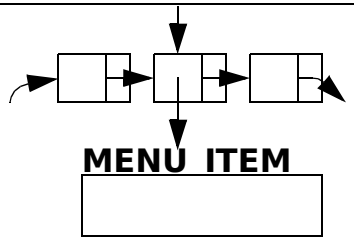
MENU_VERB

```
/* Usage */
MENU_VERB *menu_verb_lookup_subschema_fetch(
    role_folder_lookup_list(),
    folder_menu_lookup_count_list(),
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

MENU_VERB *menu_verb_insert_subschema_fetch(
    role_folder_insert_list(),
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

MENU_VERB *menu_verb_process_fetch(
    char *verb,
    LIST *role_process_list(),
    LIST *role_process_set_list(),
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

MENU_VERB *menu_verb_role_change_fetch(
    char *role_name /* verb */,
    LIST *role_name_list,
    char *application_name,
    char *login_name,
    char *session_key );

MENU_VERB *menu_verb_role_display(
    char *role_name /* verb */ );

MENU_VERB*menu_verb_login_name_display(
    char *login_name /* verb */ );

/* Process */
LIST *menu_lookup_subschema_list(
    role_folder_lookup_list(),
    folder_menu_lookup_count_list(),
    application_name,
    login_name,
    session_key,
    role_name,
    target_frame );

LIST *menu_lookup_folder_item_list(
    role_folder_subschema_missing_folder_name_list(
        role_folder_lookup_list() ),
    folder_menu_lookup_count_list(),
    application_name,
    login_name,
    session_key,
    role_name,
    target_frame );

LIST *menu_insert_subschema_list(
    role_folder_insert_list(),
    application_name,
    login_name,
    session_key,
    role_name,
    target_frame );

LIST *menu_insert_folder_item_list(
    role_folder_subschema_missing_folder_name_list(
        role_folder_insert_list() ),
    application_name,
    login_name,
    session_key,
    role_name,
    target_frame );

LIST *menu_process_item_list(
    role_process_or_set_name_list(),
    application_name,
    login_name,
    session_key,
    role_name,
    target_frame );

LIST *menu_role_change_item_list(
    role_name_list,
    application_name,
    login_name,
    session_key,
    role_name );

char *menu_verb_tag( void );

/* External */
void menu_verb_horizontal_output(
    FILE *output_stream,
    char *hide_preload_message,
    LIST *menu_verb_list );
```



MENU_SUBSCHEMA

```
/* Usage */
LIST *menu_lookup_subschema_list(
    role_folder_lookup_list(),
    folder_menu_lookup_count_list(),
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

LIST *menu_insert_subschema_list(
    role_folder_insert_list(),
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

MENU_SUBSCHEMA *menu_lookup_subschema_fetch(
    char *subschema_name,
    LIST *role_folder_lookup_name_list(),
    LIST *folder_menu_lookup_count_list(),
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

MENU_SUBSCHEMA *menu_insert_subschema_fetch(
    char *subschema_name,
    LIST *role_folder_insert_name_list(),
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

/* Input */
char *subschema_name;
char *application_name;
char *login_name;
char *session_key;
char *role_name;
char *target_frame;

/* Process */
LIST *menu_lookup_folder_item_list(
    role_folder_lookup_name_list(),
    folder_menu_lookup_count_list(),
    application_name,
    login_name,
    session_key,
    role_name,
    target_frame );

LIST *menu_insert_folder_item_list(
    role_folder_insert_name_list(),
    application_name,
    login_name,
    session_key,
    role_name,
    target_frame );

char *menu_subschema_span_tag(
    char *subschema_name );

/* External */
void menu_subschema_horizontal_output(
    FILE *output_stream,
    LIST *subschema_list );
```



MENU_ITEM

```
/* Usage */
LIST *menu_lookup_folder_item_list(
    LIST *role_folder_lookup_name_list,
    LIST *folder_menu_lookup_count_list,
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

LIST *menu_insert_folder_item_list(
    LIST *role_folder_insert_name_list,
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

LIST *menu_process_item_list(
    role_process_or_set_name_list(),
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

LIST *menu_role_change_item_list(
    LIST *role_name_list,
    char *application_name,
    char *login_name,
    char *session_key );

MENU_ITEM *menu_item_folder_new(
    char *folder_name,
    char *folder_menu_seek()->
        folder_menu_count_display,
    char *state,
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

MENU_ITEM *menu_item_process_new(
    char *process_or_set_name,
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

MENU_ITEM *menu_item_role_change_new(
    char *application_name,
    char *login_name,
    char *session_key,
    char *change_role_name );

/* Attributes */
char *folder_name;
char *folder_menu_count_display;
char *process_or_set_name;
char *application_name;
char *login_name;
char *session_key;
char *role_name;
char *target_frame;

/* Process */
char *state = “lookup” | ”insert”;

char *menu_item_folder_span_tag(
    char *folder_name,
    char *folder_menu_count_display );

char *menu_item_folder_action_tag(
    appaserver_library_http_prompt(),
    char *folder_name,
    char *state,
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

char *menu_item_process_span_tag(
    char *process_or_set_name );

char *menu_item_process_action_tag(
    appaserver_library_http_prompt(),
    char *process_or_set_name,
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

char *menu_item_role_name_span_tag(
    char *role_name );

char *menu_item_role_change_action_tag(
    appaserver_library_http_prompt(),
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name );

/* External */
void menu_item_horizontal_output(
    FILE *output_stream,
    LIST *menu_item_list );
```

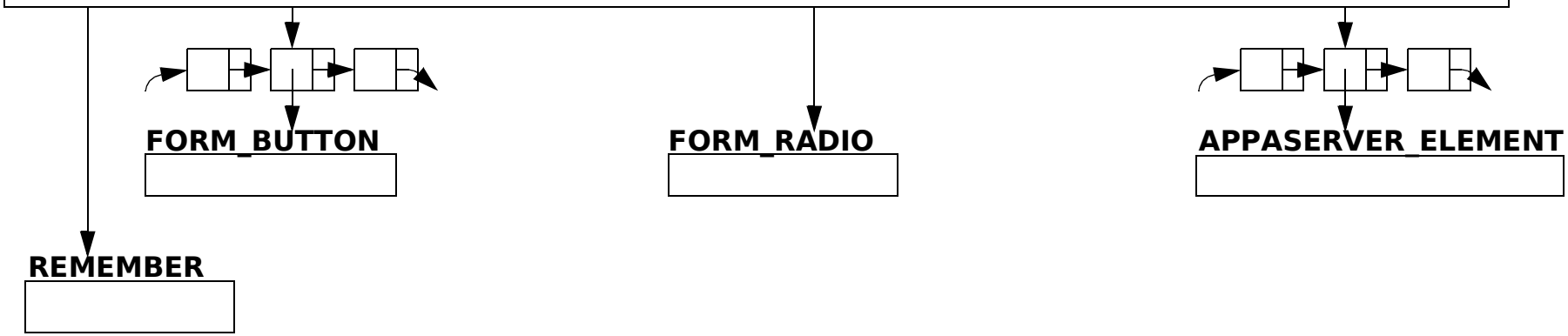
APPASERVER Data Structure

FORM (External)

```
/* External */
char *form_title_html(
    char *title );

char *form_message_html(
    char *prompt_message );

char *form_tag_html(
    char *action_string,
    char *target_frame );

char *form_next_reference_number(
    int *form_current_reference_number );
```

FORM_PROMPT

```
/* Usage */
FORM_PROMPT *form_prompt_new(
    char *title,
    char *subtitle,
    char *action_string );

/* Attributes */
char *title;
char *subtitle;
char *action_string;
LIST *form_button_bottom_list;
FORM_RADIO *form_radio;
REMEMBER *remember;
LIST *element_list;

/* Process */
LIST *form_prompt_button_list(

/* External */
void form_prompt_output(
    FILE *output_stream,
    FORM_PROMPT *form_prompt );
```

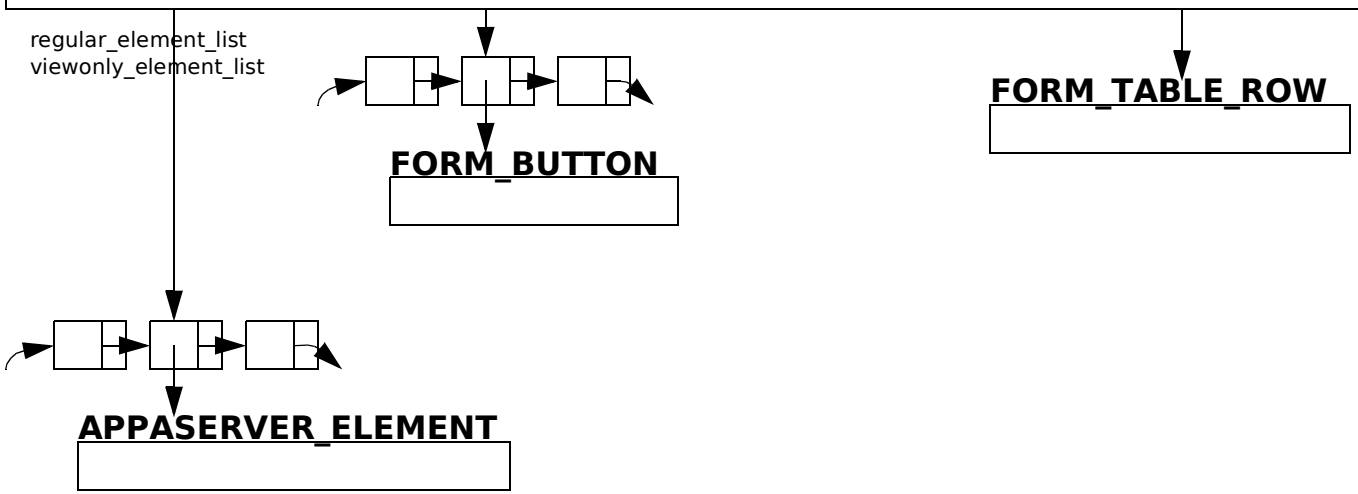


FORM_PROMPT_ISA

```
/* Usage */
FORM_PROMPT_ISA *form_prompt_isa_new(
    char *title,
    char *prompt_message,
    char *one2m_folder_name,
    LIST *primary_key_list,
    LIST *delimited_list,
    boolean no_initial_capital,
    choose_isa_action_string() );

/* Process */
char *form_title_html(
    char *title );

char *form_message_html(
    char *prompt_message );

char *form_tag_html(
    char *action_string,
    PROMPT_FRAME /* target_frame */ );

LIST *form_prompt_isa_element_list(
    char *one2m_isa_folder_name,
    LIST *primary_key_list,
    LIST *delimited_list,
    boolean no_initial_capital );

LIST *form_prompt_isa_button_list(
    void );

/* External */
void form_prompt_isa_output(
    FILE *output_stream,
    form_title_html(),
    form_message_html(),
    form_tag_html(),
    form_prompt_isa_element_list(),
    form_prompt_isa_button_list() );
```

FORM_TABLE

```
/* Usage */
FORM_TABLE *form_table_new(
    char *folder_name,
    int form_number,
    char *state,
    char *title_string,
    LIST *regular_element_list,
    LIST *view_only_element_list,
    char *action_string );

/* Attributes */
char *folder_name;
int form_number;
char *state;
char *title_string;
LIST *regular_element_list;
LIST *viewonly_element_list;
char *action_string;

/* Process */

/* External */
LIST *form_table_element_list();

LIST *form_table_button_list(
    void );

LIST *form_table_heading_list(
    LIST *regular_element_list );

for (   row_number = 1;
        row_number <= list_length( dictionary_list );
        row_number++ )
{
    FORM_TABLE_ROW *form_table_row =
        form_table_row_new(
            DICTIONARY *list_get( dictionary_list ),
            form_table_element_list(),
            row_number,
            form_table_row_background_color() );

    form_table_row_output(
        output_stream,
        form_table_row );

    form_table_row_free( form_table_row );
}
```



FORM_TABLE_ROW

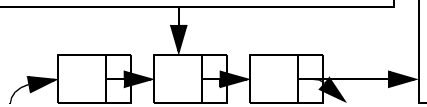
```
/* Usage */
FORM_TABLE_ROW *form_table_row_new(
    FILE *output_stream,
    DICTIONARY *row_dictionary,
    form_table_element_list(),
    int row_number,
    form_table_row_background_color() );

/* Process */
LIST *form_table_row_html_list(
    DICTIONARY *row_dictionary,
    LIST *element_list,
    int row_number,
    char *background_color );

void form_table_row_output(
    FILE *output_stream,
    form_table_row_html_list() );

/* External */
void form_table_row_free(
    FORM_TABLE_ROW *form_table_row );

char *form_table_row_background_color( void );

static char **form_table_row_background_color_array(
    int *background_color_array_length,
    environment_application_name() );
```

FORM_BUTTON

```
/* Usage */
FORM_BUTTON *form_button_submit(
    int form_number );

FORM_BUTTON *form_button_reset(
    int form_number,
    char *post_change_javascript );

FORM_BUTTON *form_button_back_to_top(
    void );

FORM_BUTTON *form_button_remember(
    char *action_string );

FORM_BUTTON *form_button_back_to_drillthru(
    char *action_string );

FORM_BUTTON *form_button_back_forward(
    void );

FORM_BUTTON *form_button_drillthru_skip(
    void );

FORM_BUTTON *form_button_html_help(
    char *application_name,
    char *html_help_file_anchor );

LIST *form_button_insert_pair_one2m_submit_list(
    LIST *pair_one2m_folder_list );

/* Attribute */
char *html;

/* External */
void form_button_list_output(
    FILE *output_stream,
    LIST *form_button_list );

void form_button_output(
    FILE *output_stream,
    char *form_button->html );

char *form_button_submit_html(
    char *submit_control_string,
    char *button_label,
    int form_number );
```

FORM_RADIO

```
/* Usage */
FORM_RADIO *form_radio_new(
    char *radio_name,
    char *initial_value,
    LIST *value_string_list,
    char *set_all_push_buttons_html );

/* Process */
LIST *form_radio_value_list(
    radio_name,
    initial_value,
    value_string_list );

char *form_radio_html(
    form_radio_value_list(),
    char *set_all_push_buttons_html );
```

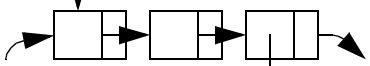


FORM_RADIO_VALUE

```
/* Usage */
LIST *form_radio_value_list(
    char *radio_name,
    char *initial_value,
    LIST *value_string_list );

FORM_RADIO_VALUE *form_radio_value_new(
    char *radio_name,
    char *initial_value,
    char *value_string );

/* Process */
char *form_radio_value_html(
    char *radio_name,
    char *initial_value,
    char *value_string );
```

PROMPT_RECURSIVE

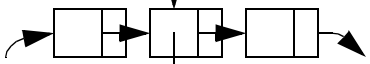
```
/* Usage */
PROMPT_RECURSIVE *prompt_recursive_new(
    char *folder_name,
    LIST *relation_mto1_non_isa_list,
    DICTIONARY *drillthru_dictionary,
    char *login_name,
    boolean non_owner_forbid,
    boolean override_row_restrictions,
    boolean drillthru_skipped() );

/* Input */
char *folder_name;

/* Process */
if ( drillthru_skipped )
{
    return (PROMPT_RECURSIVE *)0;
}

SECURITY_ENTITY *security_entity =
    security_entity_new(
        login_name,
        non_owner_forbid,
        override_row_restrictions );

LIST *prompt_recursive_folder_list(
    relation_mto1_non_isa_list,
    drillthru_dictionary,
    login_name,
    security_entity );

char *prompt_recursive_html(
    LIST *prompt_recursive_folder_list() );
```



RELATION

```
FOLDER *one_folder;
boolean prompt_mto1_recursive;
```

DICTIONARY



PROMPT_RECURSIVE_FOLDER

prompt_recursive->html

char *

PROMPT_RECURSIVE_FOLDER

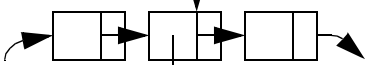
```
/* Usage */
LIST *prompt_recursive_folder_list(
    LIST *relation_mto1_non_isa_list,
    DICTIONARY *drillthru_dictionary,
    char *login_name,
    SECURITY_ENTITY *security_entity );

PROMPT_RECURSIVE_FOLDER *
    prompt_recursive_folder_new(
        FOLDER *relation->one_folder,
        relation->drop_down_multi_select,
        DICTIONARY *drillthru_dictionary,
        char *login_name,
        SECURITY_ENTITY *security_entity );

/* Input */
FOLDER *one_folder;
boolean drop_down_multi_select;

/* Process */
one_folder->relation_mto1_non_isa_list =
    relation_mto1_non_isa_list(
        one_folder->folder_name );

one_folder->delimited_list =
    query_primary_delimited_fetch_list(
        one_folder->folder_name,
        one_folder->primary_key_list,
        one_folder->folder_attribute_primary_list,
        one_folder->relation_mto1_non_isa_list,
        drillthru_dictionary,
        login_name,
        security_entity_where(
            security_entity,
            one_folder->folder_attribute_list ) );

LIST *relation_mto1_primary_key_subset_list(
        one_folder->folder_name
            /* many_folder_name */,
        one_folder->primary_key_list );

LIST *mto1_folder_list =
    prompt_recursive_mto1_folder_list(
        relation_mto1_primary_key_subset_list(),
        drillthru_dictionary,
        login_name,
        security_entity );

LIST *prompt_recursive_folder_one_element_list(
    char *one_folder->folder_name,
    LIST *one_folder->primary_key_list,
    LIST *one_folder->delimited_list,
    boolean one_folder->no_initial_capital,
    char *one_folder->notepad,
    boolean drop_down_multi_select,
    char *prompt_recursive_folder_javascript() );

LIST *element_list =
    prompt_recursive_mto1_folder_element_list(
        /* Appends to one_element_list */
        prompt_recursive_folder_one_element_list(),
        prompt_recursive_mto1_folder_list()
        prompt_recursive_folder_javascript() );

/* Internal */
char *prompt_recursive_folder_javascript(
        LIST *one_folder->primary_key_list,
        boolean drop_down_multi_select,
        relation_mto1_primary_key_subset_list() );

/* Output */
char *prompt_recursive_folder_html(
    prompt_recursive_folder_element_list() );
```

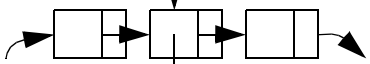
one_folder

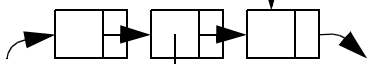
relation_mto1_primary_key_subset_list

prompt_recursive_folder_javascript

char *







RELATION

PROMPT_RECURSIVE_MTO1_FOLDER

FOLDER

```
char *folder_name;
LIST *primary_key_list;
LIST *relation_mto1_non_isa_list;
LIST *delimited_list;
```

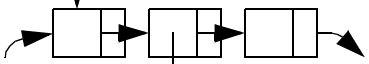
APPASERVER_ELEMENT

PROMPT_RECURSIVE_MTO1_FOLDER

```
/* Usage */
LIST *prompt_recursive_mto1_folder_list(
    LIST *relation_mto1_primary_key_subset_list(),
    DICTIONARY *drillthru_dictionary,
    char *login_name,
    SECURITY_ENTITY *security_entity );

PROMPT_RECURSIVE_MTO1_FOLDER *
    prompt_recursive_mto1_folder_new(
        FOLDER *one_folder,
        boolean drop_down_multi_select,
        DICTIONARY *drillthru_dictionary,
        char *login_name,
        SECURITY_ENTITY *security_entity );
```

```
/* Input */
FOLDER *one_folder;
boolean drop_down_multi_select;

/* Process */
one_folder->relation_mto1_non_isa_list =
    relation_mto1_non_isa_list(
        one_folder->folder_name );

one_folder->primary_delimited_list =
    query_primary_delimited_fetch_list(
        one_folder->folder_name,
        one_folder->primary_key_list,
        one_folder->folder_attribute_primary_list,
        one_folder->relation_mto1_non_isa_list,
        drillthru_dictionary,
        login_name,
        security_entity_where(
            security_entity,
            one_folder->folder_attribute_list ) );
```

```
/* External */
LIST *prompt_recursive_mto1_folder_element_list(
        /* Appends to one_element_list */
        prompt_recursive_folder_one_element_list(),
        prompt_recursive_mto1_folder_list()
        prompt_recursive_folder_javascript() );
```



FOLDER

```
char *folder_name;
LIST *primary_key_list;
LIST *relation_mto1_non_isa_list;
LIST *primary_delimited_list;
```

APPASERVER_ELEMENT

PROMPT_EDIT

```
/* Usage */
PROMPT_EDIT *prompt_edit_fetch(
    char *application_name,
    char *login_name,
    char *session_key,
    char *folder_name,
    char *role_name,
    char *target_frame,
    char *state /* maybe drillthru for state=insert */,
    char *appaserver_parameter_file->
        appaserver_mount_point,
    POST_DICTIONARY *
        post_dictionary_string_new() );

/* Process */
LIST *role_folder_list =
    role_folder_fetch_list(
        role_name,
        folder_name );

if ( boolean prompt_edit_forbid(
    role_folder_update( role_folder_list ),
    role_folder_lookup( role_folder_list ) )
{
    return (PROMPT_EDIT_FORM *)0;
}

ROLE *role =
    role_fetch(
        role_name,
        1 /* fetch_attribute_exclude_list */ );

FOLDER *folder =
    folder_fetch(
        folder_name,
        role_exclude_lookup_attribute_name_list(
            role->attribute_exclude_list ),
        /* Also sets folder_attribute_primary_list */
        /* and primary_key_list */
        1 /* fetch_folder_attribute_list */,
        1 /* fetch_relation_mto1_non_isa_list */,
        /* Also sets folder_attribute_append_isa_list */
        1 /* fetch_relation_mto1_isa_list */,
        1 /* fetch_relation_one2m_list */,
        1 /* fetch_relation_one2m_recursive_list */,
        0 /* not fetch_process */,
        0 /* not fetch_role_folder_list */,
        1 /* fetch_row_level_restriction */,
        0 /* not fetch_role_operation_list */ );

DICTIONARY_SEPARATE *dictionary_separate =
    dictionary_separate_folder_new(
        post_dictionary->original_post_dictionary,
        application_name,
        login_name,
        folder_attribute_date_name_list(
            folder->folder_attribute_isa_list ) );

DRILLTHRU *drillthru =
    drillthru_fetch(
        dictionary_separate->
            drillthru_dictionary );

PROMPT_RECURSIVE *prompt_recursive =
    prompt_recursive_new(
        folder_name,
        folder->relation_mto1_non_isa_list,
        dictionary_separate->drillthru_dictionary,
        login_name,
        folder->non_owner_forbid,
        role->override_row_restrictions,
        drillthru_skipped() );

boolean prompt_edit_omit_insert_button(
    drillthru_skipped() );

boolean prompt_edit_omit_delete_button(
    list_length( folder->relation_mto1_isa_list ) );

boolean prompt_edit_omit_new_button(
    relation_exists_multi_select(
        folder->relation_mto1_non_isa_list ) );

folder->relation_mto1_non_isa_list =
    /* Returns relation_mto1_non_isa_list or null */
    prompt_edit_drillthru_skipped(
        LIST *relation_mto1_non_isa_list,
        drillthru_skipped() );

char *prompt_edit_target_frame(
    /* Returns target_frame or “edit_form” */
    char *target_frame,
    drillthru_skipped() );

char *prompt_edit_title(
    char *folder_name );

DOCUMENT *document_new();

document->document_head_new(
    prompt_edit_form_title(),
    document_head_menu_setup_string(),
    document_head_calendar_setup_string(),
    document_head_javascript_include_string();
```

ROLE

FOLDER

DRILLTHRU

DOCUMENT

ROLE_FOLDER

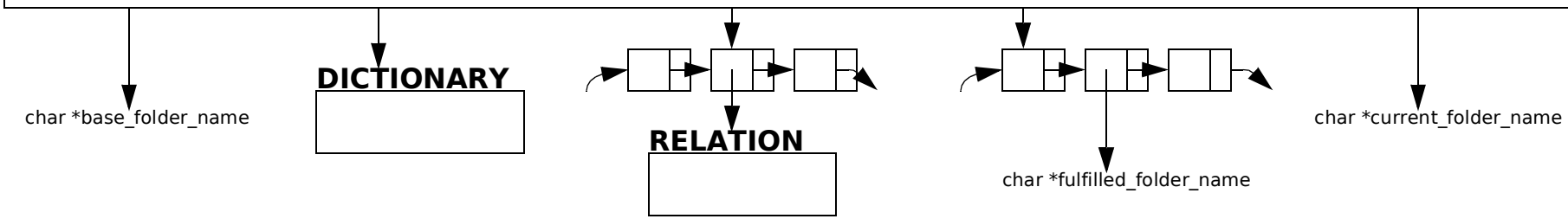
DICTIONARY_SEPARATE

```
DICTIONARY *drillthru_dictionary;
```

PROMPT_RECURSIVE

DRILLTHRU

```
/* Usage */
DRILLTHRU *drillthru_start(
    char *folder_name );

DRILLTHRU *drillthru_fetch(
    DICTIONARY *drillthru_dictionary );

/* Drillthru_start() processes */
LIST *relation_mto1_drillthru_list(
    (LIST *)0 /* relation_list */,
    folder_name,
    (LIST *)0 /* fulfilled_folder_name_list */ );

boolean drillthru_participating =
    drillthru_start_participating(
        length( relation_mto1_drillthru_list ) );

void drillthru_base_set(
    DICTIONARY *drillthru_dictionary,
    char *folder_name );

/* Drillthru_fetch() processes */
char *drillthru_base_folder_name(
    DICTIONARY *drillthru_dictionary );

boolean drillthru_participating(
    char *base_folder_name );

LIST *drillthru_fulfilled_folder_name_list(
    DICTIONARY *drillthru_dictionary );

LIST *relation_mto1_drillthru_list(
    (LIST *)0 /* relation_list */,
    base_folder_name,
    drillthru_fulfilled_folder_name_list() );

/* Generic */
char *drillthru_participating_folder_name(
        char *folder_name,
        DRILLTHRU *drillthru );

char *drillthru_current_folder_name(
    relation_mto1_drillthru_list(),
    drillthru_base_folder_name() );

void drillthru_fulfilled_current_set(
    DICTIONARY *drillthru_dictionary,
    drillthru_fulfilled_folder_name_list(),
    drillthru_current_folder_name() );

void drillthru_fulfilled_set(
    DICTIONARY *drillthru_dictionary,
    drillthru_fulfilled_folder_name_list() );

boolean drillthru_skipped(
    char *folder_name,
    drillthru_base_folder_name(),
    list_length( drillthru_fulfilled_folder_name_list() ) );

boolean drillthru_middle(
    char *folder_name,
    drillthru_current_folder_name() );

boolean drillthru_finished(
    char *folder_name,
    drillthru_current_folder_name() );
```



APPASERVER Data Structure

APPASERVER_ELEMENT

```
/* Usage */
APPASERVER_ELEMENT *appaserver_element_new(
    enum element_type element_type );

/* Input */
enum element_type element_type;

/* Attributes */
ELEMENT_TABLE_ROW *table_row;
ELEMENT_CHECKBOX *checkbox;
ELEMENT_DROP_DOWN *drop_down;
ELEMENT_MULTI_DROP_DOWN *multi_drop_down;
ELEMENT_RADIO_BUTTON *radio_button;
ELEMENT_NON_EDIT_TEXT *non_edit_text;
ELEMENT_HIDDEN *hidden;

ELEMENT_NOTEPAD *notepad;
ELEMENT_TEXT_ITEM *text_item;
ELEMENT_PASSWORD *password;
ELEMENT_PROMPT_DATA *prompt_data;
ELEMENT_UPLOAD_FILENAME *upload_filename;
ELEMENT_HTTP_FILENAME *http_filename;
ELEMENT_REFERENCE_NUMBER *reference_number;
ELEMENT_ANCHOR *anchor;
ELEMENT_BUTTON *button;
ELEMENT_BLANK *blank;
ELEMENT_BREAK_TAG *break_tag;
ELEMENT_JAVASCRIPT_FILENAME *javascript_filename;
ELEMENT_DATE *date;
ELEMENT_CURRENT_DATE *current_date;
ELEMENT_TIME *time;
ELEMENT_CURRENT_TIME *current_time;
ELEMENT_CURRENT_DATE_TIME *current_date_time;
ELEMENT_HTTP_FILENAME *http_filename;
ELEMENT_TIMESTAMP *timestamp;
ELEMENT_EMPTY_COLUMN *empty_column;

/* External */
char *appaserver_element_heading(
    APPASERVER_ELEMENT *element );

char *appaserver_element_heading_string(
        name );

int appaserver_element_tab_order(
    int tab_order );

char *appaserver_element_javascript(
    char *post_change_javascript,
    int row_number,
    char *state );

char *appaserver_element_name(
    char *name,
    int row_number );

char *appaserver_element_list_html(
    char *background_color,
    char *state,
    int row_number,
    LIST *appaserver_element_list );

char *appaserver_element_html(
    char *background_color,
    char *state,
    int row_number,
    APPASERVER_ELEMENT *appaserver_element );

char *appaserver_element_javascript(
    char *post_change_javascript,
    char *state,
    int row_number );

char *appaserver_element_javascript_html(
    char *appaserver_element_javascript() );

char *appaserver_element_background_color_html(
    char *background_color );
```

table_row
checkbox
drop_down
multi_drop_down
button
non_edit_text

hidden

radio_button
notepad
text_item
password

prompt_data
upload_filename
reference_number
anchor
blank
break_tag

javascript_filename
element_date
element_current_date
element_time
element_current_time
element_date_time

element_current_date_time
http_filename
timestamp
empty_column

ELEMENT_MULTI_DROP_DOWN

```
/* Usage */
ELEMENT_MULTI_DROP_DOWN *element_multi_drop_down_new(
    LIST *attribute_name_list,
    LIST *delimited_list,
    boolean no_initial_capital,
    char *post_change_javascript,
    char *state );

/* Process */
ELEMENT_DROP_DOWN *original_drop_down =
    element_drop_down_new(
        attribute_name_list,
        (char *)0 /* initial_data */,
        delimited_list,
        no_initial_capital,
        0 /* not output_null_option */,
        0 /* not output_not_null_option */,
        0 /* not output_select_option */,
        1 /* column_span */,
        element_multi_drop_down_size(),
        0 /* tab_order */,
        1 /* multi_select */,
        (char *0 /* post_change_javascript */,
        (char *)0 /* state */ );

/* Override element_drop_down_html() */
original_drop_down->name =
    element_multi_drop_down_original_name(
        attribute_name_list,
        ELEMENT_MULTI_DROP_DOWN_ORIGINAL_PREFIX );

ELEMENT_BUTTON *move_right_button =
    element_button_new(
        element_multi_drop_down_move_right_label(),
        element_multi_drop_down_move_right_action_string(
            attribute_name_list ),
        1 /* with_table_data_tag */ );

ELEMENT_BREAK_TAG *element_break_tag =
    element_break_tag_new();

ELEMENT_BUTTON *move_left_button =
    element_button_new(
        element_multi_drop_down_move_left_label(),
        element_multi_drop_down_move_left_action_string(
            attribute_name_list ),
        0 /* not with_table_data_tag */ );

ELEMENT_DROP_DOWN *empty_drop_down =
    element_drop_down_empty_new(
        element_multi_drop_down_name(),
        element_multi_drop_down_size(),
        1 /* multi_select */,
        post_change_javascript,
        state );

/* Internal */
int element_multi_drop_down_size( void );

char *element_multi_drop_down_move_right_label( void );

char *element_multi_drop_down_move_left_label( void );

char *element_multi_drop_down_move_right_action_string(
    LIST *attribute_name_list );

char *element_multi_drop_down_move_left_action_string(
    LIST *attribute_name_list );

char *element_multi_drop_down_original_name(
    LIST *attribute_name_list,
    char *element_name_prefix );

char *element_multi_drop_down_name(
    LIST *attribute_name_list );

/* External */
char *element_multi_drop_down_html(
    ELEMENT_DROP_DOWN *original_drop_down,
    ELEMENT_BUTTON *move_right_button,
    ELEMENT_BREAK_TAG *element_break_tag,
    ELEMENT_BUTTON *move_left_button,
    ELEMENT_DROP_DOWN *empty_drop_down,
    char *appaserver_element_javascript(
        post_change_javascript,
        state,
        0 /* row_number */ ) );
```

ELEMENT_DROP_DOWN **ELEMENT_BUTTON** **ELEMENT_BREAK_TAG** **ELEMENT_BUTTON**

ELEMENT_DROP_DOWN

```
/* Usage */
ELEMENT_DROP_DOWN *element_drop_down_calloc(
    void );

ELEMENT_DROP_DOWN *element_drop_down_new(
    LIST *attribute_name_list,
    char *initial_data,
    LIST *delimited_list,
    boolean no_initial_capital,
    boolean output_null_option,
    boolean output_not_null_option,
    boolean output_select_option,
    int column_span,
    int element_[multi_]drop_down_size(),
    int appaserver_element_tab_order(),
    boolean multi_select,
    char *post_change_javascript,
    char *state );

ELEMENT_DROP_DOWN *
    element_drop_down_empty_new(
        char *drop_down_name,
        int drop_down_size,
        boolean multi_select,
        char *post_change_javascript,
        char *state );

/* Attributes */
LIST *attribute_name_list;
char *initial_data;
LIST *delimited_list;
boolean no_initial_capital;
boolean output_null_option;
boolean output_not_null_option;
boolean output_select_option;
int column_span;
int drop_down_size;
int tab_order;
boolean multi_select;
char *post_change_javascript;
char *state;

/* External */
char *element_drop_down_html(
    char *element_drop_down_name(),
    char *initial_data,
    LIST *delimited_list,
    element_drop_down_display_list(
        delimited_list,
        no_initial_capital ),
    boolean no_initial_capital,
    boolean output_null_option,
    boolean output_not_null_option,
    boolean output_select_option,
    int column_span,
    int drop_down_size,
    boolean multi_select,
    int tab_order,
    char *appaserver_element_javascript(),
    char *background_color );

char *element_drop_down_empty_html(
    char *element_multi_drop_down_name(),
    int drop_down_size,
    boolean multi_select,
    char *appaserver_element_javascript(),
    char *background_color );

LIST *element_drop_down_display_list(
    LIST *delimited_list,
    boolean no_initial_capital );

char *element_drop_down_initial_data_html(
    char *initial_data,
    char *list_offset_seek(
        element_drop_down_display_list(),
        list_seek_offset(
            char *initial_data,
            LIST *delimtied_list) )
            /* initial_display */,
    boolean no_initial_capital );

char *element_drop_down_option_value_list_html(
    LIST *delimited_list,
    element_drop_down_display_list(),
    char *initial_data );

char *element_drop_down_option_value_html(
    char *data,
    char *display );

char *element_drop_down_close_html(
    boolean output_null_option,
    boolean output_not_null_option );

char *element_drop_down_name(
    LIST *attribute_name_list );

char *element_drop_down_heading(
    LIST *attribute_name_list );

int element_drop_down_size(
    int delimited_list_length );

#define ELEMENT_DROP_DOWN_REMEMBER 1
```

ELEMENT_TEXT_ITEM

```
/* Usage */
ELEMENT_TEXT_ITEM *element_text_item_new(
    void );

/* Attributes */
char *data;
int attribute_width;
int widget_size;
char *heading;
char onchange_null_to_slash_yn;
char *post_change_javascript;
char *on_focus_javascript_function;
char *state;
boolean dont_create_current_date;
boolean readonly;
boolean is_numeric;

#define ELEMENT_TEXT_ITEM_REMEMBER 1
```

ELEMENT_CHECKBOX

```
/* Usage */
ELEMENT_CHECKBOX *element_checkbox_calloc(
    void );

/* Attribute */
char *name;
char *prompt_string;
boolean checked;
char *action_string;
int tab_order;
char *value;

/* External */
char *html =
    element_checkbox_html(
        appaserver_element_name(
            name,
            row_number )
                /* element_name */,
        string_initial_capital(
            prompt_display,
            prompt_string ),
        boolean checked,
        char *action_string,
        int tab_order,
        char *value );

#define ELEMENT_CHECKBOX_REMEMBER 1
```

ELEMENT_BREAK_TAG

```
/* Usage */
ELEMENT_BREAK_TAG *element_break_tag_new(
    void );

/* Process */
char *element_break_tag_html( void );

#define ELEMENT_BREAK_TAG_REMEMBER 0
```

ELEMENT_HIDDEN

```
/* Usage */
ELEMENT_HIDDEN *element_break_tag_new(
    void );

/* Process */
char *element_hidden_html(
    char *attribute_name,
    char *data,
    int row_number );
```

ELEMENT_NON_EDIT_TEXT

```
/* Usage */
ELEMENT_NON_EDIT_TEXT *
    element_non_edit_text_calloc(
        void );

/* Attributes */
char *name;
char *text_html;
int column_span;
int padding_em;

/* External */
char *element_non_edit_text_html(
    char *message );

char *heading =
    appaserver_element_heading_string(
        name );

#define ELEMENT_NON _EDIT_TEXT_REMEMBER 0
```

ELEMENT_TABLE_ROW

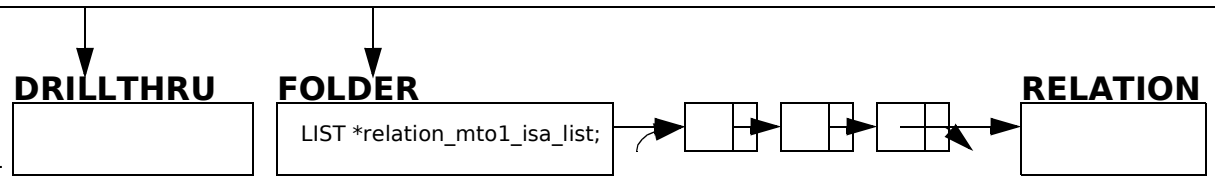
```
/* Usage */
ELEMENT_TABLE_ROW *element_table_row_calloc(
    void );

/* External */
char *element_table_row_html(
    void );

#define ELEMENT_TABLE_ROW_REMEMBER 0
```

ELEMENT_BUTTON

```
/* Usage */
ELEMENT_BUTTON *element_button_new(
    char *label,
    char *action_string,
    boolean with_table_data_tag );

/* Process */
char *element_button_html(
    char *label,
    char *action_string,
    boolean with_table_data_tag );

#define ELEMENT_BUTTON_REMEMBER 0
```

POST_CHOOSE_FOLDER

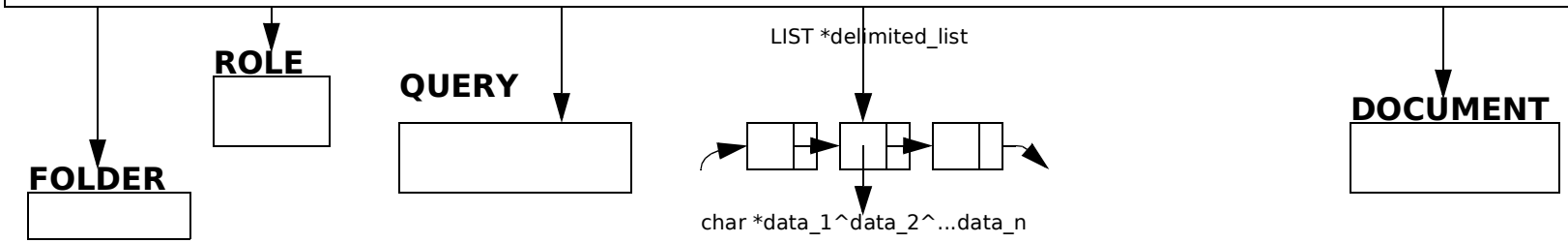
```
/* Usage */
POST_CHOOSE_FOLDER *post_choose_folder_fetch(
    /* See session_folder_integrity_exit() */
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *folder_name,
    char *state );

/* Process */
DRILLTHRU *drillthru =
    drillthru_start(
        folder_name );

folder_name =
    drillthru_participating_folder_name(
        folder_name,
        drillthru );

boolean post_choose_folder_fetch_relation_mto1_isa_list(
    char *state /* only state=insert */ );

FOLDER *folder =
    folder_fetch(
        folder_name,
        (char *)0 /* role_name */,
        (LIST *)0 /* exclude_attribute_name_list */,
        0 /* not fetch_folder_attribute_list */,
        0 /* not fetch_relation_mto1_non_isa_list */,
        post_choose_folder_fetch_relation_mto1_isa_list(),
        1 /* fetch_relation_one2m_list */,
        0 /* not fetch_relation_one2m_recursive_list */,
        0 /* not fetch_process */,
        0 /* not fetch_role_folder_list */,
        0 /* not fetch_row_level_restriction */,
        0 /* not fetch_role_operation_list */ );

LIST *relation_pair_one2m_list(
        folder->relation_one2m_list );

char *post_choose_folder_form_name(
    list_length( folder->relation_pair_one2m_list )
        /* relation_pair_one2m_list_length */,
    folder->folder_form );

boolean post_choose_folder_choose_isa_drop_down(
    list_length( folder->relation_mto1_isa_list )
        /* relation_mto1_isa_list_length */ );

boolean post_choose_folder_prompt_insert_form(
    post_choose_folder_form_name(),
    char *state );

boolean post_choose_folder_insert_table_form(
    post_choose_folder_form_name(),
    char *state );

boolean post_choose_folder_prompt_edit_form(
    post_choose_folder_form_name(),
    char *state,
    boolean drillthru->drillthru_participating );

boolean post_choose_folder_edit_table_form(
    post_choose_folder_form_name(),
    char *state );

char *post_choose_folder_target_frame(
    post_choose_folder_insert_table_form(),
    post_choose_folder_edit_table_form(),
    drillthru->drillthru_participating );

char *post_choose_folder_system_string(
    post_choose_folder_choose_isa_drop_down(),
    post_choose_folder_prompt_insert_form(),
    post_choose_folder_insert_table_form(),
    post_choose_folder_prompt_edit_form()
    post_choose_folder_edit_table_form(),
    application_name,
    login_name,
    session_key,
    folder_name,
    role_name,
    post_choose_folder_target_frame(),
    state,
    drillthru->drillthru_dictionary,
    list_first( folder->relation_mto1_isa_list )
        /* RELATION *first_one2m_isa_relation */ );
```



APPASERVER Data Structure

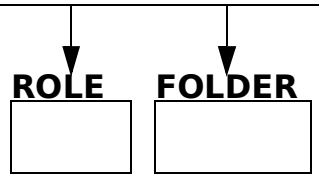
CHOOSE_ISA (Prompt)

```
/* Usage */
CHOOSE_ISA *choose_isa_prompt_fetch(
    /* See session_folder_integrity_exit() */
    char *application_name,
    char *session_key,
    char *login_name,
    char *folder_name,
    char *one2m_isa_folder_name,
    char *role_name );

/* Process */
FOLDER *folder =
    folder_fetch(
        one2m_isa_folder_name,
        (char *)0 /* role_name */,
        (LIST *)0 /* exclude_attribute_name_list */,
        /* Also sets folder_attribute_primary_list */
        /* and primary_key_list */
        1 /* fetch_folder_attribute_list */,
        0 /* not fetch_relation_mto1_non_isa_list */,
        0 /* not fetch_relation_mto1_isa_list */,
        0 /* not fetch_relation_one2m_list */,
        0 /* not fetch_relation_one2m_recursive_list */,
        1 /* fetch_process */,
        0 /* not fetch_role_folder_list */,
        1 /* fetch_row_level_restriction */,
        0 /* not fetch_role_operation_list */ );

ROLE *role =
    role_fetch(
        role_name,
        0 /* not fetch_attribute_exclude_list */ );

SECURITY_ENTITY *security_entity =
    security_entity_new(
        login_name,
        folder->non_owner_forbid,
        role->override_row_restrictions );

LIST *delimited_list;

if ( folder->populate_drop_down_process )
{
    delimited_list =
        list_pipe_fetch(
            process_choose_isa_command_line(
                folder->
                    populate_drop_down_process->
                    command_line,
                security_entity_where(
                    security_entity,
                    folder->folder_attribute_list ),
                application_name,
                login_name,
                role_name,
                one2m_isa_folder_name  ) );
}

if ( !folder->populate_drop_down_process )
{
    QUERY *query =
        query_isa_widget_new(
            one2m_isa_folder_name,
            folder->folder_attribute_primary_list,
            login_name,
            security_entity_where(
                security_entity,
                folder->folder_attribute_list ) );

    delimited_list =
        query_delimited_list(
            query->select_clause,
            query->from_clause,
            query->where_clause,
            query->order_clause,
            0 /* max_rows */ );
}

char *choose_isa_title( char *folder_name );

char *choose_isa_prompt_message(
    char *folder_name,
    char *one2m_isa_folder_name );

FOLDER_MENU *folder_menu =
    folder_menu_fetch(
        application_name,
        session_key,
        appaserver_parameter_file_data_directory(),
        role_name );

char *choose_isa_action_string(
    char *application_name,
    char *login_name,
    char *session_key,
    char *folder_name,
    char *one2m_isa_folder_name,
    char *role_name );

DOCUMENT *document =
    document_choose_isa(
        choose_isa_title(),
        choose_isa_rompt_message(),
        MENU *menu_fetch(
            application_name,
            login_name,
            session_key,
            role_name,
            FRAMESET_PROMPT_FRAME
                /* target_frame */,
            folder_menu->lookup_count_list ),
        LIST *folder->primary_key_list,
        LIST *query->delimited_list,
        choose_isa_action_string() );

/* External */
void document_choose_isa_output(
    stdout,
    DOCUMENT *document );
```



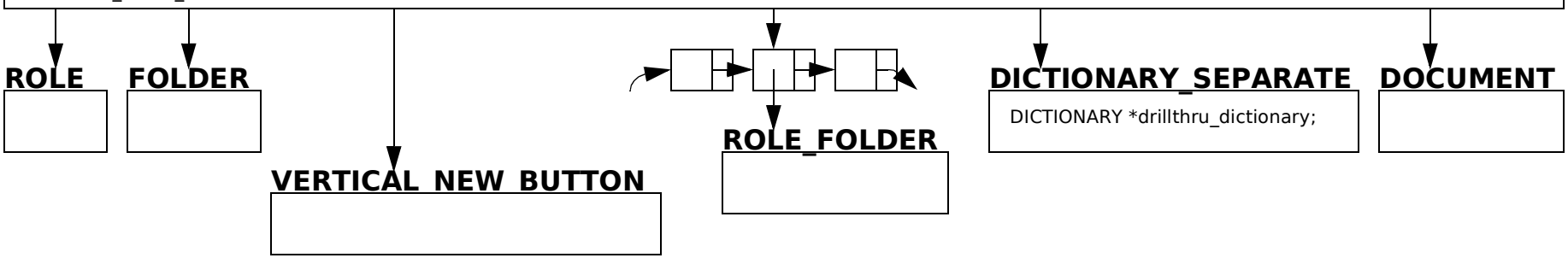
CHOOSE_ISA (Post)

```
/* Usage */
CHOOSE_ISA *choose_isa_post_fetch(
    /* See session_folder_integrity_exit() */
    char *application_name,
    char *session_key,
    char *login_name,
    char *folder_name,
    char *one2m_isa_folder_name,
    char *role_name );

/* Process */
ROLE *role =
    role_fetch(
        role_name );

FOLDER *folder =
    folder_fetch(
        one2m_isa_folder_name,
        (char *)0 /* role_name */,
        (LIST *)0 /* exclude_attribute_name_list */,
        /* Also sets primary_key_list */
        1 /* fetch_folder_attribute_list */,
        0 /* not fetch_relation_mto1_non_isa_list */,
        0 /* not fetch_relation_mto1_isa_list */,
        0 /* not fetch_relation_one2m_list */,
        0 /* not fetch_relation_one2m_recursive_list */,
        1 /* fetch_process */,
        0 /* not fetch_role_folder_list */,
        1 /* fetch_row_level_restriction */,
        0 /* not fetch_role_operation_list */ );

SECURITY_ENTITY *security_entity =
    security_entity_new(
        login_name,
        folder->non_owner_forbid,
        role->override_row_restrictions );

char *choose_isa_post_system_string(
    char *application_name,
```



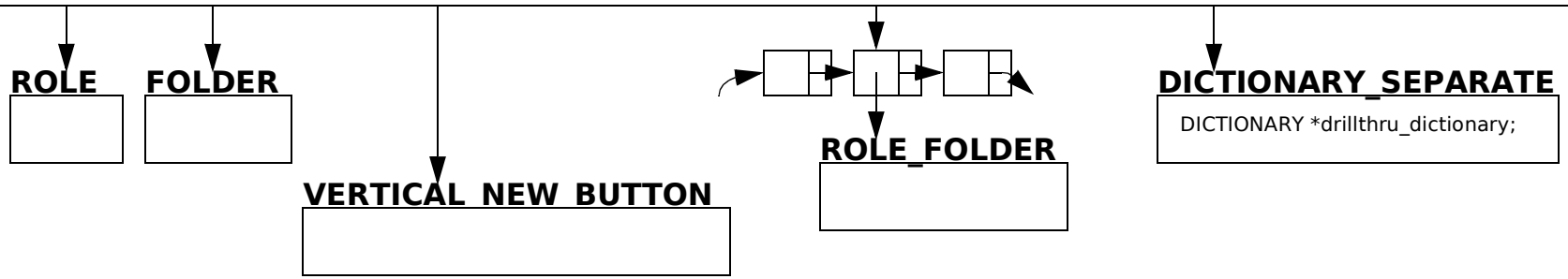
PROMPT_INSERT (Output)

```
/* Usage */
PROMPT_INSERT *
    prompt_insert_output(
        char *application_name,
        char *login_name,
        char *session_key,
        char *folder_name,
        char *role_name,
        char *state,
        char *target_frame,
        DICTIONARY_SEPARATE *dictionary_separate );

/* Process */
ROLE *role =
    role_fetch(
        role_name,
        1 /* fetch_attribute_exclude_list */ );

FOLDER *folder =
    folder_fetch(
        folder_name,
        role_exclude_insert_attribute_name_list(
            role->attribute_exclude_list ),
        /* Also sets primary_key_list */
        1 /* fetch_folder_attribute_list */,
        1 /* fetch_relation_mto1_non_isa_list */,
        /* Also sets folder_attribute_append_isa_list */
        1 /* fetch_relation_mto1_isa_list */,
        1 /* fetch_relation_one2m_list */,
        1 /* fetch_relation_one2m_recursive_list */,
        0 /* not fetch_process */,
        0 /* not fetch_role_folder_list */,
        0 /* not fetch_row_level_restriction */,
        0 /* not fetch_role_operation_list */ );

LIST *role_folder_list =
    role_folder_fetch_list(
        role_name,
        folder_name );

boolean prompt_insert_forbid(
    role_folder_insert(
        role_folder_list ) );
```



PROMPT_INSERT (Post)

```
/* Usage */
PROMPT_INSERT *
    prompt_insert_post_fetch(
        char *application_name,
        char *login_name,
        char *session,
        char *folder_name,
        char *role_name,
        char *state,
        char *target_frame,
        DICTIONARY_SEPARATE *dictionary_separate );

/* Process */
ROLE *role =
    role_fetch(
        role_name,
        1 /* fetch_attribute_exclude_list */ );

FOLDER *folder =
    folder_fetch(
        folder_name,
        role_exclude_insert_attribute_name_list(
            role->attribute_exclude_list ),
        /* Also sets primary_key_list */
        1 /* fetch_folder_attribute_list */,
        1 /* fetch_relation_mto1_non_isa_list */,
        /* Also sets folder_attribute_append_isa_list */
        1 /* fetch_relation_mto1_isa_list */,
        1 /* fetch_relation_one2m_list */,
        1 /* fetch_relation_one2m_recursive_list */,
        0 /* not fetch_process */,
        0 /* not fetch_role_folder_list */,
        0 /* not fetch_row_level_restriction */,
        0 /* not fetch_role_operation_list */ );

LIST *role_folder_list =
    role_folder_fetch_list(
        role_name,
        folder_name );

boolean prompt_insert_forbid(
    role_folder_insert(
        role_folder_list ) );
```



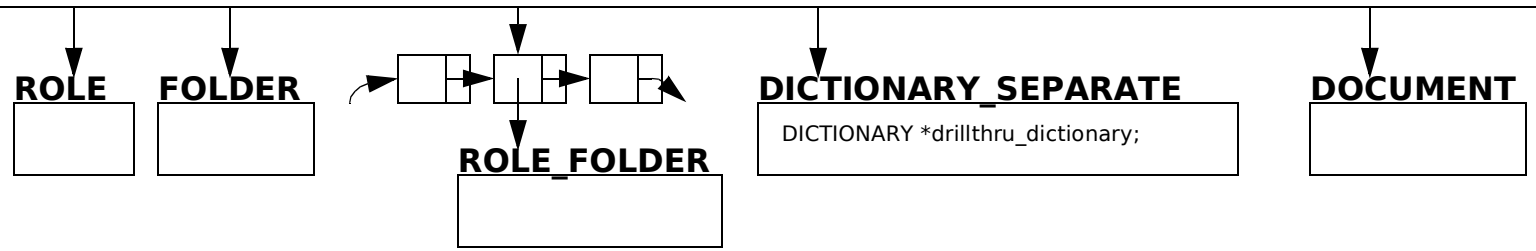
VERTICAL_NEW_BUTTON

```
/* Usage */
VERTICAL_NEW_BUTTON *vertical_new_button_calloc(
    void );

/* Attributes */
char *one_folder_name;
char *many_folder_name;

/* Output insert table form external */
char *vertical_new_button_one_folder_name(
    char *VERTICAL_NEW_BUTTON_ONE_PREFIX,
    DICTIONARY *non_prefixed_dictionary );

char *vertical_new_button_dictionary_folder_name(
    char *VERTICAL_NEW_BUTTON_MANY_HIDDEN_LABEL,
    DICTIONARY *non_prefixed_dictionary );

char *prompt_filename =
    char *vertical_new_button_blank_prompt_screen(
        char *application_name
        char *session_key,
        char *login_name,
        char *role_name,
        char *document_root_directory,
        char *appaserver_data_directory );

char *vertical_new_button_onload_control_string(
    char *prompt_filename );

void vertical_new_button_dictionary_set(
    DICTIONARY *non_prefixed_dictionary,
    char *VERTICAL_NEW_BUTTON_ONE_HIDDEN_LABEL,
    vertical_new_button->one_folder_name );

void vertical_new_button_dictionary_set(
    DICTIONARY *non_prefixed_dictionary,
    char *VERTICAL_NEW_BUTTON_MANY_HIDDEN_LABEL,
    vertical_new_button->many_folder_name );

/* Post prompt insert form external */
char *vertical_new_button_dictionary_one_folder_name(
    char *VERTICAL_NEW_BUTTON_ONE_PREFIX,
    DICTIONARY *non_prefixed_dictionary );

void vertical_new_button_dictionary_set(
    char *VERTICAL_NEW_BUTTON_ONE_HIDDEN_LABEL,
    vertical_new_button_dictionary_one_folder_name() );

void vertical_new_button_dictionary_set(
    DICTIONARY *non_prefixed_dictionary,
    char *folder_name );

system( “output_insert_table_form” ) && exit(0);

/* Post edit table form external */
char *many_folder_name =
    vertical_new_button_dictionary_folder_name(
        char *VERTICAL_NEW_BUTTON_MANY_HIDDEN_LABEL,
        DICTIONARY *non_prefixed_dictionary );

void post_state_insert( many_folder_name );

/* Related folder external */
char *vertical_new_button_one_element_name(
    char *VERTICAL_NEW_BUTTON_ONE_PREFIX,
    char *one_folder_name );

APPASERVER_ELEMENT *vertical_new_button_element(
    char *one_folder_name,
    LIST *role_folder_insert_list,
    char *form_name,
    char *vertical_new_button_one_prefix,
    char *vertical_new_button_display );
```

APPASERVER_ELEMENT

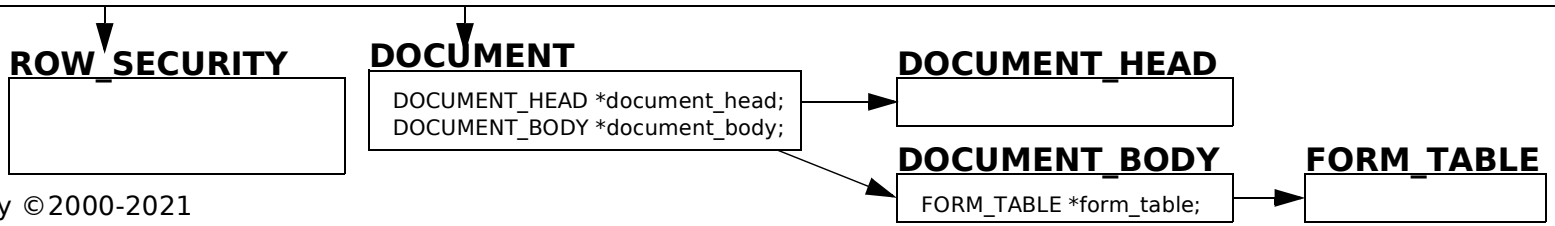
INSERT_TABLE

```
/* Usage */
INSERT_TABLE *
    insert_table_fetch(
        char *application_name,
        char *login_name,
        char *session_key,
        char *folder_name,
        char *role_name,
        char *target_frame,
        char *message,
        char *dictionary_string );

/* Process */
ROLE *role =
    role_fetch(
        role_name,
        1 /* fetch_attribute_exclude_list */ );

FOLDER *folder =
    folder_fetch(
        folder_name,
        role_exclude_insert_attribute_name_list(
            role->attribute_exclude_list ),
        /* Also sets primary_key_list */
        1 /* fetch_folder_attribute_list */,
        1 /* fetch_relation_mto1_non_isa_list */,
        /* Also sets folder_attribute_append_isa_list */
        1 /* fetch_relation_mto1_isa_list */,
        1 /* fetch_relation_one2m_list */,
        1 /* fetch_relation_one2m_recursive_list */,
        0 /* not fetch_process */,
        0 /* not fetch_role_folder_list */,
        0 /* not fetch_row_level_restriction */,
        0 /* not fetch_role_operation_list */ );

LIST *role_folder_list =
    role_folder_fetch_list(
        role_name,
        folder_name );

boolean insert_table_forbid(
    role_folder_insert(
        role_folder_list ) );

DICTIONARY_SEPARATE *dictionary_separate =
    dictionary_separate_string_new(
        dictionary_string );
```



EDIT_TABLE

```
/* Usage */
EDIT_TABLE *edit_table_new(
    char *application_name,
    char *login_name,
    char *session_key,
    char *folder_name,
    char *role_name,
    char *target_frame,
    DICTIONARY *query_dictionary,
    DICTIONARY *ignore_dictionary,
    DICTIONARY *non_prefixed_dictionary,
    DICTIONARY *query_dictionary,
    DICTIONARY *drillthru_dictionary,
    DICTIONARY *sort_dictionary,
    LIST *ignore_select_attribute_name_list,
    LIST *role_exclude_lookup_attribute_name_list(),
    LIST *role_exclude_lookup_attribute_name_list()  );

/* Process */
ROLE *role =
    role_fetch(
        role_name,
        1 /* fetch_attribute_exclude_list */ );

FOLDER *folder =
    folder_fetch(
        folder_name,
        role_name,
        role_exclude_lookup_attribute_name_list(
            role->attribute_exclude_list ),
        1 /* fetch_folder_attribute_list */,
        0 /* not fetch_relation_mto1_non_isa_list */,
        1 /* fetch_relation_mto1_isa_list */,
        0 /* not fetch_relation_one2m_list */,
        1 /* fetch_relation_one2m_recursive_list */,
        0 /* not fetch_process */,
        1 /* fetch_role_folder_list */,
        1 /* fetch_row_level_restriction */,
        1 /* fetch_role_operation_list */ );

LIST *folder->relation_join_one2m_list =
    relation_join_one2m_list(
        folder->relation_one2m_recursive_list,
        ignore_dictionary );

SECURITY_ENTITY *security_entity =
    security_entity_new(
        login_name,
        folder->non_owner_forbid,
        role->override_row_restrictions );

QUERY *query =
    query_edit_table(
        folder_name,
        security_entity,
        folder->relation_join_one2m_list,
        ignore_select_attribute_name_list,
        folder->folder_attribute_append_isa_list,
        query_dictionary,
        sort_dictionary );

char *edit_table_state(
    LIST *folder->role_folder_list() );

int edit_table_row_insert_count(
    DICTIONARY *non_prefixed_dictionary );

int edit_table_cell_update_count(
    DICTIONARY *non_prefixed_dictionary );

char *edit_table_cell_update_folder_list_string(
    DICTIONARY *non_prefixed_dictionary );

char *edit_table_results_string(
    DICTIONARY *non_prefixed_dictionary );

boolean edit_table_content_type( char *target_frame );
MENU *edit_table_menu_fetch( char *target_frame );

boolean edit_table_primary_keys_non_edit(
    list_length(
        folder->relation_mto1_isa_list ) );

ROW_SECURITY *row_security =
    row_security_edit_table(
        folder_name,
        folder->folder_attribute_append_isa_list,
        drillthru_dictionary,
        edit_table_primary_keys_non_edit(),
        folder->role_operation_list,
        ignore_select_attribute_name_list,
        edit_table_state(),
        role_exclude_lookup_attribute_name_list,
        role_exclude_lookup_attribute_name_list  );

char *edit_table_title(
    char *folder_name,
    edit_table_state() );

char *edit_table_message(
    int edit_table_row_insert_count(),
    int edit_table_cell_update_count(),
    char *edit_table_results_string() );

char *edit_table_action_string(
    char *application_name,
    char *login_name,
    char *session_key,
    char *folder_name,
    char *role_name,
    char *target_frame );

DOCUMENT *document =
    document_edit_table(
        edit_table_title(),
        edit_table_message(),
        folder_name,
        edit_table_menu_fetch(),
        folder->folder_attribute_isa_name_list,
        query->delimited_list,
        folder->no_initial_capital,
        edit_table_action_string(),
        row_security->regular_element_list,
        row_security->viewonly_element_list,
        ignore_dictionary,
        (DICTIONARY *)0 /* non_prefixed_dictionary */ );

/* External */
void edit_table_output( EDIT_TABLE *edit_table );
```



ROW_SECURITY

```
/* Usage */
ROW_SECURITY *row_security_edit_table(
    char *folder_name,
    LIST *folder->folder_attribute_append_isa_list,
    LIST *folder->relation_mto1_non_isa_list,
    LIST *folder->relation_join_one2m_list,
    DICTIONARY *drillthru_dictionary,
    boolean edit_table_primary_keys_non_edit(),
    LIST *folder->role_operation_list,
    LIST *ignore_select_attribute_name_list,
    char *edit_table_state(),
    LIST *role_exclude_update_attribute_name_list(),
    LIST *role_exclude_lookup_attribute_name_list() );

ROW_SECURITY *row_security_detail(
    application_name,
    where_attribute_data_list,
    folder,
    ignore_select_attribute_name_list );
```

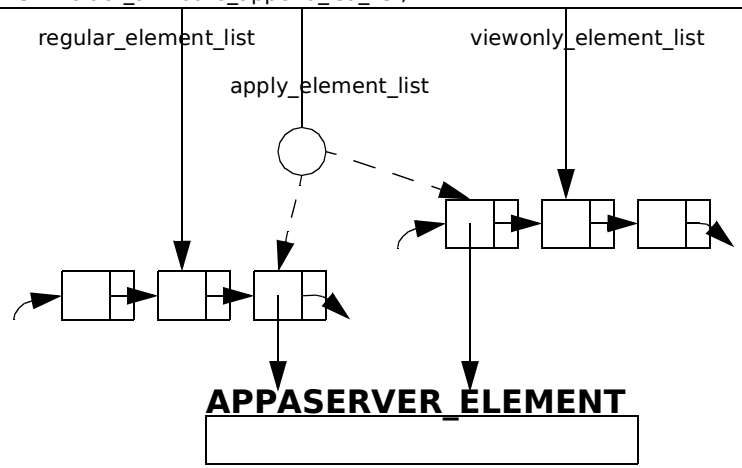
```
/* Process */
ROW_SECURITY_ROLE_UPDATE *row_security_role_update(
    char *folder_name,
    LIST *folder_attribute_primary_key_list( folder_attribute_append_isa_list ),
    boolean role_override_restrictions );

ROW_SECURITY_ELEMENT_LIST *row_security_element_list(
    LIST *folder_attribute_append_isa_list,
    LIST *relation_mto1_non_isa_list,
    LIST *relation_join_one2m_list,
    DICTIONARY *drillthru_dictionary,
    boolean primary_keys_non_edit,
    LIST *role_operation_list,
    LIST *ignore_select_attribute_name_list,
    char *state,
    LIST *role_exclude_update_attribute_name_list,
    LIST *role_exclude_lookup_attribute_name_list,
    ROW_SECURITY_ROLE_UPDATE *row_security_role_update() );
```

ROW_SECURITY_ROLE_UPDATE

ROW_SECURITY_ELEMENT_LIST

ROW_SECURITY_ELEMENT_LIST

```
/* Usage */
ROW_SECURITY_ELEMENT_LIST *row_security_element_list(
    LIST *folder_attribute_append_isa_list,
    LIST *relation_mto1_non_isa_list,
    LIST *relation_join_one2m_list,
    DICTIONARY *drillthru_dictionary,
    boolean primary_keys_non_edit,
    LIST *role_operation_list,
    LIST *ignore_select_attribute_name_list,
    char *state,
    LIST *role_exclude_update_attribute_name_list,
    LIST *role_exclude_lookup_attribute_name_list,
    ROW_SECURITY_ROLE_UPDATE *row_security_role_update() );

/* Process */
LIST *row_security_update_state_regular_element_list(
    LIST *folder_attribute_append_isa_list,
    LIST *relation_mto1_non_isa_list,
    LIST *relation_join_one2m_list,
    DICTIONARY *drillthru_dictionary,
    boolean primary_keys_non_edit,
    LIST *role_operation_list,
    LIST *ignore_select_attribute_name_list,
    LIST *role_exclude_update_attribute_name_list,
    LIST *role_exclude_lookup_attribute_name_list,
    ROW_SECURITY_ROLE_UPDATE *role_update );

LIST *row_security_update_state_viewonly_element_list(
    LIST *folder_attribute_append_isa_list,
    LIST *relation_mto1_non_isa_list,
    LIST *relation_join_one2m_list,
    LIST *role_operation_list,
    LIST *ignore_select_attribute_name_list,
    LIST *role_exclude_lookup_attribute_name_list,
    ROW_SECURITY_ROLE_UPDATE *role_update );

LIST *row_security_viewonly_element_list(

/* External */
LIST *row_security_apply_element_list(
    LIST *regular_element_list,
    LIST *viewonly_element_list,
    char *delimited_string,
    ROW_SECURITY_ROLE_UPDATE *role_update );
```



ROW_SECURITY_ROLE_UPDATE

```
/* Usage */
ROW_SECURITY_ROLE_UPDATE *row_security_role_update_new(
    char *folder_name,
    LIST *primary_key_list,
    boolean role_override_restrictions );

/* Process */
ROW_SECURITY_ROLE_UPDATE_FOLDER  *
    row_security_role_update_folder =
        row_security_role_update_folder_fetch(
            folder_name
                /* one_folder_name */,
            primary_key_list );

boolean row_security_role_update_viewonly(
    boolean role_override_restrictions );

char *row_security_role_update_system_string(
    char *folder_name );
```

ROW_SECURITY_ROLE_UPDATE_FOLDER

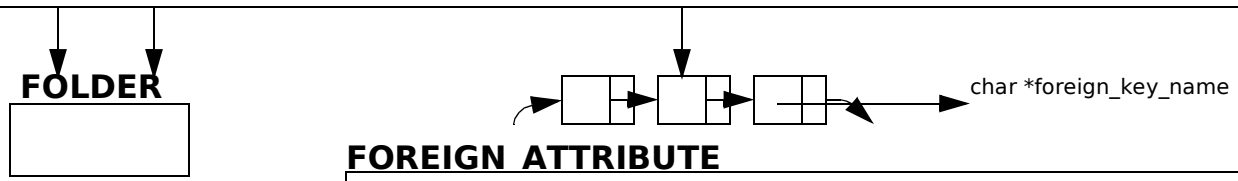
```
/* Usage */
ROW_SECURITY_ROLE_UPDATE_FOLDER *
    row_security_role_update_folder_fetch(
        char *folder_name,
        LIST *primary_key_list );

/* Attributes */
char *one_folder_name;
char *attribute_not_null;

/* Process */
LIST *relation_one2m_recursive_list(
    (LIST *)0 /* relation_list */,
    char *one_folder_name,
    0 /* not fetch_process */ );

boolean row_security_role_update_folder_participating(
    char *folder_name,
    char *one_folder_name,
    LIST *relation_one2m_recursive_list() );

char *row_security_role_update_folder_join(
    char *folder_name,
    char *one_folder_name,
    LIST *primary_key_list );
```

ROW_SECURITY (Process)

```
LIST *regular_element_list;
LIST *viewonly_element_list;
LIST *apply_element_list;
char *mto1_where_join;
LIST *row_dictionary_list;
LIST *ajax_relation_list;
```

RELATION

```
/* Usage *
RELATION *relation_new(
    char *many_folder_name,
    char *one_folder_name );

/* Attributes */
char *many_folder_name;
char *one_folder_name;
char *related_attribute_name;
int pair_one2m_order;
boolean omit_one2m_detail;
boolean prompt_mto1_recursive;
boolean relation_type_isa;
boolean copy_common_attributes;
boolean automatic_preselection;
boolean drop_down_multi_select;
boolean join_one2m_each_row;
boolean ajax_fill_drop_down;
char *hint_message;
FOLDER *many_folder;
FOLDER *one_folder;

/* External */
char *relation_system_string(
    char *where,
    char *order );

LIST *relation_system_list(
    char *relation_system_string(),
    /* Setting both will set foreign_key_list */
    boolean fetch_folder,
    boolean fetch_attribute_list,
    boolean fetch_process );

RELATION *relation_parse(
    char *input,
    boolean fetch_folder,
    boolean fetch_folder_attribute_list,
    boolean fetch_process );

LIST *relation_one2m_pair_list(
    char *one_folder_name );

LIST *relation_mto1_non_isa_list(
    char *many_folder_name,
    boolean fetch_process );

LIST *relation_mto1_primary_key_subset_list(
    char *many_folder_name,
    LIST *primary_key_list );

LIST *relation_one2m_list(
    char *one_folder_name );

LIST *relation_pair_one2m_list(
    relation_one2m_list() );

LIST *relation_join_one2m_list(
    relation_one2m_recursive_list(),
    DICTIONARY *ignore_dictionary );

LIST *relation_one2m_recursive_list(
    LIST *relation_list /* in/out */,
    char *one_folder_name,
    boolean fetch_process );

LIST *relation_mto1_isa_list(
    LIST *relation_list /* in/out */,
    char *many_folder_name,
    boolean fetch_process );

LIST *relation_mto1_drillthru_list(
    LIST *relation_list /* in/out */,
    char *base_folder_name,
    LIST *fulfilled_folder_name_list );

LIST *relation_foreign_key_list(
    folder_attribute_primary_key_list(
        one_folder->
            folder_attribute_list )
                /* primary_foreign_key_list */,
    related_attribute_name,
    foreign_attribute_name_list(
        foreign_attribute_list(
            many_folder->folder_name,
            one_folder->folder_name ) );

boolean consumes_taken;

RELATION *relation_consumes(
    LIST *many_attribute_name,
    relation_mto1_non_isa_list() );

boolean relation_is_primary_key_subset(
    LIST *foreign_key_list,
    LIST *many_primary_key_list );
```



FOREIGN_ATTRIBUTE

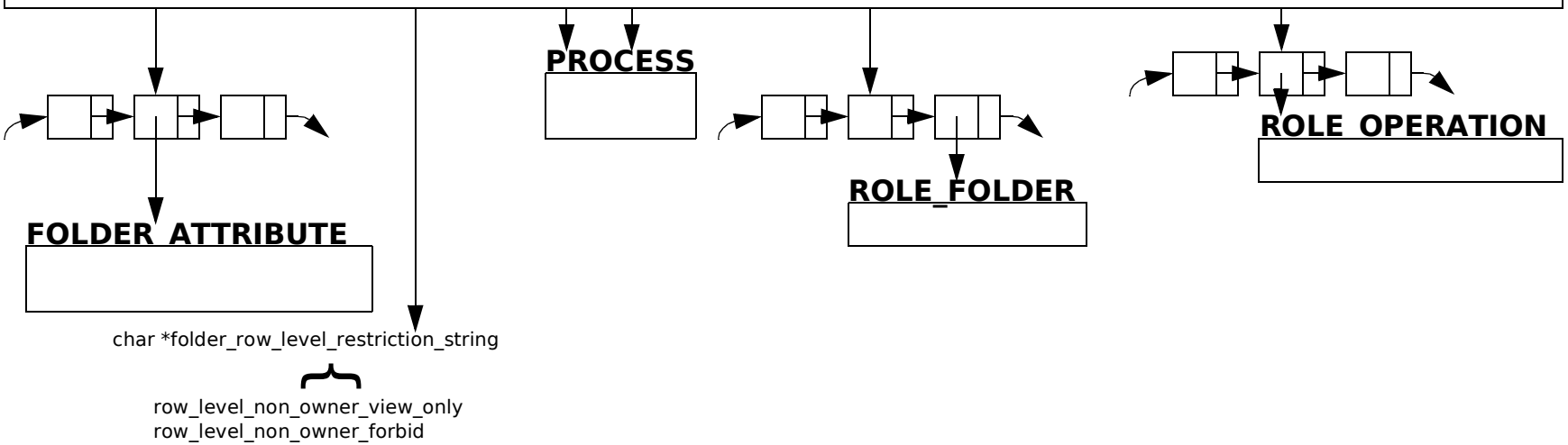
```
/* Usage */
LIST *foreign_attribute_list(
    char *many_folder_name,
    char *one_folder_name );

/* Input */
char *folder;
char *related_folder;
char *related_attribute;
char *foreign_attribute;
int foreign_key_index;

/* Process */
LIST *foreign_attribute_name_list(
    foreign_attribute_list() );
```

APPASERVER Data Structure

FOLDER (Usage)

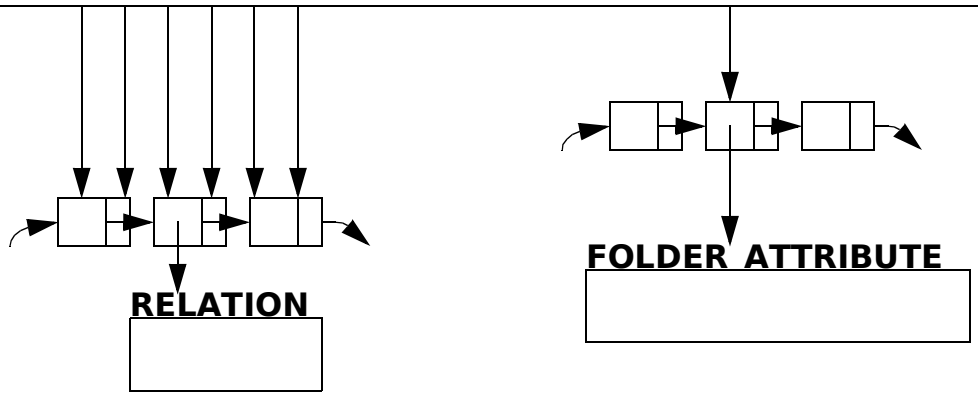
```
/* Usage */
FOLDER *folder_fetch(
    security_sql_injection_escape(
        char *folder_name ),
    /* If fetching role_folder_list */
    security_sql_injection_escape(
        char *role_name ),
    role_exclude_..._attribute_name_list(),
    /* Also sets primary_attribute_list */
    /* and primary_key_list */
    boolean fetch_folder_attribute_list,
    boolean fetch_relation_mto1_non_isa_list,
    /* Maybe sets folder_attribute_append_isa_list */
    boolean fetch_relation_mto1_isa_list,
    boolean fetch_relation_one2m_list,
    boolean fetch_relation_one2m_recursive_list,
    boolean fetch_process,
    boolean fetch_role_folder_list,
    boolean fetch_row_level_restriction,
    boolean fetch_role_operation_list );

FOLDER *folder_quick_fetch(
    char *folder_name );
```

```
/* Attributes */
char *folder_name;
char *folder_form;
int insert_rows_number;
boolean lookup_email_output;
char *notepad;
char *populate_drop_down_process_name;
char *post_change_process_name;
char *html_help_file_anchor;
char *post_change_javascript;
char *subschema;
boolean exclude_application_export;
boolean drillthru;
boolean no_initial_capital;
char *index_directory;
char *data_directory;
char *create_view_statement;
char *javascript_filename;

/* Process */
PROCESS *populate_drop_down_process;
PROCESS *post_change_process;

char *folder_primary_where(
    char *folder_name );

LIST *folder_attribute_list(
    folder_name,
    role_exclude_..._attribute_name_list() );

LIST *folder_attribute_primary_key_list(
        folder_attribute_list() );

LIST *role_folder_list =
    role_folder_fetch_list(
        sql_injection_escape_role_name,
        sql_injection_escape_folder_name ) );

LIST *role_operation_list(
    role_name,
    folder_name );
```



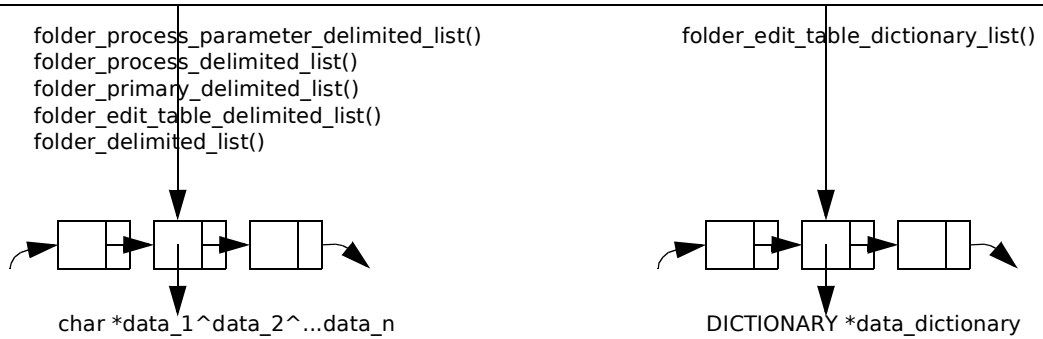
FOLDER (Relation)

```
/* Relation */
LIST *relation_mto1_isa_list(
    (LIST *)0 /* relation_list */,
    char *folder_name,
    boolean fetch_process );

LIST *relation_mto1_non_isa_list(
    char *folder_name );

LIST *relation_one2m_list(
    folder_name );

LIST *relation_pair_one2m_list(
    relation_one2m_list() );

LIST *relation_one2m_recursive_list(
    (LIST *)0 /* one2m_recursive_list */,
    char *folder_name,
    boolean fetch_process );

LIST *relation_join_one2m_list(
    relation_one2m_recursive_list(),
    DICTIONARY *ignore_dictionary );

LIST *folder_attribute_append_isa_list(
    list_copy( folder_attribute_fetch_list() ),
    relation_mto1_isa_list() );
```



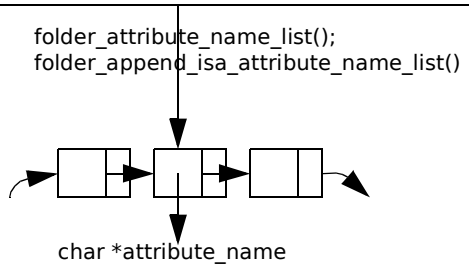
FOLDER (Data)

```
/* Data */
FOLDER *folder_process_delimited_fetch(
    char *folder_name,
    char *populate_drop_down_process_name,
    char *one2m_folder_name,
    char *state,
    char *login_name,
    char *role_name,
    DICTIONARY *drillthru_dictionary );

ROLE *role =
    role_fetch(
        role_name );

FOLDER *folder =
    folder_fetch(
        char *folder_name,
        (char *)0 /* role_name */ ),
        (LIST *)0 /* role_exclude_attribute_name_list */,
        /* Also sets primary_key_list */
        1 /* fetch_folder_attribute_list */,
        1 /* fetch_relation_mto1_non_isa_list */,
        0 /* not fetch_relation_mto1_isa_list */,
        0 /* not fetch_relation_one2m_list */,
        0 /* not fetch_relation_one2m_recursive_list */,
        0 /* not fetch_process */,
        0 /* not fetch_role_folder_list */,
        1 /* fetch_row_level_restriction */,
        0 /* not fetch_role_operation_list */ );

SECURITY_ENTITY *security_entity =
    security_entity_new(
        login_name,
        folder->non_owner_forbid,
        role->override_row_restrictions );

LIST *folder_process_delimited_list(
    char *populate_drop_down_process_name,
    SECURITY_ENTITY *security_entity_new(),
    char *one2m_folder_name,
    char *state,
    DICTIONARY *drillthru_dictionary );

char *folder_table_name(
    char *environment_application_name(),
    char *folder_name );

LIST *folder_primary_delimited_list(
    char *folder_name,
    folder_attribute_primary_key_list(),
    folder_attribute_list(),
    relation_mto1_non_isa_list(),
    SECURITY_ENTITY *security_entity_new(),
    DICTIONARY *drillthru_dictionary,
    char *login_name );

LIST *folder_edit_table_delimited_list(
    char *folder_table_name(),
    LIST *folder_attribute_append_isa_list(),
    LIST *relation_mto1_isa_list(),
    DICTIONARY *query_dictionary );

LIST *folder_delimited_list(
    char *folder_table_name(),
    LIST *attribute_name_list,
    char *where_clause );

LIST *folder_edit_table_dictionary_list(
    char *folder_table_name(),
    LIST *folder_attribute_append_isa_list(),
    LIST *relation_mto1_isa_list(),
    DICTIONARY *query_dictionary );
```



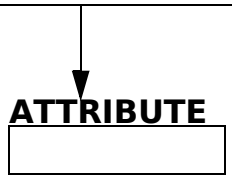
FOLDER (Generic)

```
/* Generic */
long int folder_row_count(
    char *folder_table_name() );

char *folder_row_level_restriction_string(
    sql_injection_escape_folder_name );

boolean folder_non_owner_view_only(
    folder_row_level_restriction_string() );

boolean folder_non_owner_forbid(
    folder_row_level_restriction_string() );

boolean non_owner_view_only =
    folder_row_level_restrictions_non_owner_view_only(
        row_level_restriction_string );

boolean non_owner_forbid =
    folder_row_level_restrictions_non_owner_forbid(
        row_level_restriction_string );

LIST *folder_attribute_name_list(
    folder_attribute_list() );

LIST *folder_append_isa_attribute_name_list(
    folder_attribute_append_isa_list() );

LIST *attribute_float_list =
    folder_attribute_extract_float_list(
        folder_attribute_append_isa_list );

/* Unknown */
LIST *mto1_copy_common_related_folder_list;
LIST *copy_common_attribute_list;
LIST *append_isa_copy_folder_attribute_list;
LIST *copy_common_attribute_name_list;
LIST *append_isa_copy_attribute_name_list;
LIST *lookup_attribute_exclude_name_list;
```



FOLDER_ATTRIBUTE

```
/* Usage */
LIST *folder_attribute_fetch_list(
    char *security_sql_injection_escape(
        folder_name ),
    LIST *exclude_attribute_name_list );

/* Attributes */
char *folder_name;
char *attribute_name;
int primary_key_index;
int display_order;
boolean omit_insert;
boolean omit_insert_prompt;
boolean omit_update;
boolean additional_unique_index;
boolean additional_index;
boolean insert_required;
boolean lookup_required;

/* Process */
ATTRIBUTE *attribute =
    attribute_fetch(
        attribute_name );

char *folder_attribute_primary_where(
    char *folder_name,
    char *attribute_name );

char *folder_attribute_where(
    char *sql_injection_escape_folder_name,
    LIST *exclude_attribute_name_list );

char *folder_attribute_system_string(
    folder_attribute_where() );

LIST *folder_attribute_system_list(
    folder_attribute_system_string() );

FOLDER_ATTRIBUTE *folder_attribute_parse(
    char *input );

/* External */
LIST *folder_attribute_append_isa_list(
    LIST *folder_attribute_list(),
    LIST *relation_mto1_isa_list );

LIST *folder_attribute_name_list(
    folder_attribute_list() |
    folder_attribute_append_isa_list(),
    char *folder_name );

LIST *folder_attribute_primary_list(
    folder_attribute_list() );

LIST *folder_attribute_primary_key_list(
    folder_attribute_list() );

LIST *folder_attribute_number_list(
    folder_attribute_list() );

LIST *folder_attribute_non_primary_key_list(
    folder_attribute_list() );

LIST *folder_attribute_date_name_list(
    folder_attribute_list() );

LIST *folder_attribute_time_name_list(
    folder_attribute_list() );

LIST *folder_attribute_date_time_name_list(
    folder_attribute_list() );

boolean folder_attribute_exists(
    char *folder_name,
    char *attribute_name );

FOLDER_ATTRIBUTE *folder_attribute_list_seek(
    char *attribute_name,
    LIST *folder_attribute_list );

char *folder_attribute_sort_attribute_name(
    LIST *folder_attribute_list );
```

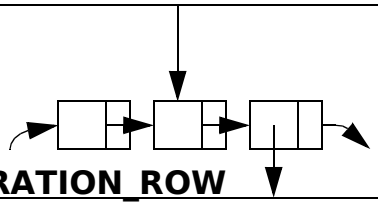


ATTRIBUTE

```
/* Usage */
LIST *attribute_fetch(
    char *attribute_name );

/* Attributes */
char *attribute_name;
char *datatype_name;
int width;
int float_decimal_places;
char *hint_message;
char *post_change_javascript;
char *on_focus_javascript_function;
boolean lookup_histogram_output;
boolean lookup_time_chart_output;

/* External */
static LIST *attribute_list( void );

LIST *attribute_seek(
    char *attribute_name,
    LIST *attribute_list );

boolean attribute_exists(
    char *attribute_name,
    LIST * attribute_list );

LIST *attribute_extract_float_list(
    LIST *attribute_list );

boolean attribute_is_date(
    char *datatype_name );

boolean attribute_is_time(
    char *datatype_name );

boolean attribute_is_number(
    char *datatype_name );

boolean attribute_is_date_time(
    char *datatype_name );

char *attribute_full_attribute_name(
    char *folder_name,
    char *attribute_name );

boolean attribute_is_yes_no(
    char *attribute_name );
```

OPERATION

```
/* Usage */
OPERATION *operation_new(
    char *operation_name,
    row_security_role_update_view_only() );

/* Input */
boolean role_update_view_only;

/* Attributes */
char *operation_name;
boolean output;

/* Process */
OPERATION *operation_fetch(
    char *operation_name );

PROCESS *process =
    process_fetch(
        operation_name,
        (char *)0 /* document_root_directory */,
        (char *)0 /* relative_source_directory */,
        1 /* check_executable_inside_filesystem */ );

boolean operation_delete_name(
    char *operation_name );

boolean operation_delete_placeholder(
    role_update_view_only,
    operation_delete_name() );

char *operation_label(
    char *operation_name,
    operation_delete_placeholder() );

/* Generic */
int operation_row_total(
    DICTIONARY *row_dictionary,
    char *operation_name,
    int highest_index );

LIST *operation_row_list(
    DICTIONARY *row_dictionary,
    char *operation_name,
    LIST *primary_key_list,
    LIST *attribute_name_list );
```



OPERATION_ROW

```
/* Usage */
OPERATION_ROW *operation_row_fetch(
    DICTIONARY *row_dictionary,
    char *operation_name,
    LIST *primary_key_list,
    LIST *attribute_name_list,
    int row_number );

/* Process */
OPERATION_ROW *operation_row_calloc(
    void );

boolean operation_row_checked(
    DICTIONARY *row_dictionary,
    char *operation_name,
    int row_number );

LIST *operation_row_primary_data_list(
    DICTIONARY *row_dictionary,
    LIST *primary_key_list,
    int row_number );

DICTIONARY *operation_single_row_dictionary(
    DICTIONARY *row_dictionary,
    LIST *attribute_name_list,
    int row_number );

/* Generic */

void operation_row_perform(
    process_operation_command_line(
        operation->process->command_line,
        environment_application_name(),
        operation_name,
        login_name,
        role_name,
        folder_name,
        getpid() /* parent_process_id */,
        session_key,
        operation_row_total(),
        operation_row_primary_data_list(),
        operation_single_row_dictionary() ) );
```

OPERATION_SEMAPHORE

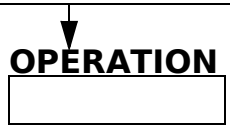
```
/* Usage */
OPERATION_SEMAPHORE *operation_semaphore_fetch(
    char *operation_name,
    char *appaserver_data_directory,
    pid_t parent_process_id,
    int operation_row_total() );

/* Process */
char *operation_semaphore_filename(
    char *operation_name,
    char *appaserver_data_directory,
    pid_t parent_process_id );

boolean operation_semaphore_group_first_time(
    operation_semaphore_filename() );

void operation_semaphore_initialize_file(
    operation_semaphore_filename() );

int operation_semaphore_row_current(
    operation_semaphore_filename() );

boolean operation_semaphore_group_last_time(
    operation_semaphore_row_current(),
    operation_row_total );

void operation_semaphore_increment(
    operation_semaphore_filename(),
    operation_semaphore_row_current() );

void operation_semaphore_remove_file(
    char *operation_semaphore_filename() );
```

SEMAPHORE

```
/* Usage */
SEMAPHORE *semaphore_new(
        key_t semaphore_key );

/* Process */
int semaphore_id(
        semaphore_key );

void semaphore_wait(
        semaphore_id() );

void semaphore_signal(
        semaphore_id() );
```

ROLE_OPERATION

```
/* Usage */
LIST *role_operation_list(
    char *folder_name,
    char *role_name );

/* Attributes */
char *folder_name;
char *role_name;
char *operation_name;

/* Process */
ROLE_OPERATION *role_operation_new(
    char *folder_name,
    char *role_name,
    char *operation_name );

char *role_operation_primary_where(
    char *folder_name,
    char *role_name,
    char *operation_name );

char *role_operation_system_string(
    char *where );

LIST *role_operation_system_list(
    role_operation_system_string() );

ROLE_OPERATION *role_operation_parse(
    char *input );

/* Generic */
OPERATION *operation =
    operation_new(
        operation_name );

LIST *role_operation_name_list(
    role_operation_list() );

OPERATION *role_operation_getset(
    LIST *operation_list,
    LIST *role_operation_list,
    char *operation_name );

boolean role_operation_delete(
    role_operation_list() );
```



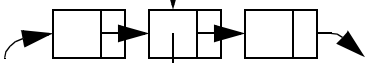
QUERY

```
/* Usage */
QUERY *query_calloc(
    void );

/* Attributes */
DICTIONARY *drillthru_dictionary;
DICTIONARY *query_dictionary;
LIST *where_attribute_data_list;
char *sql_injection_escape_folder_name;
char *sql_injection_escape_role_name;
char *sql_injection_escape_login_name;
LIST *ignore_select_attribute_name_list;
char *attribute_not_null_join;
char *attribute_not_null_folder_name;
DICTIONARY *sort_dictionary;

/* External */
char *query_yes_no_operator_name(
    char *attribute_name );

char *query_operator_name(
    char *operator_key,
    DICTIONARY *dictionary );

enum relational_operator query_relational_operator(
    query_operator_name(),
    char *datatype_name );

char *query_sort_prefix_direction_attribute_index(
    DICTIONARY *sort_dictionary );

char *query_attribute_name_list_order(
    char *folder_name,
    LIST *primary_key_list,
    LIST *folder_attribute_append_isa_list,
    char *descending_label );

LIST *query_common_attribute_name_list();

char *query_operator_character_string(
    char *operator_string );

char *query_data_where(
    char *folder_name,
    LIST *where_attribute_name_list,
    LIST *where_attribute_data_list,
    LIST *folder_attribute_list );
```

```
enum
relational_operator {
    equals
    not_equal
    not_equal_or_null
    less_than
    less_than_equal_to
    greater_than
    greater_than_equal_to
    between
    begins
    contains
    not_contains
    query_or
    is_null
    not_null
    relational_operator_empty
    relational_operator_unknown
```

QUERY (ISA WIDGET)

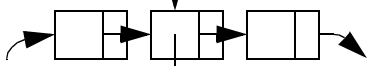
```
/* Usage */
QUERY *query_isa_widget_new(
    char *one2m_isa_folder_name,
    LIST *folder_attribute_primary_key_list(),
    LIST *folder_attribute_primary_list(),
    char *login_name,
    char *security_entity_where() );

/* Input */
char *one2m_isa_folder_name;
LIST *primary_key_list;
LIST *folder_attribute_primary_list;
char *login_name,
char *security_entity_where;

/* Process */
LIST *select_list =
    query_primary_select_list(
        primary_key_list,
        folder_attribute_primary_list,
        query_date_convert_new(
            login_name ) );

char *select_clause =
    query_select_clause(
        select_list );

char *from_clause = one2m_isa_folder_name;

char *where_clause = security_entity_where;

char *order_clause = select_clause;
```



QUERY_SELECT

QUERY (WIDGET)

```
/* Usage */
QUERY *query_widget_new(
    char *widget_folder_name,
    char *login_name,
    LIST *folder_attribute_primary_key_list(),
    LIST *folder_attribute_list(),
    LIST *relation_mto1_non_isa_list,
    char *security_entity_where(),
    DICTIONARY *drillthru_dictionary );

/* Input */
char *widget_folder_name;
char *login_name;
LIST *primary_key_list;
LIST *folder_attribute_list;
LIST *relation_mto1_non_isa_list;
char *security_entity_where;
DICTIONARY *drillthru_dictionary;

/* Process */
LIST *select_list =
    query_primary_select_list(
        primary_key_list,
        folder_attribute_list,
        query_date_convert_new(
            login_name ) );

char *select_clause =
    query_select_clause(
        select_list );

char *from_clause = widget_folder_name;

QUERY_WIDGET_WHERE *query_widget_where =
    query_widget_where_new(
        widget_folder_name,
        folder_attribute_list,
        relation_mto1_non_isa_list,
        security_entity_where,
        drillthru_dictionary );

char *where_clause =
    query_widget_where->
        where_clause;

char *order_clause = select_clause;
```



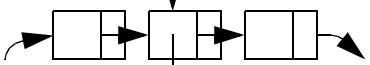
QUERY_SELECT

QUERY_WIDGET_WHERE

APPASERVER Data Structure

QUERY (Edit Table)

```
/* Usage */
QUERY *query_edit_table(
    char *folder_name,
    SECURITY_ENTITY *security_entity,
    LIST *relation_join_one2m_list,
    LIST *ignore_select_attribute_name_list,
    LIST *folder_attribute_append_isa_list,
    LIST *relation_mto1_isa_list,
    DICTIONARY *query_dictionary,
    DICTIONARY *sort_dictionary );

/* Process */
LIST *select_list =
    query_edit_table_select_list(
        LIST *folder_attribute_append_isa_list,
        LIST *ignore_select_attribute_name_list,
        QUERY_DATE_CONVERT *
            query_date_convert_new(
                security_entity->login_name ) );

char *select_clause =
    query_select_clause(
        select_list );

char *from_clause =
    query_from_clause(
        folder_name,
        relation_mto1_isa_list );

QUERY_WHERE *query_where =
    query_edit_table_where(
        folder_attribute_append_isa_list,
        relation_mto1_isa_list,
        security_entity_where(),
        query_dictionary );

char *where_clause =
    query_where->
        where_clause;

char *order_clause =
    query_order_clause(
        select_clause,
        sort_dictionary );
```



QUERY_SELECT

QUERY_WHERE

QUERY (DETAIL)

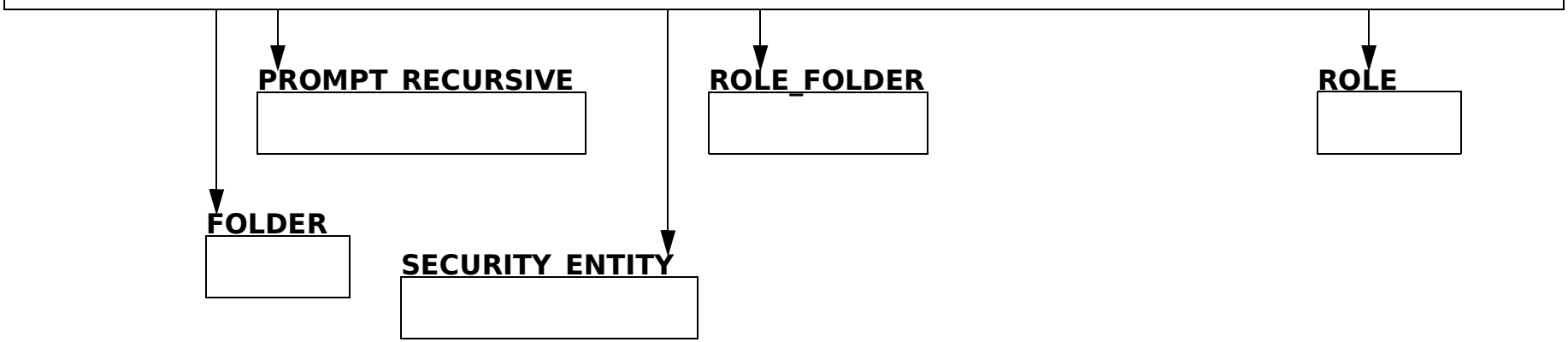
```
/* Usage */
QUERY *query_detail_new(
    LIST *where_attribute_data_list,
    char *security_sql_injection_escape(
        folder_name ),
    char *security_sql_injection_escape(
        role_name ),
    char *security_sql_injection_escape(
        login_name ),
    LIST *dictionary_separate->
        ignore_select_attribute_name_list,
    char *attribute_not_null_join,
    char *attribute_not_null_folder_name );

QUERY *query_search_replace_where_new(
    DICTIONARY *drillthru_dictionary,
    LIST *append_isa_attribute_list,
    LIST *relation_mto1_isa_list,
    boolean non_owner_forbid,
    SECURITY_ENTITY *security_entity );
```

QUERY (COMMAND_LINE_WHERE)

```
/* Usage */
QUERY *query_command_line_where_new(
    DICTIONARY *drillthru_dictionary,
    LIST *append_isa_attribute_list,
    LIST *relation_mto1_isa_list,
    boolean non_owner_forbid,
    SECURITY_ENTITY *security_entity );
```

QUERY (Process)

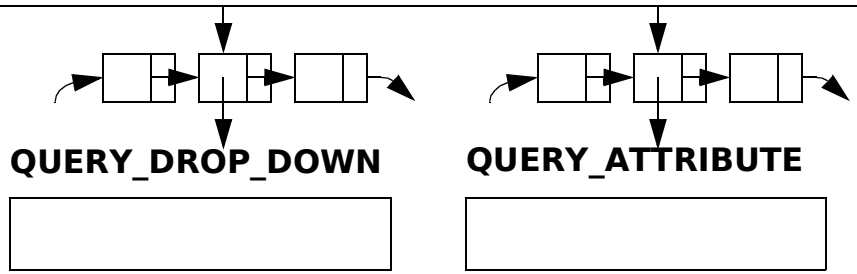
```
/* Process */
int max_rows = QUERY_MAX_ROWS;

ROLE *role =
    role_fetch(
        role_name,
        1 /* fetch_attribute_exclude_list */ );

FOLDER *query_folder =
    folder_fetch(
        sql_injection_escape_folder_name,
        sql_injection_escape_role_name,
        exclude_lookup_attribute_name_list,
        /* Also sets primary_key_list */
        1 /* fetch_folder_attribute_list */,
        0 /* not fetch_relation_mto1_non_isa_list */,
        /* Also sets folder_attribute_append_isa_list */
        1 /* fetch_relation_mto1_isa_list */,
        0 /* not fetch_relation_one2m_list */,
        0 /* not fetch_relation_one2m_recursive_list */,
        1 /* fetch_process */,
        1 /* fetch_role_folder_list */,
        1 /* fetch_row_level_restriction */,
        0 /* not fetch_role_operation_list */ );

FOLDER *search_replace_where_folder =
    folder_fetch(
        folder_name,
        role_name,
        (LIST *)0 /* exclude_lookup_..._name_list */,
        /* Also sets primary_key_list */
        1 /* fetch_folder_attribute_list */,
        0 /* not fetch_relation_mto1_non_isa_list */,
        /* Also sets folder_attribute_append_isa_list */
        1 /* fetch_relation_mto1_isa_list */,
        0 /* not fetch_relation_one2m_list */,
        0 /* not fetch_relation_one2m_recursive_list */,
        0 /* not fetch_process */,
        0 /* not fetch_role_folder_list */,
        1 /* fetch_row_level_restriction */,
        0 /* not fetch_role_operation_list */ );

SECURITY_ENTITY *security_entity =
    security_entity_new(
        login_name,
        query_folder->non_owner_forbid,
        role->override_row_restrictions );

PROMPT_RECURSIVE *prompt_recursive =
    prompt_recursive_new(
        query_folder,
        query_folder->mto1_related_folder_list );

ROLE_FOLDER *role_folder =
    role_folder_fetch(
        role_name,
        folder_name );
```



APPASERVER Data Structure

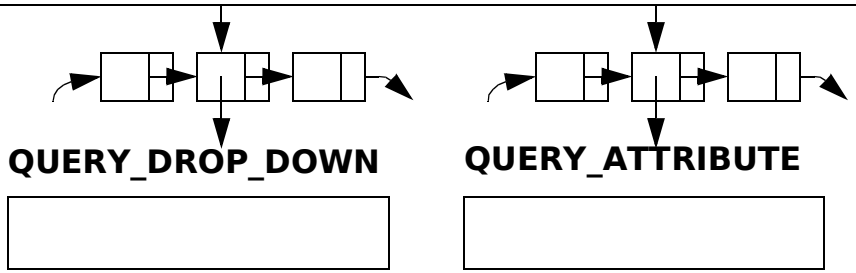
QUERY_WIDGET_WHERE

```
/* Usage */
QUERY_WIDGET_WHERE *query_widget_where_new(
    char *widget_folder_name,
    LIST *folder_attribute_list,
    LIST *relation_mto1_non_isa_list,
    char *security_entity_where(),
    DICTIONARY *drillthru_dictionary );

/* Attributes */
char *widget_folder_name;
LIST *folder_attribute_list;
LIST *primary_key_list;
LIST *relation_mto1_non_isa_list;
char *security_entity_where;
DICTIONARY *drillthru_dictionary;

/* Process */
LIST *exclude_attribute_name_list = list_new();

LIST *query_drop_down_list(
    exclude_attribute_name_list /* out */,
    widget_folder_name
        /* many_folder_name */,
    folder_attribute_list
        /* many_folder_attribute_list */,
    relation_mto1_non_isa_list,
    drillthru_dictionary );

char *query_drop_down_list_where(
    query_drop_down_list(),
    0 /* relation_mto1_isa_list_length */ );

LIST *query_attribute_list =
    query_attribute_list(
        folder_attribute_list,
        drillthru_dictionary,
        exclude_attribute_name_list );

char *query_attribute_list_where(
    query_attribute_list() );

char *where_clause =
    query_widget_where_clause(
        char *query_drop_down_where(),
        char *query_attribute_list_where(),
        char *security_entity_where );
```



QUERY_TABLE_WHERE

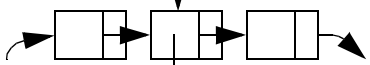
```
/* Usage */
QUERY_TABLE_WHERE *query_table_where_new(
    char *folder_name,
    LIST *folder_attribute_list,
    LIST *many_primary_key_list,
    LIST *relation_mto1_non_isa_list,
    LIST *relation_mto1_isa_list,
    char *security_entity_where(),
    DICTIONARY *drillthru_dictionary );

/* Process */
LIST *exclude_attribute_name_list = list_new();

LIST *query_drop_down_list(
    exclude_attribute_name_list /* out */,
    folder_name
        /* many_folder_name */,
    folder_attribute_list
        /* many_folder_attribute_list */,
    relation_mto1_non_isa_list,
    drillthru_dictionary );

LIST *query_attribute_list =
    query_attribute_list(
        folder_attribute_list,
        drillthru_dictionary,
        exclude_attribute_name_list );

char *query_drop_down_list_where(
    query_drop_down_list(),
    list_length(
        relation_mto1_isa_list ) );

char *query_drop_down_data_where(
    char *folder_name,
    char *attribute_name,
    char *escaped_replaced_data );

char *query_attribute_list_where(
    query_attribute_list() );

char *query_widget_where_evaluate(
    query_drop_down_where(),
    query_attribute_list_where() );

LIST *query_prompt_recursive_drop_down_list(
    exclude_attribute_name_list,
    query_folder->folder_name
        /* query_folder_name */,
    prompt_recursive->
        prompt_recursive_folder_list,
    query_dictionary );

LIST *query_search_replace_where_drop_down_list(
    LIST *exclude_attribute_name_list,
    char *folder_name,
    LIST *primary_key_list,
    LIST *relation_one2m_non_recursive_list,
    LIST *relation_mto1_isa_list,
    DICTIONARY *drillthru_dictionary );

char *query_join_where(
    query_folder->folder_name,
    query_folder->primary_key_list,
    query_folder->mto1_isa_related_folder_list,
    query->attribute_not_null_join );

char *query_related_join(
    query_folder->folder_name,
    query_folder->primary_key_list,
    isa_related_folder->folder->folder_name,
    isa_related_folder->
        foreign_attribute_name_list );

char *query_primary_key_where_clause(
    LIST *primary_key_list,
    char *input,
    char delimiter );
```



QUERY (Select)

```
LIST *select_list =
    query_select_list(
        LIST *folder_attribute_append_isa_list(),
        LIST *ignore_select_attribute_name_list,
        LIST *exclude_lookup_attribute_name_list,
        int relation_mto1_isa_length,
        query_date_convert_new() );

LIST *select_list =
    query_primary_select_list(
        LIST *folder_attribute_primary_key_list(),
        LIST *folder_attribute_primary_list(),
        query_date_convert_new() );

char *query_select_clause(
    LIST *select_list->
        query_select_string() );
```

query_select_list()
query_primary_select_list()



QUERY_DATE_CONVERT

```
/* Usage */
QUERY_DATE_CONVERT *query_date_convert_new(
    char *login_name );

/* Process */
enum date_convert_format
    query_date_convert_format(
        environment_application_name(),
        login_name );

/* Generic */
char *query_date_convert_select_string(
    char *attribute_name,
    boolean attribute_is_date,
    boolean attribute_is_date_time,
    query_date_convert_format() );

enum date_convert_format
```

international,
american,
military,
date_convert_unknown

QUERY_SELECT

```
/* Usage */
QUERY_SELECT *query_select_new(
    char *folder_name,
    char *attribute_name,
    boolean attribute_is_date(),
    boolean attribute_is_date_time(),
    QUERY_DATE_CONVERT *
        query_date_convert );

/* Process */
char *query_select_string(
    char *folder_name,
    char *attribute_name,
    boolean attribute_is_date,
    boolean attribute_is_date_time,
    enum date_convert_format
        query_date_convert_format() );
```

DATE_CONVERT

```
DATE_CONVERT *date_convert_international(
    char *application_name,
    char *login_name,
    char *date_string );

DATE_CONVERT *date_convert_string_international(
    enum date_convert_format source_format,
    char *date_string );

enum date_convert date_convert_user_date_format(
    char *application_name,
    char *login_name );
```

QUERY (From)

```
char *query_from_clause(
    char *folder_name,
    LIST *relation_mto1_isa_list,
    char *attribute_not_null_folder_name );
```

QUERY (Order)

```
char *query_order_clause(
    char *folder_name,
    LIST *primary_key_list,
    LIST *folder_attribute_append_isa_list,
    DICTIONARY *sort_dictionary );
```

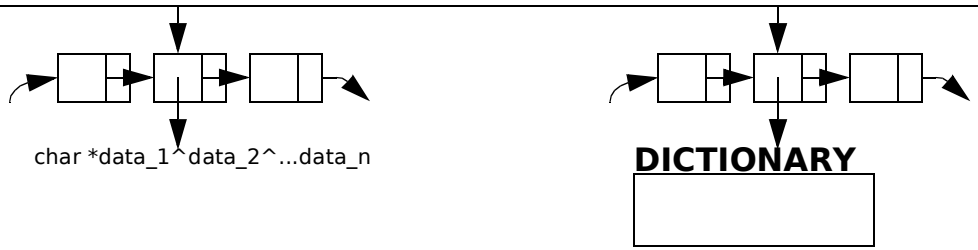
APPASERVER Data Structure

QUERY (Primary)

```
/* Usage */
LIST *query_primary_delimited_fetch_list(
    char *folder_name,
    LIST *primary_key_list,
    LIST *folder_attribute_primary_list,
    LIST *relation_mto1_non_isa_list,
    DICTIONARY *drillthru_dictionary,
    char *login_name );
```

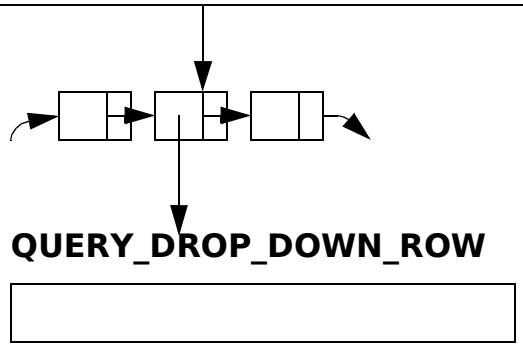
QUERY (Output)

```
/* Usage */
LIST *query_system_dictionary_list(
    query_system_string(),
    LIST *query_select_name_list(
        query->select_list ) );

LIST *query_system_delimited_list(
    query_system_string(),
    LIST *query_select_name_list(
        query->select_list ) );

/* External */
LIST *query_dictionary_list(
    char *query->select_clause,
    LIST *query->select_list,
    char *query->from_clause,
    char *query->where_clause,
    char *query->order_clause,
    int max_rows );

LIST *query_delimited_list(
    query->select_clause,
    query->from_clause,
    query->where_clause,
    query->order_clause,
    int max_rows );

char *query_system_string(
    query->select_clause,
    query->from_clause,
    query->where_clause,
    query->order_clause,
    int max_rows );

LIST *query_primary_delimited_list(
    char *folder_table_name,
    LIST *primary_key_list,
    LIST *foreign_key_list,
    LIST *foreign_data_list );
```



QUERY_DROP_DOWN

```
/* Usage */
LIST *query_drop_down_list(
    LIST *exclude_attribute_name_list /* out */,
    char *many_folder_name,
    LIST *many_folder_attribute_list,
    LIST *relation_mto1_non_isa_list,
    DICTIONARY *dictionary_separate->drillthru_dictionary );

QUERY_DROP_DOWN *query_drop_down_new(
    char *many_folder_name,
    LIST *many_folder_attribute_list,
    char *relation->one_folder->folder_name
        /* one_folder_name */,
    LIST *relation_mto1->one_folder->primary_key_list
        /* one_primary_key_list */,
    LIST *relation_mto1->foreign_key_list,
    DICTIONARY *drillthru_dictionary,
    int dictionary_string_list_highest_index(
            relation_mto1->foreign_key_list,
            drillthru_dictionary ) );

/* Attributes */
char *many_folder_name;
char *one_folder_name;
LIST *one_primary_key_list;
LIST *foreign_key_list;
DICTIONARY *drillthru_dictionary;
int highest_index;

/* Process */
QUERY_DROP_DOWN *query_drop_down_calloc();

enum relational_operator relational_operator;

LIST *query_drop_down_row_list(
    one_primary_key_list,
    foreign_key_list,
    many_folder_attribute_list,
    drillthru_dictionary,
    highest_index );

/* Generic */
char *query_drop_down_operator_key(
    LIST *foreign_key_list );

char *query_drop_down_data_where(
    char *folder_name,
    char *attribute_name,
    char *escaped_replaced_data );
```

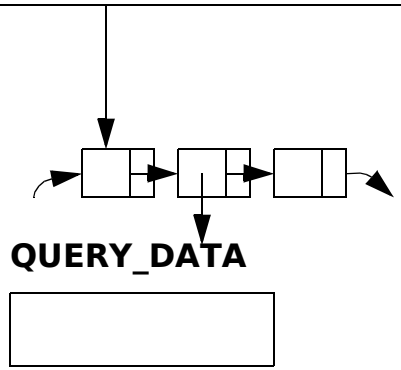


APPASERVER Data Structure

QUERY_DROP_DOWN_ROW

```
/* Usage */
LIST *query_drop_down_row_list(
    LIST *one_primary_key_list,
    LIST *foreign_key_list,
    LIST *many_folder_attribute_list,
    DICTIONARY *drillthru_dictionary,
    int dictionary_string_list_highest_index() );

QUERY_DROP_DOWN_ROW *query_drop_down_row_new(
    LIST *one_primary_key_list,
    LIST *foreign_key_list,
    LIST *many_folder_attribute_list,
    DICTIONARY *drillthru_dictionary,
    int index );

/* Input */
LIST *one_primary_key_list;
LIST *foreign_key_list;
LIST *many_folder_attribute_list;
DICTIONARY *drillthru_dictionary;
int index;
```

```
/* Process */
QUERY_DROP_DOWN_ROW *query_drop_down_row_calloc(
    void );

char *query_drop_down_row_key(
        LIST *foreign_key_list,
        int index );

char *query_drop_down_row_data_list_string(
    query_drop_down_row_key(),
    DICTIONARY *drillthru_dictionary );

LIST *query_drop_down_row_data_string_list(
        query_drop_down_row_data_list_string() );

LIST *query_data_list(
    foreign_key_list,
    many_folder_attribute_list,
    query_drop_down_row_data_string_list() );
```



QUERY_ATTRIBUTE

```
/* Usage */
LIST *query_attribute_list(
        LIST *folder_attribute_append_isa_list,
        DICTIONARY *dictionary,
        LIST *exclude_attribute_name_list );

QUERY_ATTRIBUTE *query_attribute_new(
        char *attribute_name,
        char *folder_name,
        char *datatype_name,
        enum relation_operator relation_operator;
        QUERY_DATA *from_data;
        QUERY_DATA *to_data;
        int primary_key_index );

/* Input */
char *attribute_name;
char *folder_name;
char *datatype_name;
enum relational_operator relational_operator;
QUERY_DATA *from_data;
QUERY_DATA *to_data;
int primary_key_index;

/* Process */
boolean query_attribute_date_time_between(
        QUERY_ATTRIBUTE **date_between_attribute,
        QUERY_ATTRIBUTE **time_between_attribute,
        LIST *query_attribute_list() );

char *query_attribute_between_date_time_where(
        char *date_attribute_name,
        char *time_attribute_date,
        QUERY_DATA *date_from_data,
        QUERY_DATA *time_from_data,
        QUERY_DATA *date_to_data,
        QUERY_DATA *time_to_data,
        char *folder_table_name() );

/* Generic */
char *query_attribute_operator_key(
    char *attribute_name );
```

QUERY_DATA

```
/* Usage */
LIST *query_data_list(
    LIST *foreign_key_list,
    LIST *many_folder_attribute_list,
    LIST *query_drop_down_row_data_string_list() );

QUERY_DATA *query_data_new(
    char *attribute_name,
    char *datatype_name,
    char *security_sql_injection_escape(
        security_replace_special_characters(
            string_trim_number_characters(
                char *data,
                char *datatype_name ) ) ) );

/* Input */
char *attribute_name;
char *datatype_name;
char *escaped_replaced_data;

/* Process */
char *escaped_replaced_data =
    query_data_convert_date_international(
        char *datatype_name,
        char *escaped_replaced_data );
```

QUERY_OR_SEQUENCE

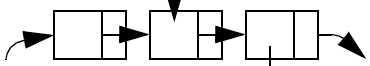
```
/* Usage */
QUERY_OR_SEQUENCE *query_or_sequence_new(
    LIST *attribute_name_list );

/* Attributes */
LIST *attribute_name_list;
LIST *data_list_list;

/* Generic */
boolean query_or_sequence_set_data_list(
    LIST *data_list_list,
    LIST *data_list );

boolean query_or_sequence_set_data_list_string(
    LIST *data_list_list,
    char *list_string );

char *query_or_sequence_where_clause(
    LIST *attribute_name_list,
    LIST *data_list_list,
    boolean with_and_prefix );
```

ROLE

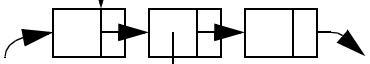
```
/* Usage */
ROLE *role_fetch(
    char *role_name,
    boolean fetch_role_attribute_exclude_list );

LIST *role_list_fetch(
    char *login_name );

/* Attributes */
char *role_name;
boolean folder_count;
boolean override_row_restrictions;

/* Process */
char *role_primary_where(
    char *role_name );

LIST *role_attribute_exclude_list =
    role_attribute_exclude_system_list();

/* External */
LIST *role_exclude_lookup_attribute_name_list(
    role_attribute_exclude_system_list() );

LIST *role_exclude_insert_attribute_name_list(
    role_attribute_exclude_system_list() );

LIST *role_exclude_update_attribute_name_list(
    role_attribute_exclude_system_list() );
```

ROLE_APPASERVER_USER

```
/* Usage */
ROLE_APPASERVER_USER *role_appaserver_user_fetch(
    char *session->login_name;
    char *security_sql_injection_escape(
        role_name ) );

LIST *role_appaserver_user_system_list(
    role_appaserver_user_system_string() );

/* Attributes */
char *login_name;
char *role_name;

/* Process */
char *role_appaserver_user_system_string(
    char *where );

char *role_appaserver_user_primary_where(
    char *login_name,
    char *role_name );
```



role_exclude_lookup_attribute_name_list()
role_exclude_insert_attribute_name_list()
role_exclude_update_attribute_name_list()



char *attribute_name

ROLE_ATTRIBUTE_EXCLUDE

```
/* Usage */
LIST *role_attribute_exclude_system_list(
    char *role_attribute_exclude_system_string(
        char *role_primary_where(
            char *role_name ) ) ) );

/* Attributes */
char *role_name;
char *attribute_name;
char *permission;

/* Process */
ROLE_ATTRIBUTE_EXCLUDE *
    role_attribute_exclude_parse(
        char *input );
```

insert
update
lookup

ROLE_FOLDER

```
/* Usage */
LIST *role_folder_list(
    char *role_name,
    char *folder_name );

LIST *role_folder_lookup_list(
    char *role_name );

LIST *role_folder_insert_list(
    char *role_name );

/* Attributes */
char *role_name;
char *folder_name;
char *permission;
char *subschema_name;

/* External */
boolean role_folder_insert(
    role_folder_list() );

boolean role_folder_update(
    role_folder_list() );

boolean role_folder_lookup(
    role_folder_list() );

LIST *role_folder_subschema_name_list(
    role_folder_lookup|insert_list() );

LIST *role_folder_subschema_missing_folder_name_list(
    role_folder_lookup|insert_list() );

LIST *role_folder_subschema_folder_name_list(
    char *subschema_name,
    role_folder_lookup|insert_list() );
```

ROLE_PROCESS

```
/* Usage */
LIST *role_process_list(
    char *role_name );

/* Attributes */
char *role_name;
char *process_name;
char *process_group_name;

/* Generic */
LIST *role_process_group_name_list(
    role_process_list() );

LIST *role_process_name_list(
    char *process_group_name,
    role_process_list() );

LIST *role_process_or_set_name_list(
        char *process_group_name,
        role_process_list(),
        role_process_set_list() );
```

ROLE_PROCESS_SET

```
/* Usage */
LIST *role_process_set_list(
    char *role_name );

/* Attributes */
char *role_name;
char *process_set_name;
char *process_group_name;

/* Generic */
LIST *role_process_set_group_name_list(
    role_process_set_list() );
```

UPDATE

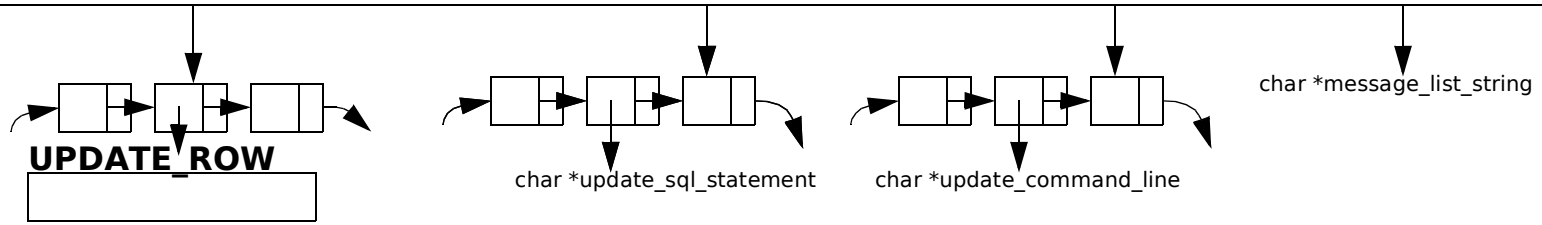
```
/* Usage */
UPDATE *update_new(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    char *login_name,
    char *role_name,
    char *folder_name );

/* Process */
ROLE *role =
    role_fetch(
        role_name,
        1 /* fetch_attribute_exclude_list */ );

FOLDER *folder =
    folder_fetch(
        folder_name,
        role_name,
        role_exclude_update_attribute_name_list(
            role->attribute_exclude_list ),
        /* Also sets primary_key_list */
        1 /* fetch_folder_attribute_list */,
        0 /* not fetch_relation_mto1_non_isa_list */,
        /* Also sets folder_attribute_append_isa_list */
        1 /* fetch_relation_mto1_isa_list */,
        0 /* not fetch_relation_one2m_list */,
        1 /* fetch_relation_one2m_recursive_list */,
        1 /* fetch_process */,
        1 /* fetch_role_folder_list */,
        1 /* fetch_row_level_restriction */,
        0 /* not fetch_role_operation_list */ );

if ( !role_folder_update(
        folder->role_folder_list ) )
{
    return (UPDATE *)0;
}

SECURITY_ENTITY *security_entity =
    security_entity_new(
        login_name,
        folder->non_owner_forbid ||
        folder->non_owner_view_only,
        role->override_row_restrictions );

UPDATE_ROW_LIST *update_row_list =
    update_row_list_new(
        post_dictionary,
        file_dictionary,
        login_name,
        folder->folder_name,
        folder->folder_attribute_list,
        folder->relation_mto1_isa_list,
        folder->relation_one2m_recursive_list,
        folder->post_change_process,
        security_entity );

/* External */
char *update_sql_statement(
    folder_table_name(
        environment_application_name(),
        folder_name ),
    update_attribute_changed_list(),
    update_where_clause() );

char *update_command_line(
    char *command_line,
    char *login_name,
    LIST *update_attribute_list() );

/* Returns message_string */
char *update_execute(
    char *update_sql_statement() );

FILE *update_sql_pipe(
    char *environment_application_name() );

void update_sql_statement_list_execute(
    FILE *update_sql_pipe(),
    LIST *sql_statement_list );

/* Returns message_list_string */
char *update_command_line_list_execute(
    LIST *command_line_list );
```

post_dictionary
file_dictionary

DICTIONARY **FOLDER** **ROLE** **SECURITY_ENTITY** **UPDATE_ROW_LIST**

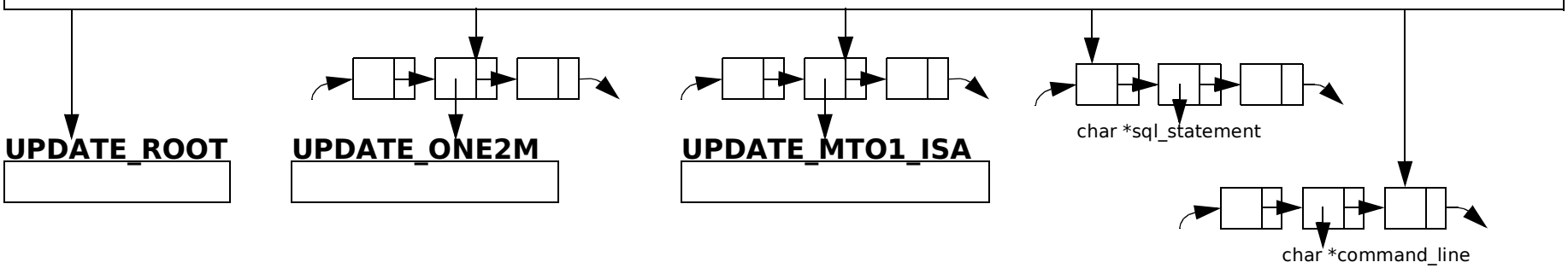
UPDATE_ROW_LIST

```
/* Usage */
UPDATE_ROW_LIST *update_row_list_new(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    char *login_name,
    char *folder->folder_name,
    LIST *folder->folder_attribute_list,
    LIST *folder->relation_mto1_isa_list,
    LIST *folder->relation_one2m_recursive_list,
    PROCESS *folder->post_change_process,
    SECURITY_ENTITY *security_entity );

/* Process */
LIST *list = list_new();

if ( dictionary_highest_row( post_dictionary )  == -1 )
    return (UPDATE_ROW_LIST *)0;

for (   row = 1;
        row <= dictionary_highest_row();
        row++ )
{
    list_set(
        list,
        update_row_new(
            post_dictionary,
            file_dictionary,
            login_name,
            folder_name,
            folder_attribute_list,
            relation_mto1_isa_list,
            relation_one2m_recursive_list,
            post_change_process,
            security_entity,
            row ) );
}

int update_row_list_cell_count(
    UPDATE_ROW_LIST *update_row_list );

/* Output */
char *message_list_string =
    char *update_row_list_execute(
        UPDATE_ROW_LIST *update_row_list )
{
    char *message_list_string = {0};
    char *message_string;

    for update_row in update_row_list->list
    {
        message_string = (char *)0;

        if ( update_row->update_root )
        {
            if ( ( message_string =
                    update_execute(
                        update_row->
                            update_root->
                            sql_statement,
                        (char *)0 /* application */ ) ) )
            {
                message_list_string =
                    string_append(
                        message_list_string,
                        message_string,
                        “<br>” );
            }
        }

        if ( message_string )
        {
            continue;
        }

        if ( list_length( update_row->
                        update_mto1_isa_list ) )
        {
            update_sql_statement_list_execute(
                update_sql_pipe(),
                update_mto1_isa_sql_statement_list(
                    update_row->
                        update_mto1_isa_list ) );
        }

        if ( update_row->
                update_root->
                changed_primary_key
        &&  list_length( update_row->
                        update_one2m_list ) )
        {
            update_sql_statement_list_execute(
                update_sql_pipe(),
                update_one2m_sql_statement_list(
                    update_row->
                        update_one2m_list ) );
        }

        if ( ( message_string =
                update_command_line_list_execute(
                    update_row->
                        command_line_list ) ) )
        {
            message_list_string =
                string_append(
                    message_list_string,
                    message_string,
                    “<br>” );
        }
    }
}

/* Debug */
LIST *update_row_list_sql_statement_list(
    UPDATE_ROW_LIST *update_row_list );

LIST *update_row_list_command_line_list(
    UPDATE_ROW_LIST *update_row_list );
```



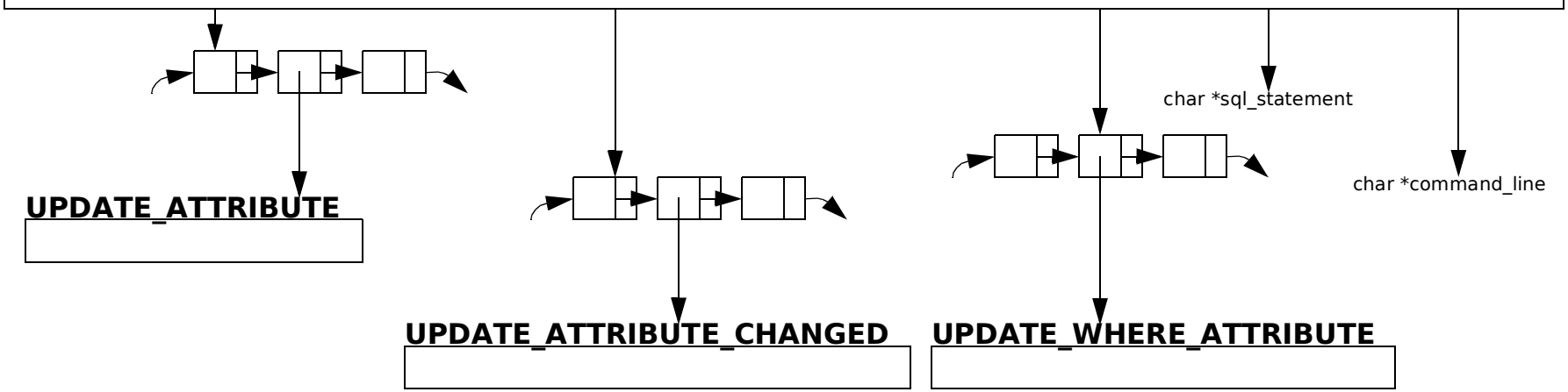
UPDATE_ROW

```
/* Usage */
UPDATE_ROW *update_row_new(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    char *login_name,
    char *folder_name,
    LIST *folder_attribute_list,
    LIST *relation_mto1_isa_list,
    LIST *relation_one2m_recursive_list,
    PROCESS *post_change_process,
    SECURITY_ENTITY *security_entity,
    int row );

/* Input */
int row;

/* Process */
UPDATE_ROOT *update_root =
    update_root_new(
        post_dictionary,
        file_dictionary,
        login_name,
        folder_name,
        folder_attribute_list,
        post_change_process,
        security_entity,
        int row );

LIST *update_mto1_isa_list(
    post_dictionary,
    file_dictionary,
    login_name,
    relation_mto1_isa_list,
    row );

LIST *update_one2m_list(
    post_dictionary,
    file_dictionary,
    login_name,
    relation_one2m_recursive_list,
    row );

int update_row_cell_count(
    UPDATE_ROOT *update_root,
    update_mto1_isa_list(),
    update_one2m_list() );

LIST *update_row_sql_statement_list(
    UPDATE_ROOT *update_root,
    LIST *update_mto1_isa_list(),
    LIST *update_one2m_list() );

LIST *update_row_command_line_list(
    UPDATE_ROOT *update_root,
    LIST *update_mto1_isa_list(),
    LIST *update_one2m_list() );
```



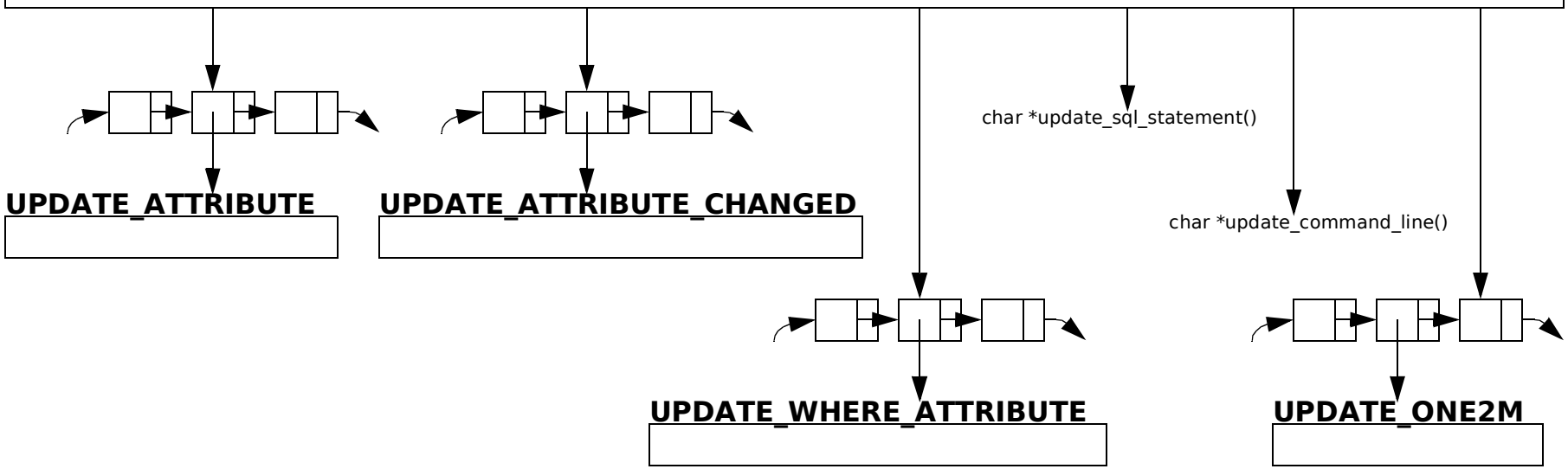
UPDATE_ROOT

```
/* Usage */
UPDATE_ROOT *update_root_new(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    char *login_name,
    char *folder_name,
    LIST *folder_attribute_list,
    PROCESS *post_change_process,
    SECURITY_ENTITY *security_entity,
    int row );

/* Process */
LIST *update_attribute_list(
    post_dictionary,
    file_dictionary,
    folder_attribute_list,
    row );

LIST *update_attribute_changed_list(
    update_attribute_list() );

if ( !update_changed_attribute_list() ) return (UPDATE_ROOT *)0;

LIST *update_where_attribute_list(
    update_attribute_list() );

char *update_root_where_clause(
    char *update_where_clause(
        update_where_attribute_list(),
    SECURITY_ENTITY *security_entity,
    folder_attribute_list );

char *update_sql_statement(
    folder_table_name(
        environment_application_name(),
        folder_name ),
    update_attribute_changed_list(),
    update_root_where_clause() );

char *update_command_line(
    post_change_process->command_line,
    login_name,
    update_attribute_list() );

boolean update_attribute_changed_primary_key(
    update_attribute_changed_list() );
```



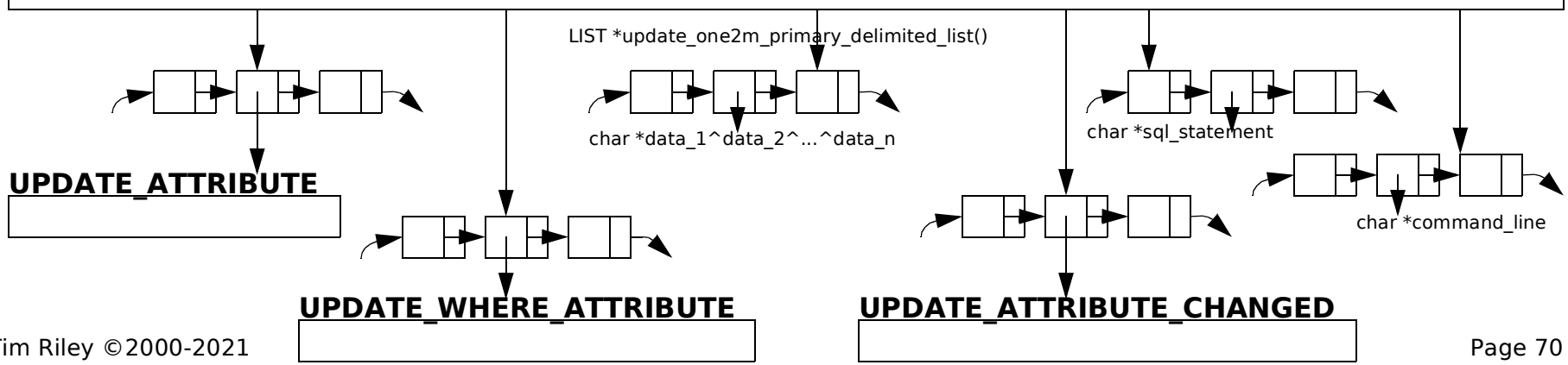
UPDATE_MTO1_ISA

```
/* Usage */
LIST *update_mto1_isa_list(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    char *login_name,
    LIST *relation_mto1_isa_list,.
    int row );

UPDATE_MTO1_ISA *update_mto1_isa_new(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    char *login_name,
    RELATION *relation_mto1_isa,
    int row  );

/* Process */
LIST *update_attribute_list(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    LIST *relation_mto1_isa->
        one_folder->
        folder_attribute_list,
    int row );

LIST *update_attribute_changed_list(
    update_attribute_list() );

LIST *update_where_attribute_list(
    update_attribute_list() );

char *update_sql_statement(
    folder_table_name(
        environment_application_name(),
        relation_mto1_isa->
        one_folder->
        folder_name ),
    update_attribute_changed_list(),
    update_where_clause(
        update_where_attribute_list() )  );

char *update_command_line(
    relation_mto1_isa->
        one_folder->
        post_change_process->
        command_line,
    login_name,
    update_attribute_list() );

LIST *update_mto1_isa_one2m_list(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    char *login_name,
    char *relation_mto1_isa->
        one_folder->
        folder_name,
    int row );

/* Output */
LIST *update_mto1_isa_sql_statement_list(
    update_mto1_isa_list() );

LIST *update_mto1_isa_command_line_list(
    update_mto1_isa_list() );

LIST *update_mto1_isa_one2m_sql_statement_list(
    LIST *update_mto1_isa_one2m_list() );

LIST *update_mto1_isa_one2m_command_line_list(
    LIST *update_mto1_isa_one2m_list() );
```



UPDATE_ONE2M

```
/* Usage */
LIST *update_one2m_list(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    char *login_name,
    LIST *relation_one2m_recursive_list,
    int row  );

UPDATE_ONE2M *update_one2m_new(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    char *login_name,
    RELATION *relation_one2m,
    int row );

/* Process */
LIST *update_attribute_list(
    post_dictionary,
    file_dictionary,
    relation_one2m->many_folder->folder_attribute_list,
    row );

LIST *update_attribute_changed_list(
    update_attribute_list() );

LIST *update_where_attribute_list(
    update_attribute_list() );

LIST *update_one2m_primary_delimited_list(
    char *folder_table_name(
        environment_application_name(),
        relation_one2m->many_folder->folder_name ),
    LIST *relation_one2m->many_folder->primary_key_list,
    char *update_where_clause(
        update_where_attribute_list() );

LIST *update_one2m_sql_statement_list(
    char *folder_table_name(),
    LIST *update_attribute_changed_list(),
    LIST *relation_one2m->many_folder->primary_key_list,
    LIST *update_one2m_primary_delimited_list()  );

LIST *update_one2m_command_line_list(
    char *relation_one2m->many_folder->post_change_process->command_line,
    char *login_name,
    LIST *relation_one2m->many_folder->primary_key_list,
    LIST *update_one2m_primary_delimited_list(),
    LIST *update_attribute_changed_list() );

char *update_one2m_sql_statement(
    char *folder_table_name(),
    LIST *update_attribute_changed_list(),
    LIST *primary_key_list,
    LIST *list_string_list(
        list_get(
            update_one2m_primary_delimited_list() ),
        SQL_DELIMITER ) /* primary_data_list */ );

char *update_one2m_command_line(
    char *command_line,
    char *login_name,
    LIST *primary_key_list,
    LIST *primary_data_list,
    LIST *update_attribute_changed_list() );

/* Output */
LIST *update_one2m_list_sql_statement_list(
    LIST *update_one2m_list() );

LIST *update_one2m_list_command_line_list(
    LIST *update_one2m_list() );
```



UPDATE_ATTRIBUTE

```
/* Usage */
LIST *update_attribute_list(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    LIST *folder_attribute_list,
    int row );

UPDATE_ATTRIBUTE *update_attribute_new(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    FOLDER_ATTRIBUTE *folder_attribute,
    int row );

/* Process */
char *file_data =
    dictionary_row_get(
        folder_attribute->attribute_name,
        row,
        file_dictionary );

char *sql_injection_escape_file_data =
    security_sql_injection_escape(
        file_data );

UPDATE_ATTRIBUTE_CHANGED *
    update_attribute_changed =
        update_attribute_changed_new(
            post_dictionary,
            folder_attribute->attribute_name,
            folder_attribute->
                attribute->
                datatype_name,
            folder_attribute->
                attribute->
                primary_key_index,
            file_data,
            row );

/* External */
LIST *update_attribute_changed_list(
    LIST *update_attribute_list() );
```

UPDATE_ATTRIBUTE_CHANGED

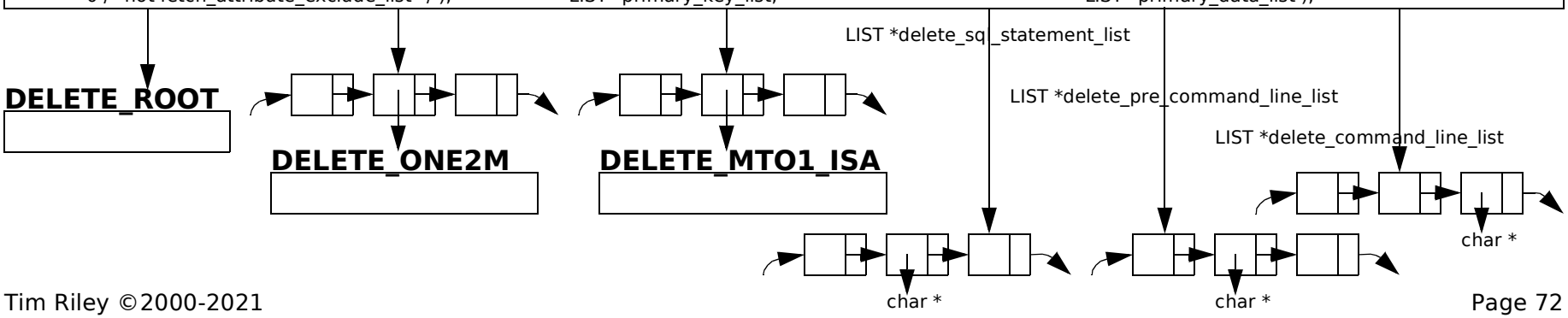
```
/* Usage */
UPDATE_ATTRIBUTE_CHANGED *
    update_attribute_changed_new(
        DICTIONARY *post_dictionary,
        char *attribute_name,
        char *datatype_name,
        int primary_key_index,
        char *file_data,
        int row );

/* Process */
char *post_data =
    dictionary_row_get(
        attribute_name,
        row,
        post_dictionary );

boolean attribute_changed =
    ( strcasecmp( file_data, post_data ) != 0 );

if ( !attribute_changed )
{
    return (UPDATE_ATTRIBUTE_CHANGED *)0;
}

char *sql_injection_escape_post_data =
    security_sql_injection_escape(
        security_replace_special_characters(
            string_trim_number_characters(
                post_data,
                datatype_name ) ) );

/* External */
boolean update_attribute_changed_primary_key(
    LIST *update_attribute_changed_list() );

UPDATE_CHANGED_ATTRIBUTE *
    update_attribute_changed_seek(
        char *attribute_name,
        LIST *update_attribute_changed_list() );
```

UPDATE_WHERE_ATTRIBUTE

```
/* Usage */
LIST *update_where_attribute_list(
        LIST *update_attribute_list );

UPDATE_WHERE_ATTRIBUTE *
    update_where_attribute_new(
        char *primary_attribute_name,
        char *datatype_name,
        char *sql_injection_escape_file_data );

/* External */
char *update_where_clause(
    LIST *update_where_attribute_list() );
```

DELETE

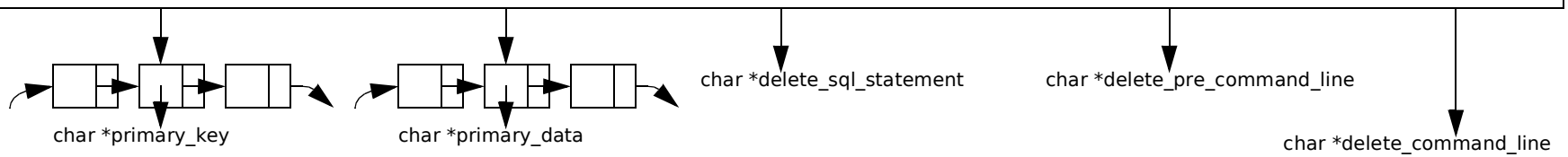
```
/* Usage */
DELETE *delete_new(
    char *application_name,
    char *session_key,
    char *login_name,
    char *folder_name,
    char *role_name,
    LIST *primary_data_list,
    boolean dont_delete_mto1_isa );

/* Process */
FOLDER *folder =
    folder_fetch(
        folder_name,
        role_name,
        (LIST *)0 /* role_exclude_attribute_name_list */,
        /* Also sets primary_key_list */
        1 /* fetch_folder_attribute_list */,
        0 /* not fetch_relation_mto1_non_isa_list */,
        /* Also sets folder_attribute_append_isa_list */
        1 /* fetch_relation_mto1_isa_list */,
        0 /* not fetch_relation_one2m_list */,
        1 /* fetch_relation_one2m_recursive_list */,
        1 /* fetch_process */,
        0 /* not fetch_role_folder_list */,
        1 /* fetch_row_level_restriction */,
        1 /* fetch_role_operation_list */ );

if ( !role_operation_delete(
        folder->role_operation_list ) )
{
    return (DELETE *)0;
}

ROLE *role =
    role_fetch(
        role_name,
        0 /* not fetch_attribute_exclude_list */ );

SECURITY_ENTITY *security_entity =
    security_entity_new(
        login_name,
        folder->non_owner_forbid ||
        folder->non_owner_view_only,
        role->override_row_restrictions );

DELETE_ROOT *delete_root =
    delete_root_new(
        application_name,
        folder_name,
        login_name,
        folder->primary_key_list,
        primary_data_list,
        folder->post_change_process,
        security_entity_where() );

LIST *delete_one2m_list(
    application_name,
    folder->relation_one2m_recursive_list,
    primary_data_list
        /* foreign_data_list */ );

if ( !dont_delete_mto1_isa )
{
    LIST *delete_mto1_isa_list(
        application_name,
        folder->relation_mto1_isa_list,
        primary_data_list );
}

/* External */
char *delete_sql_statement(
    char *folder_table_name(
        application_name,
        folder_name ),
    LIST *primary_key_list,
    LIST *primary_data_list,
    char *security_entity_where );

char *delete_command_line(
    char *post_change_process->command_line,
    char *login_name,
    LIST *primary_key_list,
    LIST *primary_data_list,
    char *state );

/* Output */
LIST *delete_sql_statement_list(
    DELETE_ROOT *delete_root,
    LIST *delete_one2m_list(),
    LIST *delete_mto1_isa_list() );

LIST *delete_pre_command_line_list(
    DELETE_ROOT *delete_root,
    LIST *delete_one2m_list(),
    LIST *delete_mto1_isa_list() );

LIST *delete_command_line_list(
    DELETE_ROOT *delete_root,
    LIST *delete_one2m_list(),
    LIST *delete_mto1_isa_list() );

LIST *delete_distinct_folder_name_list(
    DELETE_ROOT *delete_root,
    LIST *delete_one2m_list(),
    LIST *delete_mto1_isa_list() );

char *delete_message_html(
    char *primary_data_list,
    int list_length( delete_sql_statement_list() ),
    LIST *delete_distinct_folder_name_list() );

char *delete_primary_data_list_display(
    LIST *primary_data_list );
```



DELETE_ROOT

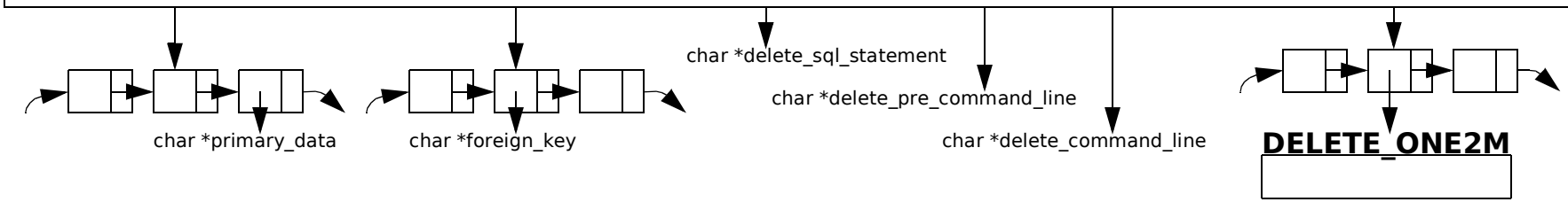
```
/* Usage */
DELETE_ROOT *delete_root_new(
    char *application_name,
    char *folder_name,
    char *login_name,
    LIST *primary_key_list,
    LIST *primary_data_list,
    PROCESS *post_change_process,
    char *security_entity_where );

/* Process */
```

```
char *delete_sql_statement(
    folder_table_name(),
    primary_key_list,
    primary_data_list,
    security_entity_where );

char *delete_pre_command_line =
    delete_command_line(
        post_change_process->command_line,
        login_name,
        primary_key_list,
        primary_data_list,
        “predelete” /* state */ );

char *delete_command_line =
    delete_command_line(
        post_change_process->command_line,
        login_name,
        primary_key_list,
        primary_data_list,
        “delete” /* state */ );
```



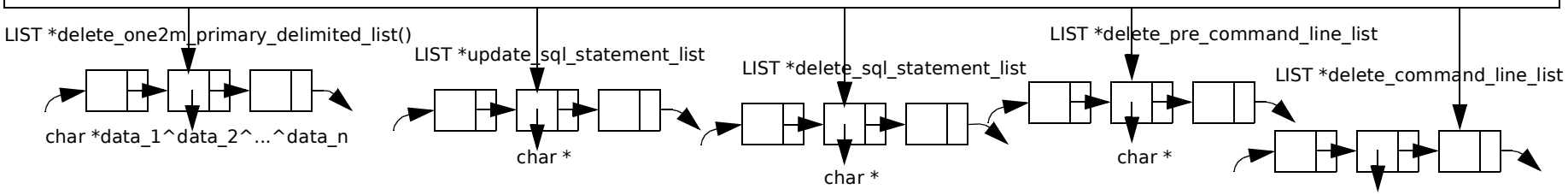
DELETE_MTO1_ISA

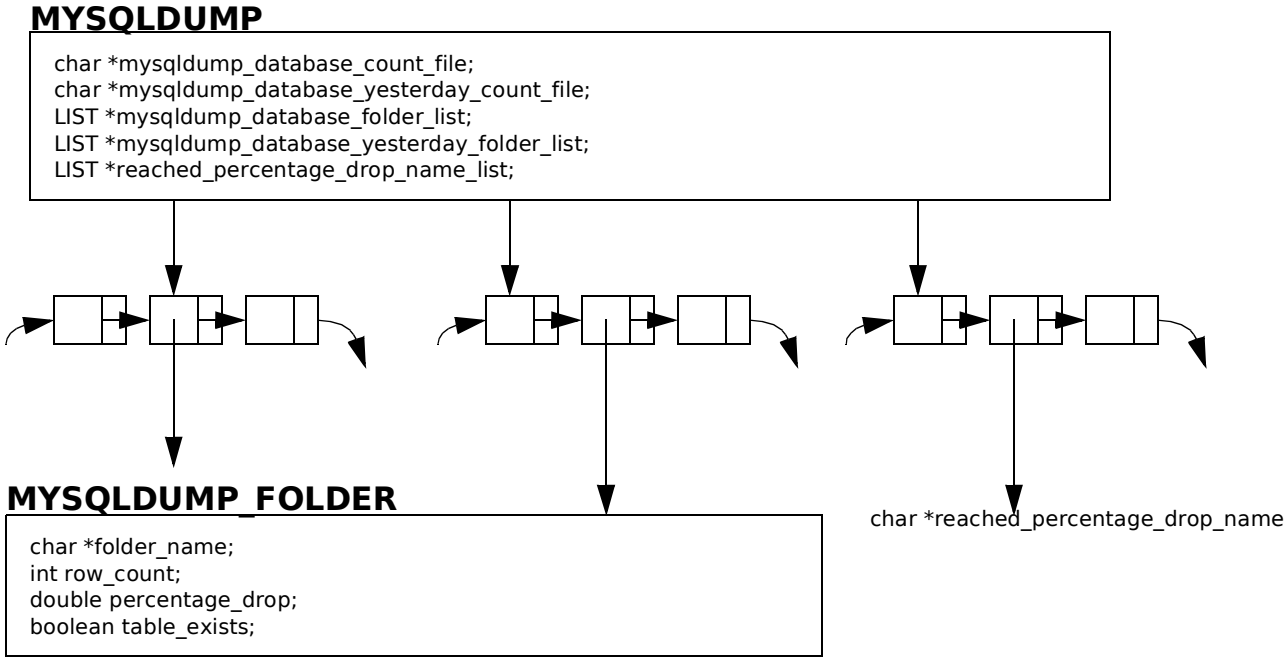
```
/* Usage */
LIST *delete_mto1_isa_list(
    char *application_name,
    char *login_name,
    LIST *folder->relation_mto1_isa_list,
    LIST *primary_data_list );

DELETE_MTO1_ISA *delete_mto1_isa_new(
    char *application_name,
    char *login_name,
    RELATION *relation_mto1_isa,
    LIST *primary_data_list );

/* Process */
char *delete_sql_statement(
    folder_table_name(
        application_name,
        relation_mto1_isa->
            one_folder->
            folder_name ),
    relation_mto1_isa->foreign_key_list,
    primary_data_list );

char *delete_pre_command_line =
    delete_command_line(
        relation_mto1_isa->
            one_folder->
            post_change_process->
            command_line,
        login_name,
        relation_mto1_isa->foreign_key_list,
        primary_data_list,
        "predelete" );

char *delete_command_line =
    delete_command_line(
        relation_mto1_isa->
            one_folder->
            post_change_process->
            command_line,
        login_name,
        relation_mto1_isa->foreign_key_list,
        primary_data_list,
        "delete" );

LIST *delete_mto1_isa_one2m_list(
    char *application_name,
    char *login_name,
    char *relation_mto1_isa->
        one_folder->
        folder_name,
    LIST *primary_data_list );

/* Output */
LIST *delete_mto1_isa_sql_statement_list(
    LIST *delete_mto1_isa_list() );

LIST *delete_mto1_isa_pre_command_line_list(
    LIST *delete_mto1_isa_list() );

LIST *delete_mto1_isa_command_line_list(
    LIST *delete_mto1_isa_list() );

LIST *delete_mto1_isa_one2m_sql_statement_list(
    LIST *delete_mto1_isa_one2m_list() );

LIST *delete_mto1_isa_one2m_pre_command_line_list(
    LIST *delete_mto1_isa_one2m_list() );

LIST *delete_mto1_isa_one2m_command_line_list(
    LIST *delete_mto1_isa_one2m_list() );

LIST *delete_mto1_isa_distinct_folder_name_list(
    LIST *delete_mto1_isa_list() );

LIST *
    delete_mto1_isa_one2m_distinct_folder_name_list(
        LIST *delete_mto1_isa_one2m_list() );
```



DELETE_ONE2M

```
/* Usage */
LIST *delete_one2m_list(
    char *application_name,
    char *login_name,
    LIST *folder->relation_one2m_recursive_list,
    LIST *foreign_data_list );

DELETE_ONE2M *delete_one2m_new(
    char *application_name,
    char *login_name,
    RELATION *relation_one2m,
    LIST *foreign_data_list );

/* Process */
LIST *delete_one2m_primary_delimited_list(
    char *folder_table_name(
        application_name,
        relation_one2m->
            many_folder->
            folder_name ),
    LIST *relation_one2m->
        many_folder->
        primary_key_list,
    LIST *relation_one2m->foreign_key_list,
    LIST *foreign_data_list );

if ( !relation_one2m->is_primary_key_subset )
{
    LIST *delete_one2m_update_sql_statement_list(
        char *folder_table_name(
            application_name,
            relation_one2m->
                many_folder->
                folder_name ),
        LIST *relation_one2m->foreign_key_list,
        LIST *relation_one2m->
            many_folder->
            primary_key_list,
        LIST *delete_one2m_primary_delimited_list() );
}
else
{
    LIST *delete_one2m_sql_statement_list(
        char *folder_table_name(
            application_name,
            relation_one2m->
                many_folder->
                folder_name ),
        LIST *relation_one2m->
            many_folder->
            primary_key_list,
        LIST *delete_one2m_primary_delimited_list() );
}

LIST *delete_pre_command_line_list =
    delete_one2m_command_line_list(
        char *relation_one2m->
            many_folder->
            post_change_process->
            command_line,
        char *login_name,
        LIST *relation_one2m->foreign_key_list,
        LIST *delete_one2m_primary_delimited_list(),
        “predelete” /* state */ );

LIST *delete_one2m_command_line_list(
    char *relation_one2m->
        many_folder->
        post_change_process->
        command_line,
    char *login_name,
    LIST *relation_one2m->foreign_key_list,
    LIST *delete_one2m_primary_delimited_list(),
    “delete” /* state */ );

/* Internal */
char *delete_one2m_sql_statement(
    char *folder_table_name(),
    LIST *relation_one2m->
        many_folder->
        primary_key_list,
    list_get( delete_one2m_primary_delimited_list() )
        /* primary_data_list */ );

char *delete_one2m_update_sql_statement(
    char *folder_table_name(),
    LIST *relation_one2m->foreign_key_list,
    LIST *relation_one2m->
        many_folder->
        primary_key_list,
    list_get( delete_one2m_primary_delimited_list() )
        /* primary_data_list */ );

char *delete_one2m_update_set_null_statement(
    LIST *relation_one2m->foreign_key_list );

char *delete_command_line(
        post_change_process->command_line,
        login_name,
        relation_one2m->foreign_key_list,
        list_get( delete_one2m_primary_delimited_list() )
            /* primary_data_list */,
        state );

/* Output */
LIST *delete_one2m_list_sql_statement_list(
    LIST *delete_one2m_list() );

LIST *delete_pre_one2m_list_command_line_list(
    LIST *delete_one2m_list() );

LIST *delete_one2m_list_command_line_list(
    LIST *delete_one2m_list() );

LIST *delete_one2m_list_distinct_folder_name_list(
    LIST *delete_one2m_list() );
```



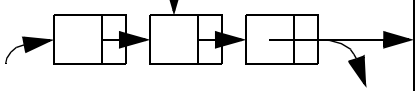


STRING

```
boolean string_strcpy(
    char *destination,
    char *source,
    int buffer_size );

boolean string_strcmp(
    char *string1,
    char *string2 );

char *string_escape_character(
    char *destination,
    char *data,
    char character );

char *string_escape_character_array(
    char *destination,
    char *data,
    char *character_array );

/* Position is 1 based */
int string_character_position(
    char *data,
    char character );

char *string_trim_character(
    char *data,
    char character );

char *string_trim_character_array(
    char *data,
    char *character_array );

char *string_trim_number_characters(
    char *number,
    char *datatype_name )
{
    return string_trim_character_array(
        number,
        ",$" );
}

char *string_decode_html_post(
    char *destination,
    char *source );

char *string_trim(
    char *buffer );

char *string_extract_lt_gt_delimited(
    char *destination,
    char *source );

int string_index(
    char *source );
```

PAIR_ONE2M

```
char *many_folder_name;
char *one_folder_name;

PAIR_ONE2M *pair_one2m_prompt_form_new(
    char *one_folder_name,
    char *keystrokes_save_function );

LIST *pair_one2m_prompt_form_folder_list(
    char *keystrokes_save_function,
    LIST *relation_one2m_pair_relation_list() );

char *pair_one2m_prompt_element_name(
    char *PAIR_ONE2M_PREFIX,
    char *folder_label );

boolean pair_one2m_participating(
    DICTIONARY *pair_one2m_dictionary );

PAIR_ONE2M *pair_one2m_post_new(
    DICTIONARY *pair_one2m_dictionary );

boolean pair_one2m_duplicate(
    char *MYSQL_DUPLICATE_ERROR_MESSAGE_KEY,
    char *message );

void pair_one2m_duplicate_set(
    DICTIONARY *pair_one2m_dictionary,
    char *PAIR_ONE2M_DUPLICATE_KEY );

boolean pair_one2m_inserted_duplicate(
    char *PAIR_ONE2M_DUPLICATE_KEY,
    DICTIONARY *pair_one2m_dictionary );

LIST *pair_one2m_fulfilled_folder_name_list(
    char *PAIR_ONE2M_FULFILLED_LIST_LABEL,
    DICTIONARY *pair_one2m_dictionary );

char *pair_one2m_folder_name(
    char *folder_label,
    DICTIONARY *pair_one2m_dictionary );

void pair_one2m_folder_set(
    DICTIONARY *pair_one2m_dictionary,
    char *folder_label,
    char *folder_name );

char *pair_one2m_next_folder_name(
    LIST *pair_one2m_fulfilled_folder_name_list(),
    LIST *relation_one2m_pair_relation_list(),
    char *one_folder_name,
    char *many_folder_name );

DICTIONARY *pair_one2m_fulfilled_dictionary(
    DICTIONARY *pair_one2m_dictionary,
    char *PAIR_ONE2M_FULFILLED_LIST_LABEL,
    LIST *pair_one2m_fulfilled_folder_name_list );

/* Insert form */
PAIR_ONE2M *pair_one2m_insert_form_new(
    DICTIONARY *pair_one2m_dictionary );
```

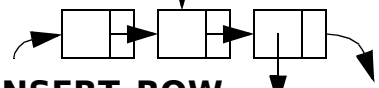


PAIR_ONE2M_FOLDER

```
/* Usage */
pair_one2m_folder_new(
    char *many_folder_name );

char *many_folder_name;

char *pair_folder_element_copy_function(
    prompt_many_element_name,
    many_folder_name,
    pair_one2m_prompt_form_element_name(),
    folder_name );

char *pair_folder_button_string(
    many_folder_name,
    keystrokes_save_function,
    pair_folder_element_copy_function() );
```

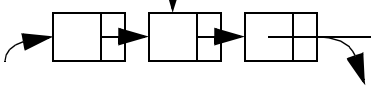
INSERT

```
/* Usage */
INSERT *insert_new(
    char *application_name,
    char *folder_name,
    LIST *primary_key_list,
    LIST *relation_mto1_non_isa_list(),
    LIST *folder_attribute_name_list(),
    LIST *folder_attribute_list(),
    DICTIONARY *row_dictionary,
    char *post_change_process->command_line );

/* Process */
LIST *insert_row_list = list_new();

for (    row = 0;
         row <= dictionary_highest_row(
             row_dictionary );
         row++ )
{
    list_set(
         insert_row_list,
         insert_row_new(
             folder_table_name(
                 application_name,
                 folder_name ),
             primary_key_list,
             relation_mto1_non_isa_list,
             folder_attribute_name_list,
             folder_attribute_list,
             row_dictionary,
             row,
             command_line ) );
}

/* External */
LIST *insert_sql_statement_list(
    insert_row_list );

LIST *insert_post_change_command_line_list(
    insert_row_list );

char *insert_statement(
    char *folder_table_name(),
    LIST *insert_data_name_list(),
    LIST *insert_data_extract_list() );
```



INSERT_ROW

```
/* Usage */
INSERT_ROW *insert_row_new(
    char *folder_table_name(),
    LIST *primary_key_list,
    LIST *relation_mto1_non_isa_list,
    LIST *folder_attribute_name_list,
    LIST *folder_attribute_list,
    DICTIONARY *row_dictionary,
    char *post_change_command_line,
    int row );

/* Process */
LIST *insert_data_list = list_new();

list_set_list(
    insert_data_list,
    insert_data_relation_extract_list(
        list_get( relation_mto1_non_isa_list ),
        folder_attribute_list,
        row_dictionary,
        row ) );

list_set(
    insert_data_list,
    insert_data_attribute_extract(
        list_get( folder_attribute_name_list ),
        folder_attribute_list,
        row_dictionary,
        row ) );

boolean insert_row_primary_key_okay(
    LIST *insert_data_name_list(),
    LIST *primary_key_list );

char *insert_row_sql_statement(
    char *folder_table_name,
    LIST *insert_data_name_list(
        insert_data_list ),
    LIST *insert_data_extract_list(
        insert_data_list ) );

char *insert_row_command_line(
    char *post_change_command_line,
    LIST *insert_data_list );
```



INSERT_DATA

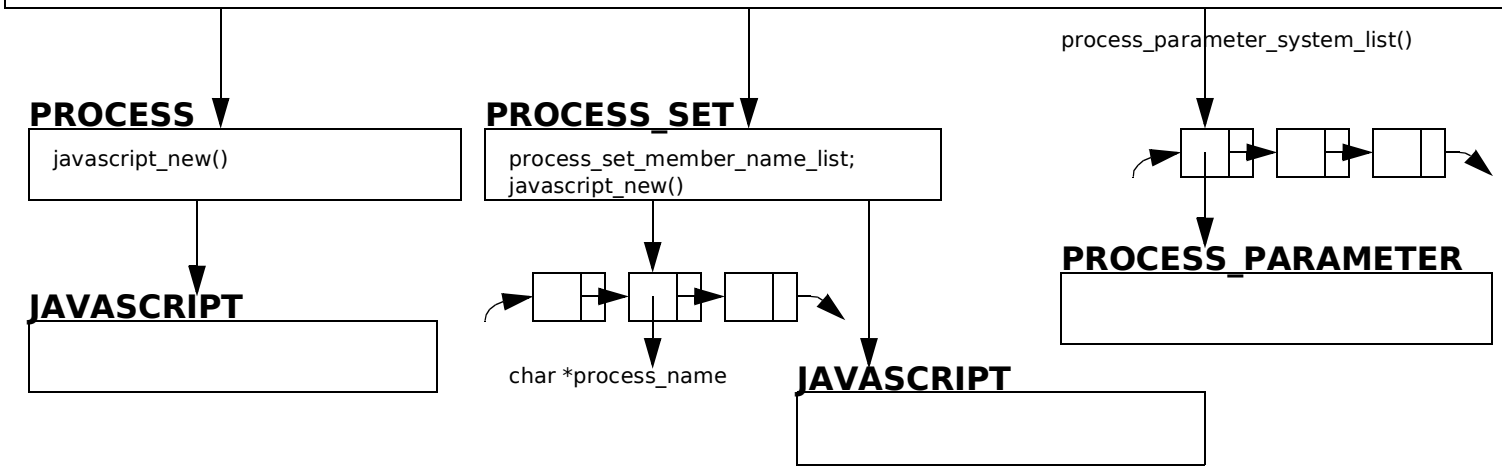
```
/* Usage */
INSERT_DATA *insert_data_new(
    char *attribute_name,
    char *security_sql_injection_escape(
        security_replace_special_characters(
            string_trim_number_characters(
                char *data,
                char *datatype_name ) ) ) );

/* Input */
char *attribute_name;
char *escaped_replaced_data;

/* External */
LIST *insert_data_relation_extract_list(
    RELATION *relation_mto1,
    LIST *folder_attribute_list,
    DICTIONARY *row_dictionary,
    int row );

INSERT_DATA *insert_data_attribute_extract(
    char *attribute_name,
    LIST *folder_attribute_list,
    DICTIONARY *row_dictionary,
    int row );

LIST *insert_data_name_list(
    LIST *insert_data_list );

LIST *insert_data_extract_list(
    LIST *insert_data_list );
```

PROCESS_PROMPT_OUTPUT

```
/* Usage */
PROCESS_PROMPT_OUTPUT *
    process_prompt_output_fetch(
        char *process_or_set_name,
        char *role_name,
        char *login_name,
        boolean is_preprompt,
        appaserver_parameter_file->
            document_root,
        application_relative_source_directory(),
        DICTIONARY *drillthru_dictionary );

/* Input */
char *process_or_set_name;
char *role_name;
char *login_name;
boolean is_preprompt;
char *document_root_directory;
relative_source_directory;
DICTIONARY *drillthru_dictionary;
```

```
/* Process */
char *process_name =
    process_name_fetch(
        process_or_set_name );

char *process_set_name =
    process_set_name_fetch(
        process_or_set_name );

PROCESS *process =
    process_fetch(
        process_name,
        document_root_directory,
        relative_source_directory,
        0 /* not check_executable_inside */ );

PROCESS_SET *process_set =
    process_set_fetch(
        process_set_name,
        role_name,
        1 /* fetch_member_process_name_list */ );
```

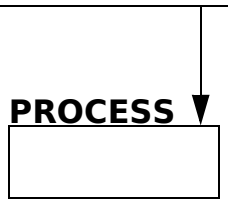
```
LIST *process_parameter_list =
    process_parameter_system_list(
        process_set_parameter_system_string(
            process_set_parameter_where(
                process_set_name,
                is_preprompt ) ),
        role_name,
        login_name,
        drillthru_dictionary );

LIST *process_parameter_list =
    process_parameter_system_list(
        process_parameter_system_string(
            process_parameter_where(
                process_name,
                is_preprompt ) ),
        role_name,
        login_name,
        drillthru_dictionary );
```



APPASERVER Data Structure

PROCESS_PROMPT_SUBMIT

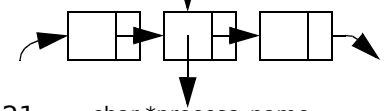
```
/* Usage */
PROCESS_PROMPT_SUBMIT *
    process_prompt_submit_fetch(
        char *process_name,
        char *role_name,
        char *login_name,
        DICTIONARY *working_post_dictionary );

/* Input */
char *process_name;
char *role_name;
char *login_name;
DICTIONARY *working_post_dictionary;

/* Process */
PROCESS *process =
    process_fetch(
        process_name,
        appaserver_parameter_file->
            document_root,
        application_relative_source_directory(),
        1 /* check_executable_inside_filesystem */ );

char *process_prompt_submit_command_line(
    char *command_line,
    char *process_name,
    char *role_name,
    char *login_name,
    DICTIONARY *working_post_dictionary );
```



PROCESS_SET

```
/* Usage */
PROCESS_SET *process_set_fetch(
    char *process_set_name,
    char *role_name,
    char *application_parameter_file->
        document_root,
    char *application_relative_source_directory(),
    boolean fetch_process_set_member_name_list );

/* Attributes */
char *notepad;
char *html_help_file_anchor;
char *post_change_javascript;
char *prompt_display_text;
char *process_group;
char *preprompt_help_text;
boolean prompt_display_bottom;
char *javascript_filename;

PROCESS_SET *process_set_parse(
    char *input,
    char *role_name,
    char *document_root_directory,
    char *application_relative_source_directory,
    boolean fetch_process_set_member_list );

char *process_set_primary_where(
    char *process_set_name );

LIST *process_set_member_name_list(
    process_set_primary_where(),
    role_name );

JAVASCRIPT *javascript =
    javascript_new(
        javascript_filename,
        document_root_directory,
        application_relative_source_directory );

/* Generic */
char *process_set_process_where(
    process_set_member_name_list() );
```

process_set_member_name_list()



char *process_name

PROCESS

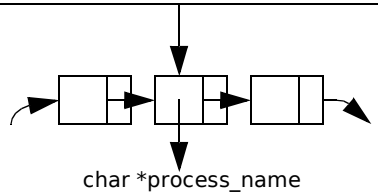
```
/* Usage */
PROCESS *process_fetch(
    char *process_name,
    char *application_parameter_file->
        document_root,
    char *application_relative_source_directory(),
    boolean check_executable_inside_filesystem );

/* Attributes */
char *command_line;
char *notepad;
char *html_help_file_anchor;
int execution_count;
char *post_change_javascript;
char *process_set_display;
char *process_group;
char *preprompt_help_text;
char *javascript_filename;

char *process_primary_where(
    char *process_name );

char *process_system_string(
    char *where );

PROCESS *process_parse(
    char *input,
    char *document_root_directory,
    char *relative_source_directory,
    boolean check_executable_inside_filesystem );

JAVASCRIPT *javascript =
    javascript_new(
        javascript_filename,
        document_root_directory,
        relative_source_directory );
```

JAVASCRIPT

```
/* Usage */
JAVASCRIPT *javascript =
    javascript_new(
        char *javascript_filename,
        appaserver_parameter_file->
            document_root,
        application_relative_source_directory() );

/* Process */
char *javascript_html(
    javascript_filename,
    document_root_directory,
    application_relative_source_directory );

char *javascript_source(
    javascript_html(),
    document_root_directory );

/* Generic */
void javascript_replace_state(
    char *post_change_javascript /* in/out */,
    char *state );

void javascript_replace_row(
    char *post_change_javascript /* in/out */,
    int row_number );

char *javascript_post_change_append(
    char *original_post_change_javascript,
    char *post_change_javascript,
    boolean place_first );

void javascript_output_cat(
    FILE *output_stream,
    javascript_source() );

void javascript_output_link(
    FILE *output_stream,
    javascript_html() );
```

PROCESS (External)

```
boolean process_prompt_exists_preprompt(
    char *process_or_set_name );

JAVASCRIPT *javascript_new(
    char *javascript_filename,
    char *appaserver_parameter_file->
        document_root,
    application_relative_source_directory() );

boolean process_executable_ok(
    char *environment_application_name(),
    char *role_name,
    char *column( command_line, 0 )
        /* executable_filename */ );

char *process_name_fetch(
    char *process_or_set_name );

char *process_set_name_fetch(
    char *process_or_set_name );

LIST *process_role_process_name_list(
    role_primary_where(
        char *role_name ) );

LIST *process_delimited_list(
    char *command_line );
```



PROCESS (Command line)

```
char *process_parameter_command_line(
    char *command_line,
    char *process_name,
    char *login_name,
    char *role_name,
    DICTIONARY *drillthru_dictionary );

char *process_choose_isa_command_line(
    char *command_line,
    char *application_name,
    char *security_entity_where,
    char *login_name,
    char *role_name,
    char *one2m_isa_folder_name );

char *process_operation_command_line(
    char *command_line,
    char *environment_application_name(),
    char *operation_name,
    char *login_name,
    char *role_name,
    char *folder_name,
    pid_t parent_process_id,
    char *session_key,
    int operation_row_total(),
    LIST *operation_row_primary_data_list(),
    DICTIONARY *operation_single_row_dictionary() );

char *process_drop_down_command_line(
    char *command_line,
    char *environment_application_name(),
    char *one2m_folder_name,
    char *state,
    DICTIONARY *drillthru_dictionary,
    DICTIONARY *working_post_dictionary );

char *process_update_row_command_line(
    char *command_line,
    DICTIONARY *drillthru_dictionary,
    char *login_name,
    char *role_name,
    char *folder_name,
    char *state,
    int row,
    char *process_name,
    char *one2m_folder_name,
    LIST *primary_data_list );

char *process_search_replace_where(
    char *command_line,
    DICTIONARY *drillthru_dictionary,
    char *login_name,
    char *role_name,
    char *folder_name );

char *process_replace_one2m_folder_name(
    char *command_line,
    char *one2m_folder_name );
```

PROCESS_PARAMETER

```
/*  Usage */
LIST *process_parameter_system_list(
    char *process_[set_]parameter_system_string(),
    char *role_name,
    char *login_name,
    DICTIONARY *drillthru_dictionary );

/* Input */
char *role_name;
char *login_name;
DICTIONARY *drillthru_dictionary;

/* Attributes */
char *process_or_set_name;
char *folder_name;
char *attribute_name;
char *drop_down_prompt_name;
char *prompt_name;

/* Process */
int display_order;
boolean drop_down_multi_select;
boolean preprompt;
char *populate_drop_down_process_name;
char *populate_helper_process_name;

char *process_parameter_system_string(
    char *where );

char *process_set_parameter_system_string(
    char *where );

PROCESS_PARAMETER *process_parameter_parse(
    char *input,
    char *role_name,
    char *login_name,
    DICTIONARY *drillthru_dictionary );

PROCESS_PARAMETER *process_parameter_new(
    char *process_or_set_name,
    char *folder_name,
    char *attribute_name,
    char *drop_down_prompt_name,
    char *prompt_name );

LIST *process_parameter_process_delimited_list(
    char *populate_drop_down_process_name,
    char *login_name,
    char *role_name,
    DICTIONARY *drillthru_dictionary );

LIST *process_parameter_folder_delimited_list(
    char *widget_folder_name,
    char *login_name,
    char *role_name,
    DICTIONARY *drillthru_dictionary );

ATTRIBUTE *attribute =
    attribute_fetch(
        attribute_name );

DROP_DOWN_PROMPT *drop_down_prompt =
    drop_down_prompt_fetch(
        drop_down_prompt_name );

PROMPT *prompt =
    prompt_fetch(
        prompt_name );

/* Generic */
char *process_parameter_where(
    char *where,
    boolean is_preprompt );
```

ATTRIBUTE

```
char *attribute_name;
```

DROP_DOWN_PROMPT

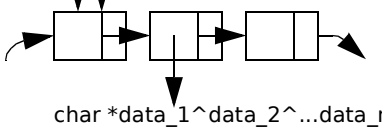
```
/* Usage */
DROP_DOWN_PROMPT *drop_down_prompt_fetch(
    char *drop_down_prompt_name );

char *drop_down_prompt_name;

/* Process */
char *hint_message;
char *optional_display;

LIST *drop_down_prompt_data_list =
    drop_down_prompt_data_system_list(
        drop_down_prompt_data_system_string(
            drop_down_prompt_primary_where(
                this->drop_down_prompt_name ) ) );
```

PROMPT

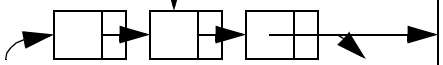
```
/* Usage */
PROMPT *prompt_fetch(
        char *prompt_name );

char *prompt_name;

/* Process */
int input_width;
char *hint_message;
boolean upload_filename;
boolean date;
```

FOLDER

```
char *folder_name;
LIST *primary_delimited_list
LIST *primary_delimited_display_list
```



char *data_1^data_2^...data_n



DROP_DOWN_PROMPT_DATA

```
char *drop_down_prompt_name;
char *drop_down_prompt_data;
int display_order;
```

PROCESS_GENERIC (1)

```
/* Usage */
PROCESS_GENERIC *process_generic_fetch(
    char *process_set_name,
    char *process_name,
    char *login_name,
    char *output_medium_string,
    DICTIONARY *post_dictionary );

/* Input */
char *process_set_name;
char *login_name;
char *output_medium_string;
DICTIONARY *post_dictionary;

/* Process */
char *process_name =
    process_generic_process_name(
        process_set_name,
        post_dictionary );

char *value_folder_name =
    process_generic_value_folder_name(
        process_name,
        process_set_name );

PROCESS_GENERIC_VALUE_FOLDER *value_folder =
    process_generic_value_folder_fetch(
        value_folder_name );

PROCESS_GENERIC_PARAMETER *parameter =
    process_generic_parameter_parse(
        output_medium_string,
        post_dictionary,
        value_folder->datatype->aggregation_sum,
        value_folder->primary_key_list,
        value_folder->date_attribute_name,
        value_folder->time_attribute_name );

char *process_generic_select(
    value_folder->primary_key_list,
    parameter->value_attribute_name );

char *process_generic_date_where(
    value_folder->date_attribute_name,
    parameter->begin_date,
    parameter->end_date );

PROCESS_GENERIC_WHERE *process_generic_where =
    process_generic_where_new(
        value_folder_name,
        value_folder->foreign_folder->foreign_key_list
            /* where_attribute_name_list */,
        value_folder->foreign_folder->foreign_attribute_data_list
            /* where_attribute_data_list */,
        value_folder->foreign_folder->folder->folder_attribute_primary_list,
        process_generic_date_where() );

char *process_generic_system_string(
    process_generic_select(),
    value_folder_name,
    process_generic_where->where_clause,
    parameter->aggregate_level,
    parameter->aggregate_statistic,
    parameter->accumulate,
    parameter->date_piece,
    parameter->time_piece,
    parameter->value_piece,
    parameter->end_date,
    value_folder->datatype->unit /* datatype_unit */,
    value_folder->foreign_folder->unit /* foreign_folder_unit */,
    parameter->units_converted );
```

PROCESS_GENERIC_VALUE_FOLDER

PROCESS_GENERIC_PARAMETER

PROCESS_GENERIC_WHERE

PROCESS_GENERIC (2)

```
char *process_generic_heading(
    value_folder->primary_key_list,
    value_folder->value_attribute_name,
    value_folder->datatype->unit /* datatype_unit */,
    value_folder->foreign_folder->unit /* foreign_folder_unit */,
    parameter->units_converted,
    parameter->aggregate_level,
    parameter->aggregate_statistic,
    parameter->accumulate );

char *process_generic_subtitle(
    value_folder_name,
    parameter->begin_date,
    parameter->end_date,
    parameter->aggregate_level,
    parameter->aggregate_statistic
    additional_message );

/* External */
char *process_generic_units_label(
    units,
    units_converted,
    enum aggregate_statistic );

enum date_convert_format process_generic_date_convert_format(
    login_name );

boolean process_generic_no_cycle_color(
    role_name );
```

PROCESS_GENERIC_PARAMETER

```
/* Attributes */
char *output_medium_string;
enum process_generic_medium output_medium;
char *begin_date;
char *end_date;
enum aggregate_level aggregate_level;
enum aggregate_statistic aggregate_statistic;
boolean accumulate;
char *email_address;
char *units_converted;
pidt process_id;
int date_piece;
int time_piece;
int value_piece;

/* Usage */
PROCESS_GENERIC_PARAMETER *process_generic_parameter_parse(
    process_generic->output_medium_string,
    process_generic_>post_dictionary,
    process_generic->value_folder->datatype->aggregation_sum,
    value_folder_primary_key_list,
    value_folder_date_attribute_name,
    value_folder_time_attribute_name );
```

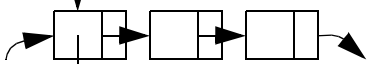
{ text_file, output_medium_stdout, table, spreadsheet, gracechart, output_medium_unknown

PROCESS_GENERIC_WHERE

```
/* Usage */
PROCESS_GENERIC_WHERE *process_generic_where_new(
    char *value_folder_name,
    LIST *where_attribute_name_list,
    LIST *where_attribute_data_list,
    LIST *folder_attribute_primary_list,
    char *date_where );

/* Process */
char *data_where =
    query_data_where(
        value_folder_name,
        where_attribute_name_list,
        where_attribute_data_list,
        folder_attribute_primary_list );

char *process_generic_where_clause(
        data_where,
        date_where );
```

PROCESS_GENERIC_VALUE_FOLDER

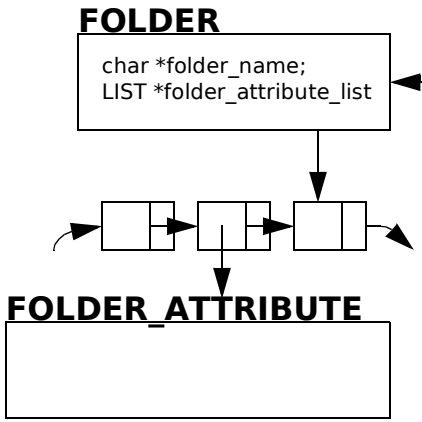
```
/* Usage */
PROCESS_GENERIC_VALUE_FOLDER *value_folder =
    process_generic_value_folder_fetch(
        process_generic->value_folder_name,
        process_generic->post_dictionary );

/* Input */
char *value_folder_name;
DICTIONARY *post_dictionary;

/* Process */
PROCESS_GENERIC_VALUE_FOLDER *value_folder_parse(
    input );

char *datatype_folder_name;
char *foreign_folder_name;
char *datatype_attribute_name;
char *date_attribute_name;
char *time_attribute_name;
char *value_attribute_name;
boolean exists_aggregation_sum;
boolean exists_bar_graph;
boolean exists_scale_graph_zero;
boolean datatype_exists_unit;
boolean foreign_exists_unit;

LIST *folder_attribute_list =
    folder_attribute_fetch_list(
        value_folder_name,
        (LIST *)0 /* exclude_attribute_name_list */ );

PROCESS_GENERIC_DATATYPE *datatype =
    process_generic_datatype_fetch(
        datatype_folder_name,
        datatype_attribute_name,
        exists_aggregation_sum,
        exists_bar_graph,
        exists_scale_graph_zero,
        datatype_exists_units,
        post_dictionary );

PROCESS_GENERIC_FOREIGN_FOLDER *foreign_folder =
    process_generic_foreign_folder_fetch(
        foreign_folder_name,
        foreign_exists_unit,
        post_dictionary );

LIST *process_generic_datatype_compare_list(
    process_generic->post_dictionary,
    exists_aggregation_sum,
    exists_bar_graph,
    exists_scale_graph_zero,
    datatype_exists_unit );
```



PROCESS_GENERIC_DATATYPE

PROCESS_GENERIC_FOREIGN_FOLDER

char *



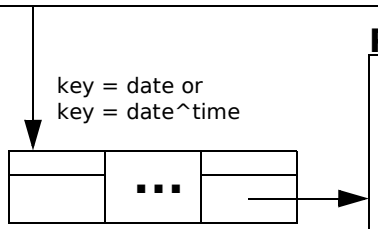
PROCESS_GENERIC_FOREIGN_FOLDER

```
/* Usage */
PROCESS_GENERIC_FOREIGN_FOLDER *process_generic_foreign_folder_fetch(
    process_generic->value_folder->foreign_folder_name,
    process_generic->post_dictionary,
    process_generic->value_folder->foreign_exists_unit );

/* Process */
FOLDER *folder =
    folder_fetch(
        foreign_folder_name,
            (char *)0 /* role_name */,
            (LIST *)0 /* exclude_attribute_name_list */,
            /* Also sets folder_attribute_primary_list */
            /* and primary_key_list. */
            1 /* fetch_folder_attribute_list */,
            0 /* not fetch_relation_mto1_non_isa_list */,
            0 /* not fetch_relation_mto1_isa_list */,
            0 /* not fetch_relation_one2m_list */,
            0 /* not fetch_relation_one2m_recursive_list */,
            0 /* not fetch_process */,
            0 /* not fetch_role_folder_list */,
            0 /* not fetch_row_level_restriction */,
            0 /* not fetch_role_operation_list */ );

LIST *foreign_key_list = folder->primary_key_list;

LIST *foreign_attribute_data_list =
    dictionary_key_list_fetch(
        foreign_key_list,
        post_dictionary );

char *unit = (foreign_exists_unit) ? list_last( foreign_attribute_data_list ) : (char *)0;
```

PROCESS_GENERIC_DATATYPE

```
/* Usage */
PROCESS_GENERIC_DATATYPE *process_generic_datatype_fetch(
    process_generic_value_folder->datatype_folder_name,
    process_generic_value_folder->datatype_attribute_name,
    process_generic_value_folder->exists_aggregation_sum,
    process_generic_value_folder->exists_bar_graph,
    process_generic_value_folder->exists_scale_graph_zero,
    process_generic_value_folder->datatype_exists_units,
    process_generic_value_folder->post_dictionary );

/* Input */
char *datatype_folder_name;
char *datatype_attribute_name;
boolean exists_aggregation_sum;
boolean exists_bar_graph;
boolean exists_scale_graph_zero;
boolean datatype_exists_units;
DICTIONARY *post_dictionary;

/* Process */
char *datatype_name;
boolean aggregation_sum;
boolean bar_graph;
boolean scale_graph_zero;
char *unit;
```

```
char *process_generic_datatype_select(
    datatype_attribute_name,
    exists_aggregation_sum,
    exists_bar_graph,
    exists_scale_graph_zero,
    datatype_exists_unit );

char *datatype_name =
    dictionary_get(
        datatype_attribute_name,
        post_dictionary );

char *process_generic_datatype_where(
    datatype_attribute_name,
    datatype_name );

char *process_generic_datatype_system_string(
    process_generic_datatype_select(),
    process_generic_datatype->datatype_folder_name,
    process_generic_datatype_where() );

PROCESS_GENERIC_DATATYPE *process_generic_datatype_parse(
    string_pipe_fetch(
        process_generic_datatype_system_string() )
            /* input_buffer */ );
```



PROCESS_GENERIC_VALUE

```
char *date;
char *time;
char *unit;
double value;
boolean is_null;
double accumulate
```

```
PROCESS_GENERIC_VALUE *process_generic_value_parse(
    double *accumulate_value,
    char *input,
    char *time_attribute,
    boolean unit_value_folder,
    boolean accumulate_boolean,
    enum aggregate_level );
```

POST_DICTIONARY

```
/* Usage */
POST_DICTIONARY *post_dictionary_stdin_new(
    /* Used when expecting a spooled file */
    char *appaserver_data_directory,
    char *session_key );

POST_DICTIONARY *post_dictionary_string_new(
    char *dictionary_string );

/* Process */
char *post_dictionary_apache_key(
    string_input() );

DICTIONARY *original_post_dictionary =
    post_dictionary_fetch(
        FILE *stdin,
        char *appaserver_data_directory,
        char *session_key,
        post_dictionary_apache_key() );

char *post_dictionary_apache_label(
    char *input );

char *post_dictionary_separate_key(
    char *input );

char *post_dictionary_input_filename(
    char *input );

void post_dictionary_stream_set(
    DICTIONARY *original_post_dictionary /* out */,
    FILE *stdin,
    post_dictionary_separate_key(),
    post_dictionary_apache_key() );

char *post_dictionary_spool_filename(
    search_replace_character(
        post_dictionary_input_filename(),
        ' ',
        '_' ),
    appaserver_data_directory,
    session_key );

void post_dictionary_spool_file(
    FILE *stdin,
    char *apache_key,
    post_dictionary_spool_filename() );
```

DICTIONARY_SEPARATE

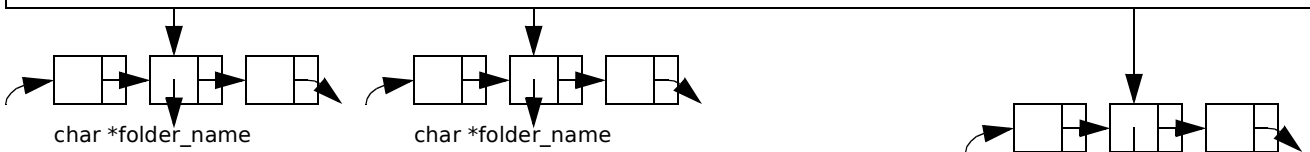
```
/* Usage */
DICTIONARY_SEPARATE *dictionary_separate_folder_new(
    DICTIONARY *original_post_dictionary,
    char *application_name,
    char *login_name,
    LIST *folder_attribute_date_name_list(
        folder->folder_attribute_list ) );

DICTIONARY_SEPARATE *dictionary_separate_row_new(
    DICTIONARY *post_dictionary_row_new()->row_dictionary
        /* original_row_dictionary */,
    char *application_name,
    char *login_name,
    LIST *folder_attribute_name_list(
        folder->folder_attribute_list ),
    LIST *folder_attribute_date_name_list(
        folder->folder_attribute_list ),
    LIST *operation_name_list );

DICTIONARY_SEPARATE *dictionary_separate_string_new(
    DICTIONARY *post_dictionary_string_new()->dictionary
        /* original_post_dictionary */ );

/* Process */
DICTIONARY *dictionary_separate_working_post(
    DICTIONARY *original_post_dictionary,
    char *application_name,
    LIST *folder_attribute_date_name_list,
    char *login_name );

DICTIONARY *dictionary_separate_sort(
    dictionary_separate_working_post() );

DICTIONARY *dictionary_separate_query(
    dictionary_separate_working_post() );

DICTIONARY *dictionary_separate_drillthru(
    dictionary_separate_working_post() );

DICTIONARY *dictionary_separate_ignore(
    dictionary_separate_working_post() );

LIST *dictionary_separate_ignore_select_attribute_name_list(
    dictionary_separate_ignore() );

DICTIONARY *dictionary_separate_pair_one2m(
    dictionary_separate_working_post() );

DICTIONARY *dictionary_separate_non_prefixed(
    dictionary_separate_working_post() );

DICTIONARY *dictionary_separate_row_zero(
    dictionary_separate_working_post(),
    folder_attribute_name_list );

void dictionary_separate_date_convert(
    DICTIONARY *working_post_dictionary,
    char *application_name,
    LIST *folder_attribute_date_name_list,
    char *login_name );

void dictionary_separate_string_date_convert(
    DICTIONARY *dictionary,
    char *application_namem,
    char *date_string,
    char *key,
    char *login_name );

DICTIONARY *dictionary_separate_row_dictionary_multi_row(
    dictionary_separate_non_prefixed(),
    folder_attribute_name_list,
    operation_name_list );

void dictionary_separate_parse_multi_attribute_keys(
    DICTIONARY *working_post_dictionary,
    char *prefix );

void dictionary_separate_trim_multi_drop_down_index(
    DICTIONARY *working_post_dictionary,
    char multi_select_move_left_right_index_delimiter );

void dictionary_separate_trim_double_bracked_data(
    DICTIONARY *working_post_dictionary );

/* Generic */
DICTIONARY *dictionary_separate_send_dictionary(
    DICTIONARY *sort_dictionary,
    DICTIONARY *query_dictionary,
    DICTIONARY *drillthru_dictionary,
    DICTIONARY *ignore_dictionary,
    DICTIONARY *pair_one2m_dictionary,
    DICTIONARY *non_prefixed_dictionary );

char *dictionary_separate_send_string(
    dictionary_separate_send_dictionary() );
```

FOLDER_MENU

```
/* Usage */
FOLDER_MENU *folder_menu_fetch(
    char *application_name,
    char *session_key,
    char *appaserver_data_directory,
    char *role_name );

/* Input */
char *application_name;
char *session_key;
char *appaserver_data_directory;
char *role_name;
```

```
/* Process */
ROLE *role =
    role_fetch(
        role_name,
        0 /* not fetch_role_attribute_exclude_list */ );

LIST *folder_menu_lookup_folder_name_list(
    char *role_name );

LIST *folder_menu_insert_folder_name_list(
    char *role_name );

if ( role->folder_count )
```

```
{
    char *folder_menu_filename(
        char *application_name,
        char *session_key,
        char *appaserver_data_directory,
        char *role_name );

    LIST *folder_menu_lookup_count_list(
        application_name,
        folder_menu_lookup_folder_name_list(),
        folder_menu_filename() );
}
```



FOLDER_MENU_COUNT

```
/* Usage */
LIST *folder_menu_lookup_count_list(
    char *application_name,
    folder_menu_lookup_folder_name_list(),
    folder_menu_filename() );

FOLDER_MENU_COUNT *folder_menu_count_new(
        char *application_name,
        char *folder_name );

/* Attributes */
```

```
char *folder_name;
unsigned long count;
char *display;

/* Process */
unsigned long folder_menu_count_fetch(
    char *folder_table_name() );

char *folder_menu_count_display(
    folder_menu_count_fetch() );
```

```
/* External */
void folder_menu_write_count_list(
    char *filename,
    folder_menu_lookup_count_list() );

LIST *folder_menu_read_count_list(
    char *filename );

FOLDER_MENU_COUNT *folder_menu_count_seek(
    char *folder_name,
    LIST *folder_menu_lookup_count_list() );
```

SESSION

```
/* Usage */
/* Sets appaserver environment and outputs argv */
/* Each parameter is security inspected */
SESSION *session_folder_integrity_exit(
    int argc,
    char **argv,
    char *application_name,
    char *login_name,
    char *session_key,
    char *folder_name,
    char *role_name,
    char *state );

/* Sets appaserver environment and outputs argv */
/* Each parameter is security inspected */
SESSION *session_process_integrity_exit(
    int argc,
    char **argv,
    char *application_name,
    char *login_name,
    char *session_key,
    char *process_name,
    char *role_name );

SESSION *session_fetch(
    /* Sets APPASERVER_DATABASE */
    char *security_sql_injection_escape(
        application_name ),
    char *security_sql_injection_escape(
        session_key ),
    char *login_name
        /* integrity_check_login_name */ );

LIST *session_system_list(
    session_system_string() );

/* Input */
char *sql_injection_escape_application_name;
char *sql_injection_escape_session_key;

/* Attributes */
char *session_key;
char *login_name;
char *login_date;
char *login_time;
char *last_access_date;
char *last_access_time;
char *http_user_agent;
char *remote_ip_address;

/* Process */
char *sql_injection_escape_role_name =
        security_sql_injection_escape(
            role_name );

char *sql_injection_escape_folder_name =
        security_sql_injection_escape(
            folder_name );

char *session_current_ip_address(
    void );

char *session_primary_where(
    char *session_key );

char *session_system_string(
    char *where );

boolean session_remote_ip_address_changed(
    session->remote_ip_address,
    session_currrent_ip_address() );

void session_message_ip_address_changed_exit(
    char *sql_injection_escape_application_name,
    char *session_key,
    char *remote_ip_address,
    char *session_current_ip_address(),
    char *login_name );

void session_access_failed_message_exit(
    char *application_name,
    char *session_current_ip_address(),
    char *login_name );

char *session_state_integrity(
    char *state,
    LIST *role_folder_fetch_list(
        sql_injection_escape_role_name,
        sql_injection_escape_folder_name ) );

void session_update_access_date_time(
    char *session_key );

void session_environment_set(
    char *application_name );
```

POST_LOGIN

```
/* Usage */
POST_LOGIN *
    prompt_edit_form_new(
        int argc,
        char **argv );

/* Process */
DICTIONARY *post_login_dictionary( void );

char *post_login_application_name(
    int argc,
    char **argv,
    post_login_dictionary() );

char *sql_injection_escape_login_name =
    security_sql_injection_escape(
        dictionary_get(
            “login_name”,
            post_login_dictionary() );

char *sql_injection_escape_password =
    security_sql_injection_escape(
        dictionary_get(
            “password”,
            post_login_dictionary() );

boolean post_login_missing_name(
    sql_injection_escape_login_name );

boolean post_login_public_name(
    sql_injection_escape_login_name );

char *post_login_database_password(
    post_login_application_name(),
    sql_injection_escape_login_name );

boolean post_login_missing_database_password(
    sql_injection_escape_password );

boolean post_login_name_email_address(
    sql_injection_esacpe_login_name );

enum post_login_password_match_return
    post_login_password_match(
        post_login_application_name(),
        post_login_name_email_address(),
        post_login_public_name(),
        sql_injection_escape_password,
        post_login_database_password() );

char *post_login_session(
    post_login_application_name(),
    sql_injection_escape_login_name );

void post_login_frameset_output(
    post_login_application_name(),
    sql_injection_escape_login_name,
    post_login_session() );

enum post_login_password_match_return
```

```
password_match,
password_fail,
public_login,
email_login
```

APPASERVER Data Structure

APPASERVER_LINK

```
/* Usage */
APPASERVER_LINK *appaserver_link_new(
    char *application_http_prefix(),
    char *appaserver_library_server_address(),
    boolean (application_prepend_http_protocol_yn() == ‘y’),
    char *document_root_directory,
    char *filename_stem,
    char *application_name,
    pid_t process_id,
    char *session_key,
    char *begin_date_string,
    char *end_date_string,
    char *extension );

/* Input */
char *http_prefix;
char *server_address;
boolean prepend_http;
char *document_root_directory;
char *filename_stem;
char *application_name;
pid_t process_id;
char *session_key;
char *begin_date_string;
char *end_date_string;
char *extension;

/* Process */
APPASERVER_LINK_PROMPT *
    appaserver_link_prompt_new(
        http_prefix,
        sever_address,
        prepend_http );

char *tail_half =
    appaserver_link_output_tail_half(
        application_name,
        filename_stem,
        begin_date_string,
        end_date_string,
        process_id,
        session_key,
        extension );

APPASERVER_LINK_OUTPUT *appaserver_link_output_new(
    document_root_directory,
    tail_half );

/* External */
char *appaserver_link_working_directory(
    char *document_root_directory,
    char *application_name );

void appaserver_link_pid_filename(
    char **prompt_filename,
    char **output_filename,
    char *application_name,
    char *document_root_directory,
    pid_t process_id,
    char *process_name,
    char *extension );
```

APPASERVER_LINK_PROMPT

APPASERVER_LINK_OUTPUT

APPASERVER_LINK_PROMPT

```
/* Usage */
APPASERVER_LINK_PROMPT *appaserver_link_prompt_new(
    char *http_prefix,
    char *server_address,
    boolean prepend_http,
    char *application_name,
    char *filename_stem,
    char *begin_date_string,
    char *end_date_string,
    pid_t process_id,
    char *session_key,
    char *extension );

/* Process */
char *appaserver_link_prompt_link_half(
    boolean prepend_http,
    char *http_prefix,
    char *server_address );

char *appaserver_link_prompt_filename(
    appaserver_link_prompt_link_half(),
    char *application_name,
    char *filename_stem,
    char *begin_date_string,
    char *end_date_string,
    pid_t process_id,
    char *session_key,
    char *extension );
```

APPASERVER_LINK_OUTPUT

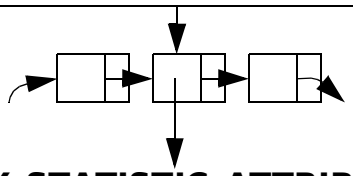
```
/* Usage */
APPASERVER_LINK_OUTPUT *
    appaserver_link_output_new(
        char *document_root_directory,
        char *appaserver_link_output_tail_half() );

/* Process */
char *appaserver_link_output_filename(
    char *document_root_directory,
    appaserver_link_output_tail_half() );

/* External */
char *appaserver_link_output_tail_half(
    char *application_name,
    char *filename_stem,
    char *begin_date_string,
    char *end_date_string,
    pid_t process_id,
    char *session_key,
    char *extension );

char *appaserver_link_output_abbreviated_filename(
    char *filename_stem,
    char *begin_date_string,
    char *end_date_string,
    pid_t process_id,
    char *session_key,
    char *extension );
```

POPULATE_HELPER_PROCESS

```
/* Usage */
POPULATE_HELPER_PROCESS *
    populate_helper_process_new(
        char *
            process_parameter->
                populate_helper_process_name ||
            process_set_parameter->
                populate_helper_process_name,
        char *document_root_directory,
        char *application_name,
        pid_t getpid() );

/* Input */
char *name;
char *document_root_directory;
char *application_name;
pid_t process_id;

/* Process */
char *populate_helper_process_output_filename(
    char *name,
    char *document_root_directory,
    char *application_name,
    pid_t process_id );

char *populate_helper_process_prompt(
    char *name,
    char *application_name,
    pid_t process_id );
```

QUERY_STATISTIC

```
/* Usage */
QUERY_STATISTIC *query_statistic_new(
    LIST *folder_attribute_name_list(
        folder_attribute_number_list(
            query->folder->folder_attribute_list ) ),
    LIST *folder_attribute_name_list(
        folder_attribute_date_list(
            query->folder->folder_attribute_list ) ),
    LIST *folder_attribute_name_list(
        folder_attribute_date_time_list(
            query->folder->folder_attribute_list ) ),
    char *query_from_clause(),
    char *query_where_clause() );

/* Process */
LIST *query_statistic_record_list(
    LIST *folder_attribute_number_name_list,
    LIST *folder_attribute_date_name_list,
    LIST *folder_attribute_date_time_name_list,
    char *from_clause,
    char *where_clause );

LIST *query_statistic_attribute_list(
    LIST *folder_attribute_number_name_list,
    LIST *folder_attribute_date_name_list,
    LIST *folder_attribute_date_time_name_list,
    LIST *query_statistic_record_list() );

char *query_statistic_html(
    char *query_from_clause(),
    char *query_where_clause(),
    query_statistic_attribute_list() );

/* Internal */
char *query_statistic_title_html(
    char *from_clause,
    char *where_clause );

char *query_statistic_heading_html( void );

char *query_statistic_select(
    LIST *folder_attribute_number_name_list,
    LIST *folder_attribute_date_name_list,
    LIST *folder_attribute_date_time_name_list );
```



QUERY_STATISTIC_ATTRIBUTE

QUERY_STATISTIC_ATTRIBUTE

```
/* Usage */
LIST *query_statistic_attribute_list(
    LIST *folder_attribute_number_name_list,
    LIST *folder_attribute_date_name_list,
    LIST *folder_attribute_date_time_name_list,
    LIST *query_statistic_record_list );

QUERY_STATISTIC_ATTRIBUTE *
    query_statistic_attribute_new(
        char *folder_attribute_name,
        boolean is_date,
        int piece_offset,
        LIST *query_statistic_record_list );

/* Process */
QUERY_STATISTIC_AGGREGATE *
    query_statistic_aggregate =
        query_statistic_aggregate_new(
            is_date,
            piece_offset,
            query_statistic_record_list );

char *query_statistic_attribute_html(
    char *folder_attribute_name,
    boolean is_date,
    QUERY_STATISTIC_AGGREGATE *
        query_statistic_aggregate );
```

QUERY_STATISTIC_AGGREGATE

```
/* Usage */
QUERY_STATISTIC_AGGREGATE *
    query_statistic_aggregate_new(
        boolean is_date,
        int piece_offset,
        LIST *record_list );

/* Attributes */
unsigned long count;
double sum;
double average;
double minimum;
double maximum;
double range;
double standard_deviation;
double coefficient_of_variation;
double standard_error;
double percent_missing;
char *begin_date;
char *end_date;

/* Process */
char *aggregate_filename =
    timlib_temp_filename();

FILE *output_pipe =
    query_statistic_aggregate_output_pipe(
        boolean is_date,
        char *aggregate_filename );

void query_statistic_aggregate_generate(
        FILE *output_pipe,
        int piece_offset,
        LIST *record_list );

pclose( output_pipe );

QUERY_STATISTIC_AGGREGATE *
    query_statistic_aggregate_read(
        boolean is_date,
        char *aggregate_filename );

timlib_remove_file( aggregate_filename );
```